

Project Problem Statement:

The main problem we identified was that vulnerability remediation was largely manual and time-consuming. Organizations may have hundreds or thousands of VMs, and scanners continuously generate vulnerabilities in different formats. Analysts had to manually triage and prioritize them, create Jira tickets, engineers had to manually research and apply fixes, and finally the fixes were manually validated.

Because there was no standardized end-to-end automation, the process was slow, inconsistent, difficult to audit, and increased the mean time to remediate. So, our project focuses on automating this complete lifecycle—from vulnerability detection and prioritization to remediation and validation.”

Solution

To solve this problem, we built an end-to-end Automated Vulnerability Management System with only one human approval gate before production.

First, the system takes vulnerability reports from multiple scanners, automatically parses, normalizes, and deduplicates them into a common format. Then, it enriches the findings using sources like NVD, EPSS, and CISA KEV and prioritizes them based on factors such as vulnerability severity, exploit likelihood, and asset criticality.

For high-priority vulnerabilities, the system automatically creates a Jira ticket, clones the target VM, and generates a remediation script. The script is tested on the cloned VM through a CI/CD pipeline. If the fix fails, a self-correction agent analyzes the error and automatically modifies and retries the script.

Once the vulnerability is successfully fixed and validated, the system captures before-and-after evidence and raises a Pull Request. **A human reviews and approves the PR, and only then is the fix deployed to production.**

So, essentially, our system automates the complete vulnerability lifecycle while keeping **human approval only at the final production stage.**”

Some Important Terms:

1. NVD — National Vulnerability Database

“NVD is a vulnerability database maintained by NIST. It provides detailed information about publicly known vulnerabilities, mainly identified using CVE IDs. It gives us information such as vulnerability description, affected products, CVSS score, severity, and references.”

Project mein: NVD se hum vulnerability ki **severity and technical details** enrich karte hain.

2. EPSS — Exploit Prediction Scoring System

“EPSS is a probability-based scoring system developed by FIRST that estimates how likely a vulnerability is to be exploited in the wild within the near future. The score ranges from 0 to 1, or 0% to 100%.”

Project mein: EPSS hume batata hai ki **kaunsi vulnerability actually exploit hone ki zyada possibility rakhti hai**, so we can prioritize it accordingly.

Example:

Agar do vulnerabilities ka CVSS high hai, but one has a much higher EPSS score, we can prioritize the one with the higher exploit likelihood.

3. CISA KEV — Known Exploited Vulnerabilities Catalog

“CISA KEV is a catalog maintained by CISA that contains vulnerabilities that are known to have been actively exploited in the real world. So, if a vulnerability is present in KEV, it is a strong indicator that it should be treated as high priority.”

Project mein: KEV hume identify karne mein help karta hai ki vulnerability **already actively exploited** ho rahi hai ya nahi.

Solution Workflow

“This is the overall workflow of our Automated Vulnerability Management System.

We start by running multiple security scanners such as Trivy, Snyk, Gypse, ZAP, Nmap, and Lynis. Their outputs can be in different formats, so our system first automatically parses them and converts them into CSV, followed by normalization into a common schema.

After normalization, we prioritize the vulnerabilities using factors like CVSS, EPSS, CISA KEV, and asset criticality. The high-priority vulnerabilities are then added to the remediation queue.

For a selected vulnerability, we clone the target VM so that we can safely test the remediation without directly affecting production. At the same time, we create a Jira ticket and a GitHub branch.

Then our remediation agent generates a remediate.sh script and pushes it to GitHub. This triggers GitHub Actions, which executes the script on the cloned VM.

If the execution fails, we capture the error logs and send them to the self-correction agent. The agent analyzes the failure, modifies the remediation script, and the process is retried automatically.

If the remediation succeeds, we validate the fix by running the scanners again on the cloned VM. We then generate before-and-after evidence and update the Jira ticket. A Pull Request is created for human review.

This is the only human approval point in our workflow. If the reviewer rejects the PR, the feedback goes back to the remediation process and the script is regenerated. If the reviewer approves it, the PR is merged, the fix is applied to the actual production VM, and a final compliance scan is performed.

Finally, the cloned VM is cleaned up and the Jira ticket is resolved.

So, the key idea is that everything from vulnerability detection to remediation and validation is automated, while human intervention is retained only at the final production approval stage.”

Technical Implementation

“This diagram shows how we implemented the backend architecture of our vulnerability management system.

The process starts from the User UI, where the user uploads the scanner CSV file. This request goes to our upload API, which stores the raw file and starts the background processing.

We have a sequential preprocessing pipeline consisting of three main LangGraph workflows: **Parsing, Normalization, and Prioritization**. Parsing extracts the vulnerability information from the scanner output, normalization converts it into our standard schema, and prioritization calculates the remediation priority.

After prioritization, the vulnerabilities are inserted into our **SQLite task queue** as pending tasks. Our background remediation engine continuously picks the next pending task and processes it through the **LangGraph remediation node**.

We designed the remediation engine as a background worker so that long-running remediation tasks don't block the API. The worker can also save and resume the agent's state, which is important because remediation may involve multiple steps, failures, retries, or human approval.

We also integrated GitHub through webhooks. Whenever a Pull Request is reviewed or merged, GitHub sends an event to our webhook API. We use this event to identify the corresponding agent and resume the appropriate workflow.

Finally, SQLite maintains three important types of information: the **vulnerability task queue**, the **workflow or agent state**, and the **PR-to-agent mapping**. This allows us to

track which vulnerability is being processed, resume interrupted workflows, and connect GitHub PRs back to the correct remediation agent.

So overall, the architecture separates the API layer, preprocessing pipeline, background remediation engine, GitHub integration, and database state management, which makes the system asynchronous, resumable, and easier to manage.”

If interviewer asks specifically “Why did you use a background worker?”

Say:

“Remediation is a long-running process and can involve script generation, execution, retries, validation, and human approval. We didn't want the API request to remain blocked during this entire process. So we moved the remediation workflow to a background worker. The API can return quickly, while the worker processes the task asynchronously and stores its state in the database.”

If they ask “Why LangGraph?”

“We used LangGraph because our remediation workflow is stateful and has multiple conditional paths. For example, if remediation fails, we need to capture the error, perform self-correction, and retry. Similarly, after creating a PR, the workflow may pause for human approval and later resume based on the GitHub webhook. LangGraph provides a structured way to manage these states, transitions, retries, and resumptions.”

Component 1 — Amazon EC2

1. Interviewer: What is Amazon EC2?

Tum bolo:

“Amazon EC2 stands for Elastic Compute Cloud. It provides virtual servers in the AWS cloud where we can run applications, scripts, and other workloads. Basically, instead of maintaining a physical server, we can create and manage a virtual machine using EC2.”

2. Interviewer: Why did you use EC2 in your project?

“We used EC2 as the execution environment for our vulnerability remediation process. Our AI agent generates remediation scripts, and we need a controlled environment where those scripts and security scanners can be executed. EC2 provides us with that virtual machine environment.”

3. Most Important: What exactly did you use EC2 for?

“In our project, EC2 is mainly used as the remediation execution environment. We execute the AI-generated remediation scripts on the target or cloned VM, run security scanners for validation, and collect the execution and validation logs. We use the cloned environment first so that we can test the remediation safely before making any changes to production.”

4. Why did you use a cloned VM?

This is **very important**.

“We don't want to directly execute an AI-generated remediation script on production because there is always a possibility that the script could fail or make an unintended change. So, we first create a clone of the target VM and execute the remediation there. We validate the fix using security scanners, and only after successful validation and human approval do we apply the fix to the production VM.”

Remember this flow:

Production VM → Clone → Remediate → Validate → Human Approval → Production

5. How does EC2 fit into your complete workflow?

“Once a vulnerability is prioritized, our system identifies the affected VM and creates a clone of it. The remediation agent generates a remediation script and the script is executed on the cloned EC2 environment. After execution, we run the scanners again to verify whether the vulnerability has been resolved. If the remediation is successful, we generate the evidence and proceed towards the Pull Request approval stage. After approval, the same fix is applied to the production VM.”

6. How do you execute commands on EC2?

Tumhare project mein important answer:

“We use AWS Systems Manager, or SSM, for remote command execution rather than directly using SSH. Our backend can send commands through SSM to the EC2 instance, and the SSM Agent running on the instance executes those commands and returns the output.”

Ye answer **EC2 + SSM** ko connect karta hai.

7. Why didn't you use SSH?

“We preferred AWS Systems Manager because it allows us to manage and execute commands on EC2 instances without exposing an SSH port or maintaining SSH keys for every instance. It also integrates with IAM, so we can control which resources and commands the automation is allowed to access.”

Short version:

“SSM gives us secure, IAM-controlled remote execution without depending on SSH access.”

8. How do you validate that the vulnerability was actually fixed?

“After executing the remediation script, we don't simply assume that the fix worked. We run the relevant security scanner again on the cloned VM and compare the before-and-after results. If the vulnerability is no longer detected, we consider the remediation successful and generate evidence for the Pull Request.”

This is a **very strong answer** because it shows you understand that:

Script execution success ≠ vulnerability fixed.

9. What happens if the remediation script fails?

“If the script fails, we capture the error logs from the execution environment. These logs are passed back to our self-correction agent. The agent analyzes the failure, modifies the remediation script, and the updated script is executed again on the cloned VM. This creates an automatic feedback loop until the remediation succeeds or the workflow reaches its failure condition.”

Flow:

Script → Fail → Logs → Self-Correction Agent → New Script → Retry

10. What happens if the vulnerability still exists even though the script executed successfully?

This is a **tricky interview question**.

“Successful script execution doesn't necessarily mean that the vulnerability has been fixed. That's why we perform post-remediation validation using the security

scanner. If the vulnerability is still detected, the remediation is considered unsuccessful and the workflow can send the result back to the remediation or self-correction stage.”

🔥 This distinction is important.

11. How did you secure your EC2 environment?

You can say:

“We followed the principle of least privilege. Access to AWS resources is controlled through IAM roles and permissions, and remote execution is handled through SSM instead of exposing unnecessary SSH access. We also isolate remediation testing from production by using a cloned environment.”

12. What is a Security Group?

“A Security Group is a virtual firewall associated with an EC2 instance. It controls inbound and outbound network traffic based on defined rules such as protocol, port, and source or destination.”

If asked:

Security Group vs NACL?

“A Security Group works at the instance level and is stateful, while a Network ACL works at the subnet level and is stateless.”

13. What is an IAM Role?

“An IAM role provides temporary permissions to AWS resources without embedding access keys directly into the application or server. In our project, we use IAM-based permissions so that EC2 and our automation components can access only the AWS resources they actually need.”

14. Why EC2 instead of Docker?

Possible cross-question.

“Our use case involves vulnerability remediation at the virtual-machine level, so we need an environment that closely represents the actual target infrastructure, including its operating system, packages, configurations, and security state. EC2

gives us that VM-level environment. Containers could be useful for isolated application workloads, but they wouldn't always accurately represent a VM-level remediation scenario.”

15. Why EC2 instead of Lambda?

“Lambda is designed for short-lived, event-driven functions, whereas our remediation workflow can involve long-running operations such as script execution, scanner execution, retries, and validation. EC2 gives us more control over the execution environment and is better suited for this workload.”

Scenario Questions

Interviewer:

“Suppose your remediation script accidentally deletes an important file. What happens?”

Answer:

“That's exactly why we don't directly execute an AI-generated script on production. The script is first executed on the cloned VM. We validate its behavior and the vulnerability status there. Only after successful validation and human approval is the change allowed to reach production.”

Interviewer:

“What if the cloned EC2 instance is unreachable?”

“The workflow would detect the execution failure through SSM. We would capture the error and mark the task accordingly rather than blindly proceeding to production. Depending on the failure, the workflow can retry or move the task into a failure state for further investigation.”

Interviewer:

“Can you explain EC2's role in one sentence?”

“EC2 provides the controlled VM environment where our remediation scripts and security scanners are executed and validated before changes are applied to production.”

Component 2 — Amazon S3

S3 ko bhi **interview perspective** se prepare karte hain. Tumhe theory se zyada ye clear hona chahiye ki **“mere project mein S3 ka actual role kya tha?”**

1. Interviewer: What is Amazon S3?

“Amazon S3 stands for Simple Storage Service. It is an object storage service provided by AWS. We can use it to store and retrieve files such as logs, reports, documents, and other artifacts. It is highly scalable and designed for durable storage.”

2. Why did you use S3 in your project?

“We used S3 as a centralized storage layer for the artifacts generated during our vulnerability remediation workflow. Instead of keeping logs and reports only on the EC2 instance, we store them centrally in S3 so they can be accessed later for auditing, validation, and compliance purposes.”

Keywords yaad rakho:

Centralized Storage → Logs → Reports → Artifacts → Audit

3. What exactly are you storing in S3?

Ye **most important question** hai.

“We use S3 to store remediation-related artifacts such as execution logs, validation reports, before-and-after evidence, and other workflow artifacts generated during the remediation process.”

Agar interviewer further pooche:

“Why are these artifacts important?”

“They provide evidence of what happened during remediation. For example, we can see what the vulnerability status was before remediation, what action was performed, and what the scanner reported after remediation. This gives us an audit trail.”

4. Why not store these logs directly in SQLite?

Very likely cross-question.

“SQLite is mainly used in our system for transactional workflow state, such as the vulnerability queue, agent state, and PR mappings. S3 is better suited for storing large and persistent artifacts like logs, reports, and evidence. So we separate workflow state from artifact storage.”

Simple distinction:

SQLite → What is the current state of the workflow?

S3 → What artifacts were generated during the workflow?

5. How does S3 fit into your workflow?

“During remediation, different artifacts are generated, such as execution logs and scanner reports. These artifacts are uploaded to our S3 bucket. After successful remediation, the before-and-after validation evidence is also stored there. This information can later be referenced for auditing and compliance.”

6. Why didn't you store everything on EC2?

“EC2 is primarily our execution environment. If we stored everything only on EC2, the data would remain tied to a particular instance and wouldn't provide a centralized storage mechanism. S3 gives us durable and centralized storage that can be accessed independently of the EC2 lifecycle.”

7. What is a Bucket?

“A bucket is a logical container in Amazon S3 where objects are stored. In our project, we use an S3 bucket to organize the remediation logs, reports, and other workflow artifacts.”

8. What is an Object in S3?

“An object is the actual piece of data stored in S3, such as a log file, report, or JSON document. Each object has data and metadata and is identified by a unique key within the bucket.”

9. Bucket vs Object

Interviewer may ask this basic question.

“A bucket is the container, while an object is the actual file or data stored inside that bucket.”

Example:

remediation-logs-bucket

```
|
|  └─ vulnerability-123/
|    └─ execution.log
|    └─ before-scan.json
|    └─ after-scan.json
|
|  └─ vulnerability-456/
|    └─ execution.log
|    └─ validation-report.json
```

10. How do you secure S3?

Important because your project deals with **security data**.

“We should keep the bucket private and use IAM-based access control so that only authorized components can access it. We also follow the principle of least privilege, meaning the application or EC2 instance gets only the permissions required to upload or retrieve the necessary artifacts.”

If implemented, you can additionally mention:

- Encryption
- Bucket policies
- Block public access
- Versioning
- Lifecycle policies

But **don't claim you implemented something if you didn't**.

11. What is S3 Versioning?

“S3 Versioning allows multiple versions of the same object to be maintained. If an object is accidentally overwritten or deleted, previous versions can potentially be recovered.”

12. What is S3 Lifecycle Policy?

“A lifecycle policy automatically manages objects based on rules. For example, older logs could be transitioned to a cheaper storage class or deleted after a defined retention period. This helps control storage costs.”

13. How does S3 help with auditing?

This is **very relevant to your project**.

“Our system generates evidence throughout the remediation process. By storing execution logs, scanner results, and before-and-after reports in S3, we maintain a persistent record of what happened during remediation. This makes the process more auditable and helps demonstrate that a vulnerability was actually fixed.”

14. S3 vs EC2 Storage

Very common interview question.

EC2 Storage	S3
Used with a compute instance	Independent object storage
Mainly for application/OS data	Files, logs, reports, artifacts
Tied to infrastructure depending on storage type	Highly durable centralized storage
Good for active processing	Good for persistent artifacts

Interview answer:

“EC2 is primarily our compute environment, whereas S3 is our centralized object storage. We use EC2 to execute remediation and S3 to persist the resulting artifacts.”

15. S3 vs Database

Another possible question:

“A database is optimized for structured, queryable application data, while S3 is optimized for storing objects such as files, logs, reports, and large artifacts. In our architecture, SQLite maintains workflow state, while S3 stores the generated artifacts.”

Scenario Questions

Interviewer:

“Suppose your EC2 instance gets terminated. What happens to the remediation logs?”

“Since we store the remediation artifacts centrally in S3 rather than keeping them only on the EC2 instance, the artifacts can remain available even if the execution instance is terminated.”

Interviewer:

“What if the same vulnerability is remediated multiple times?”

“We can organize the S3 objects using structured keys based on identifiers such as vulnerability ID, task ID, or remediation run, so that artifacts from different remediation runs can be distinguished and traced.”

Interviewer:

“What if someone makes your S3 bucket public?”

“That would be a serious security issue because vulnerability reports can contain sensitive infrastructure information. The bucket should therefore remain private, with public access blocked and access controlled through IAM and appropriate bucket policies.”

Most Important S3 Question

“Explain exactly how S3 is used in your project.”

Tum ye answer confidently bolna:

“In our project, S3 acts as the centralized storage layer for remediation artifacts. During the vulnerability remediation workflow, we generate execution logs, scanner reports, and before-and-after validation evidence. These artifacts are stored in S3

so that they are not dependent on the execution VM and can be accessed later for auditing and compliance. We use SQLite for workflow state and task management, while S3 is specifically used for persistent artifact storage.”

Component 3 — AWS Systems Manager (SSM)

Ye component **bahut important** hai because interviewer tumse directly pooch sakta hai:

“How does your system execute the remediation script on the VM?”

Aur iska answer tumhare architecture mein **AWS SSM** hai.

1. What is AWS Systems Manager?

“AWS Systems Manager is an AWS service that allows us to manage and automate operations on our EC2 instances. In our project, we mainly use its Run Command capability to execute commands and remediation scripts remotely on EC2 instances without requiring direct SSH access.”

2. Why did you use SSM?

“We used SSM because our remediation system needs to remotely execute AI-generated scripts on EC2 instances. Instead of opening SSH access and managing SSH keys, SSM allows us to execute commands through AWS infrastructure using IAM-based permissions.”

Simple:

SSM = Secure remote command execution

3. What exactly did you use SSM for?

“In our project, SSM is used as the execution bridge between our automation system and the EC2 instance. When the remediation workflow generates a script, our system uses SSM to send the required command to the target or cloned EC2 instance. The instance executes it through the SSM Agent, and we collect the output and error logs for further processing.”

4. How does SSM actually work?

This is **very important**.

Flow:

Our Backend / Remediation Agent



AWS SSM API



SSM Service



SSM Agent on EC2



Execute Command



Output / Error Logs



Backend

Interview mein:

“Our backend sends a command through the AWS Systems Manager API. The SSM service communicates with the SSM Agent running on the EC2 instance. The agent executes the command on the instance and returns the execution status and output, which our workflow can then use for validation or error handling.”

5. What is SSM Agent?

“SSM Agent is software installed on the EC2 instance that allows AWS Systems Manager to communicate with and manage the instance. It receives commands from Systems Manager, executes them locally, and reports the result back.”

Easy:

SSM Service → SSM Agent → EC2

6. What is Run Command?

“Run Command is a capability of AWS Systems Manager that allows us to remotely execute commands on managed instances without needing an interactive SSH

session. In our project, it can be used to execute the remediation-related commands on the EC2 environment.”

7. Why SSM instead of SSH?

🔥 Very likely question.

“We chose SSM over SSH because SSM provides IAM-controlled remote execution without requiring us to expose an SSH port or distribute SSH keys across the infrastructure. It also integrates naturally with AWS services and is better suited for automated infrastructure operations.”

Comparison:

SSH	SSM
Requires SSH access	No direct SSH required
Usually uses SSH keys/passwords	IAM-based permissions
Port 22 generally involved	No need to expose port 22
Manual/interactive access common	Automation-friendly
Key management required	AWS IAM controls access

8. How do you authenticate SSM?

“We use AWS IAM permissions to control access to Systems Manager. The EC2 instance also needs the appropriate IAM role so that it can communicate with AWS Systems Manager. This allows us to follow the principle of least privilege.”

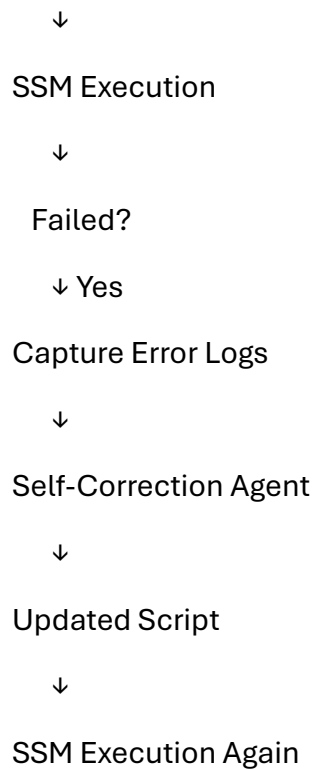
9. What happens when the remediation script fails?

This connects directly with your **self-correction agent**.

“SSM returns the execution status and output of the command. If the command fails, we capture the error logs and pass them into our remediation workflow. The self-correction agent analyzes those logs, modifies the remediation script, and the workflow retries the execution on the cloned VM.”

Flow:

Generate Script



10. How do you know whether the command succeeded?

“We use the command execution status and output returned by SSM. But an important point is that command success alone doesn't mean the vulnerability is fixed. After successful execution, we run the security scanner again and verify the vulnerability status.”

🔥 This is a **strong answer**.

Remember:

SSM success ≠ Vulnerability fixed

You need:

Command Success → Scanner Validation → Vulnerability Resolved

11. What happens if the EC2 instance is unreachable?

“If the instance is not reachable through SSM, the command execution will fail or remain unsuccessful. Our workflow captures that failure instead of proceeding to production. Depending on the failure condition, the task can be retried or marked as failed for further investigation.”

12. How does SSM fit into your overall architecture?

“SSM acts as the execution layer between our remediation engine and the EC2 environment. LangGraph manages the remediation workflow and generates the remediation action, while SSM is responsible for actually executing the required commands on the EC2 instance. After execution, the result comes back to the workflow for validation or self-correction.”

This is a **very good architecture-level answer**.

13. Why didn't you let LangGraph directly execute shell commands?

Possible tricky question.

“LangGraph is responsible for orchestrating the workflow and deciding which step should happen next. We don't want the agent itself to have unrestricted direct access to infrastructure. Instead, infrastructure operations are exposed through controlled tools or services, and SSM acts as the secure execution mechanism. This gives us better separation of responsibilities and access control.”

14. How do you make AI-generated scripts safer?

“We don't directly trust the AI-generated script and execute it on production. We first execute it on the cloned environment through SSM, validate the result using security scanners, capture the evidence, and require human approval before the change reaches production.”

This answer connects:

AI → SSM → Clone → Scanner → Evidence → Human Approval

15. SSM vs EC2 — what's the difference?

Interviewer may deliberately test this.

“EC2 provides the virtual machine where the workload runs, while SSM provides the management and remote execution mechanism. So EC2 is the execution environment, and SSM is the communication and command-execution layer.”

One line:

EC2 = Where the command runs

SSM = How we remotely send the command

Scenario Questions

Q1. “Suppose SSM executes the script successfully, but the vulnerability is still present. What happens?”

“We don't consider the remediation successful just because the command returned successfully. We run the scanner again on the cloned VM. If the vulnerability is still detected, the workflow treats the remediation as unsuccessful and can trigger the self-correction process.”

Q2. “What if SSM itself is unavailable?”

“The remediation task cannot proceed because SSM is our execution bridge. We would capture the infrastructure-level failure and prevent the workflow from moving toward production deployment. The task can then be retried or moved to a failure state.”

Q3. “What if the AI generates a malicious or destructive command?”

“That's why we don't directly execute the generated command on production. We isolate the initial execution on a cloned VM, restrict permissions through IAM, validate the result, and retain human approval before production deployment.”

Q4. “Why is SSM better for your automation?”

“Because our system is designed for automated vulnerability remediation across infrastructure. SSM provides programmatic remote command execution with AWS-native authentication and access control, making it much more suitable for automation than managing SSH access manually.”

Most Important SSM Answer

If interviewer asks:

“Explain how SSM is used in your project.”

Say:

“In our project, AWS Systems Manager acts as the remote execution layer between our remediation engine and EC2 instances. Once our remediation agent generates

a script, we use SSM to execute the required commands on the cloned EC2 environment. The SSM Agent on the instance executes the command and returns the execution status and logs. If the execution fails, those logs are passed to our self-correction workflow, which can modify and retry the remediation. After successful execution, we run the security scanner again to verify that the vulnerability is actually resolved.”

Component 4 — Jira

Tumhare project mein **Jira sirf ticketing tool nahi hai**; it is part of the **vulnerability lifecycle management and audit trail**.

1. Interviewer: What is Jira?

“Jira is a project and issue-tracking platform commonly used for managing software development and operational workflows. In our project, we use Jira to create and track vulnerability remediation tasks from identification until resolution.”

2. Why did you use Jira in your project?

★ Very important.

“We used Jira to maintain a structured record of every vulnerability that needs remediation. Instead of manually creating tickets, our system automatically creates a Jira ticket with the vulnerability details, priority, affected asset, and remediation status. This allows us to track the complete lifecycle and maintain an audit trail.”

3. When is a Jira ticket created?

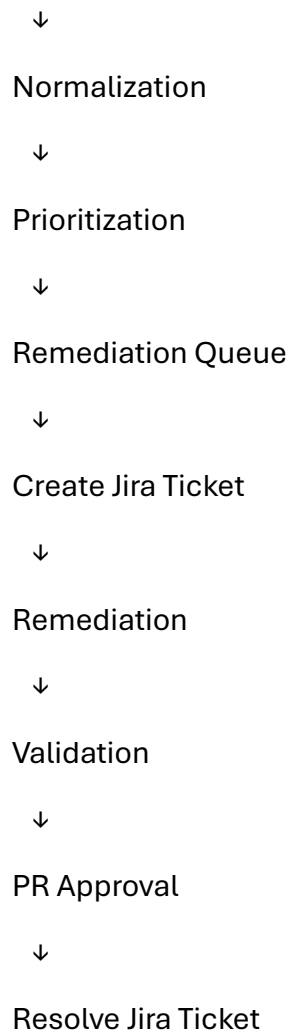
“Once a vulnerability has been normalized and prioritized and enters the remediation queue, our system creates a Jira ticket for the vulnerability before starting the remediation process. The ticket is then updated as the remediation progresses.”

Your workflow:

Scanner

↓

Parsing



4. What information do you put in the Jira ticket?

Tumhare project ke context mein:

“The Jira ticket contains the relevant vulnerability information, such as the vulnerability or CVE ID, severity, affected host or asset, affected component, priority, remediation information, and the current remediation status. We can also link the ticket with the corresponding GitHub Pull Request and remediation evidence.”

Agar interviewer bole **“Why all this information?”**

“It gives the engineer and reviewer enough context to understand what vulnerability is being addressed, why it was prioritized, what remediation was performed, and what the current status is.”

5. How does Jira integrate with your remediation workflow?

“Jira is integrated with our remediation workflow through APIs and our automation layer. When a vulnerability enters the remediation queue, the system creates a Jira issue automatically. As the workflow progresses, the issue can be updated with the remediation status and relevant information. Once the fix is successfully validated and deployed, the Jira issue is marked as resolved.”

6. What are the Jira statuses in your project?

Your overall workflow shows the important lifecycle.

You can say:

“The exact workflow depends on the remediation stage, but conceptually the Jira ticket moves from an open or pending state, through remediation and validation, and finally to resolved once the production fix and final verification are completed.”

If your actual Jira workflow has specific statuses like **Open → HITL → Resolved → Unresolved**, then say those exact statuses.

7. What happens if remediation fails?

“If remediation fails, we don't resolve the Jira ticket. The failure information can be added to the ticket, while the remediation workflow captures the error and sends it to the self-correction agent. The task is retried after generating the corrected remediation script.”

8. What happens if the Pull Request is rejected?

🔥 Important because this is your **Human-in-the-Loop** stage.

“If the reviewer rejects the Pull Request, the production deployment is not performed. The rejection or feedback is used to send the workflow back to the remediation stage, where the agent can regenerate or modify the remediation script. The Jira ticket remains unresolved until the vulnerability is successfully remediated and approved.”

9. When is the Jira ticket resolved?

“The Jira ticket is resolved only after the remediation has been successfully validated, the Pull Request has received human approval, the fix has been applied

to the production VM, and the final compliance scan confirms that the vulnerability has been addressed.”

This is a **very strong answer** because it shows:

Ticket resolution ≠ script execution success

10. Why not manually create Jira tickets?

“Manual ticket creation becomes a bottleneck when an organization has thousands of vulnerabilities. It is repetitive and can lead to inconsistent ticket formats. Our system automatically generates structured tickets, which makes the process faster and more consistent.”

11. How do you prevent duplicate Jira tickets?

★ Good technical question.

“We can use a unique combination of identifiers such as the vulnerability ID, affected asset, and remediation task to identify an existing ticket before creating a new one. This prevents the same vulnerability on the same asset from generating duplicate tickets.”

If your actual implementation uses a specific unique key, mention that.

12. How do you connect Jira with GitHub?

Your architecture has both systems.

“The remediation workflow maintains the relationship between the vulnerability, Jira ticket, and GitHub Pull Request. When the remediation script is generated, a corresponding GitHub branch and PR are created. The Jira issue can reference that PR, allowing the reviewer to move between the vulnerability record and the code or remediation change.”

13. Why do you need Jira if you already have a database?

🔥 Very likely.

“Our database is primarily used for internal workflow state and task management, while Jira provides an operational and human-facing tracking system. The database tells our automation engine what task needs to be processed and what state the

agent is in, while Jira provides visibility, approvals, status tracking, and an audit trail for the vulnerability lifecycle.”

Remember:

SQLite → Internal automation state

Jira → Human-facing vulnerability lifecycle

14. What is Jira API?

“Jira provides APIs that allow external applications to interact with Jira programmatically. In our project, the automation layer can use these APIs to create issues, update issue information, add comments, and change the status of a vulnerability ticket without requiring a person to perform those actions manually.”

15. How did you authenticate with Jira?

General answer:

“The application authenticates with Jira using configured credentials or API-based authentication, and those credentials are kept outside the source code using environment variables or a secure secrets mechanism.”

⚠ Don't say a specific authentication mechanism unless you actually implemented it.

16. What if Jira API is down?

🔥 Scenario question.

“The remediation workflow should not incorrectly mark the vulnerability as resolved if Jira is unavailable. The Jira operation can be retried, and the workflow state is maintained separately so that the task can resume the Jira update later. This separation prevents a temporary Jira failure from losing the remediation state.”

This connects nicely with your **SQLite workflow state**.

17. What if remediation succeeds but Jira update fails?

Very good interviewer question.

“The remediation result and Jira update are treated as separate operations. If the remediation succeeds but the Jira update fails, we preserve the workflow state and retry the Jira update rather than running the remediation again unnecessarily. The ticket should only be marked resolved once the required Jira update is successfully completed.”

18. How does Jira help with auditability?

“Jira gives us a human-readable record of the vulnerability lifecycle. We can see when the vulnerability was identified, what priority it received, what remediation was performed, whether human approval was given, and when it was finally resolved. Combined with the evidence stored in S3, this provides a stronger audit trail.”

Important Scenario

Interviewer:

“Suppose the scanner says the vulnerability is fixed, but Jira still shows Open. What would you do?”

“I would treat them as separate states. The technical validation may already be successful, but the Jira state has not been synchronized yet. The system should update Jira or retry the failed Jira API operation. We should not manually mark it as resolved without ensuring that the required workflow steps have completed.”

Very Important: Explain Jira's role in one answer

If interviewer asks:

“Explain how Jira is used in your project.”

Say:

“In our project, Jira acts as the vulnerability lifecycle and tracking layer. Once a vulnerability is prioritized for remediation, our system automatically creates a structured Jira ticket containing the vulnerability and asset information. As remediation progresses, the ticket is updated with the relevant status and information. The ticket is linked with the corresponding remediation workflow and Pull Request. We only resolve the Jira ticket after the fix has been validated, approved through the human-in-the-loop stage, deployed to production, and finally verified through a compliance scan.”

1. “What is GitHub Actions?”

Tum bolo:

“GitHub Actions is a CI/CD and workflow automation platform provided by GitHub. In our project, we used it to automate the remediation execution pipeline and to connect our GitHub repository with AWS services.”

Tumhare documentation mein specifically GitHub Actions ko **workflow execution, CI/CD automation, AWS integration aur workflow triggering** ke liye use kiya gaya hai.

2. Tumhare project mein iska exact role kya hai?

Interviewer: **“Why did you use GitHub Actions in your project?”**

“We used GitHub Actions as the automation layer between our remediation code repository and AWS infrastructure. Once the remediation workflow reaches the execution stage, GitHub Actions authenticates with AWS and triggers the required AWS Systems Manager command on the EC2 environment. The remediation script is executed there, and the execution logs are stored in S3 for further analysis.”

Tumhare documented end-to-end flow mein:

GitHub Actions → AWS authentication → SSM → EC2 → remediation script → S3 logs → AI analysis.

3. Complete flow explain karo

Agar interviewer bole:

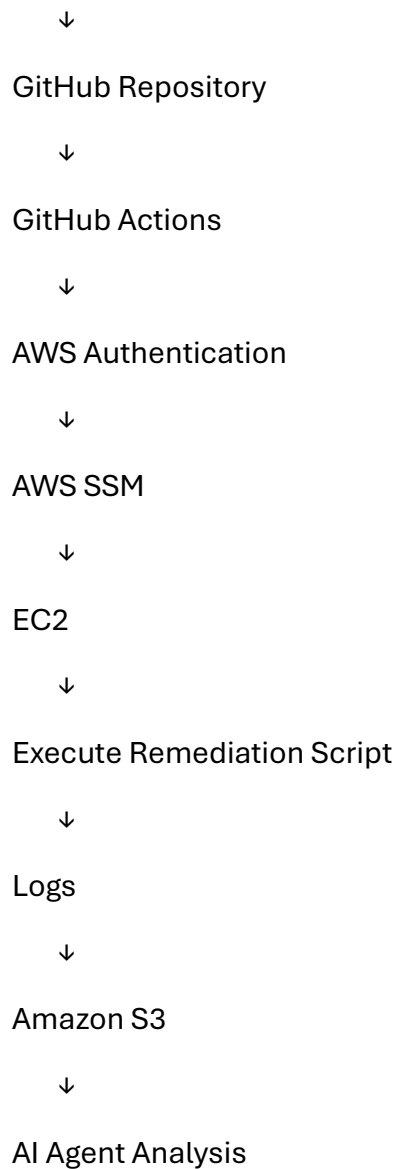
“Can you explain how GitHub Actions works in your project?”

Ye answer yaad karo:

“Our remediation agent generates the remediation script and manages it through GitHub. GitHub Actions acts as the execution automation layer. It authenticates with AWS using securely stored GitHub Secrets, sends the required command through AWS Systems Manager, and the command is executed on the EC2 environment. The execution output and logs are then stored in Amazon S3. If the execution or validation fails, those logs are used by the AI agent for further analysis and self-correction.”

Simple flow:

Remediation Agent



4. GitHub Actions AWS se kaise communicate karta hai?

Interviewer: **“How does GitHub Actions communicate with AWS?”**

“We configured AWS credentials as GitHub Secrets. GitHub Actions uses these credentials to authenticate with AWS and invoke the required AWS services, particularly Systems Manager for remote command execution.”

Documentation mein configured secrets include:

- AWS_ACCESS_KEY_ID
- AWS_SECRET_ACCESS_KEY
- AWS_REGION
- EC2_INSTANCE_ID

- REMEDIATION_LOGS_S3_BUCKET
-

5. Why GitHub Secrets?

Interviewer: **“Why didn't you put AWS credentials directly in the workflow?”**

“We don't hardcode credentials in the workflow or source code. We store them as GitHub Secrets so that sensitive AWS credentials are separated from the application code and can be securely injected into the workflow when required.”

6. GitHub Actions aur SSM mein difference?

 **Very important.**

Interviewer: **“If you already have GitHub Actions, why do you need AWS SSM?”**

Tum bolo:

“GitHub Actions is responsible for workflow automation, whereas AWS Systems Manager is responsible for securely executing commands on the EC2 instance. So GitHub Actions decides and triggers the execution, while SSM acts as the remote execution bridge to the EC2 environment.”

Tumhare documentation mein exactly SSM ko **execution bridge between automation workflows and remediation infrastructure** describe kiya gaya hai.

Yaad rakho:

GitHub Actions → Automation

SSM → Remote execution

EC2 → Actual execution environment

S3 → Logs/evidence storage

7. SSM kyun use kiya instead of SSH?

Interviewer: **“Why didn't you simply use SSH from GitHub Actions to EC2?”**

“We used AWS Systems Manager because it provides remote command execution without requiring direct SSH access. This reduces the need to expose SSH for day-to-day automation and gives us a more controlled AWS-native execution mechanism.”

Documentation specifically says that after validating SSM, SSH was no longer required for day-to-day operations and remote execution should be performed through SSM.

8. GitHub Actions failure ho jaye toh?

Interviewer: “What happens if remediation execution fails?”

“The execution logs are stored in S3. If a remediation stage fails, the workflow retrieves those logs and provides them as feedback to the LLM. The agent analyzes the failure, generates an improved remediation script, updates the Pull Request, and another validation cycle is triggered. This creates our self-healing remediation loop.”

Ye tumhare documentation ka **actual workflow** hai.

Flow:

GitHub Actions

↓

Execute remediation

↓

Failure

↓

Logs → S3

↓

AI / LLM Analysis

↓

Improved Script

↓

Update PR

↓

Retry Validation

9. GitHub Actions aur Human Approval ka relation?

Interviewer: **“Does GitHub Actions directly deploy the fix to production?”**

Best answer:

“No, not directly after generation. The remediation first has to successfully pass validation in the sandboxed environment. Then the workflow waits for Human-in-the-Loop approval. If the Pull Request is approved and merged, the validated script is executed on the original target VM. So automation handles the technical workflow, but the final production decision remains with the human reviewer.”

Documentation confirms that after successful sandbox validation the workflow enters `WAITING_FOR_HUMAN_APPROVAL`; after PR approval and merge, the validated script is executed on the original target VM.

10. GitHub Webhook ka role kya hai?

Ye **GitHub Actions se separate but related** concept hai.

Interviewer: **“How does the system know that the human has approved or commented on the PR?”**

“We implemented an event-driven HITL workflow using GitHub Webhooks. Instead of continuously polling GitHub, our FastAPI backend receives events such as pull request reviews, pull requests, and issue comments. Based on the event, the paused workflow is resumed.”

Important distinction:

GitHub Actions

→ workflow automation

GitHub Webhook

→ GitHub se backend ko event notification

MCP GitHub Server

→ AI agent ko GitHub operations perform karne ki capability

Ye teen ko mix **mat karna**.

11. GitHub Actions vs GitHub Webhook

GitHub Actions

Automation execute karta hai

GitHub Webhook

Event notify karta hai

GitHub Actions

Workflow run karta hai

AWS integration

Remediation execution pipeline PR review/comment detection

Tumhare project mein webhook ka purpose specifically **agent ko human feedback ke baad resume karna** hai.

GitHub Webhook

Backend ko event bhejta hai

HITL event handling

12. GitHub Actions vs MCP

🔥 Interview mein definitely aa sakta hai.

GitHub Actions

“GitHub Actions is our automation and CI/CD mechanism.”

MCP

“MCP provides a standardized interface through which our AI agents can interact with external tools and enterprise systems.”

Tumhare documentation mein MCP ka role tool integration, context sharing aur action execution hai.

So:

AI Agent

↓

MCP

↓

GitHub Operations

GitHub

↓

GitHub Actions

↓

Automated Workflow

13. Most important interview answer ★

Agar interviewer sirf bole:

“Explain GitHub Actions in your project.”

Directly ye bolo:

“In our Zero-Touch Vulnerability Remediation Platform, GitHub Actions acts as the CI/CD and workflow automation layer between our GitHub repository and AWS infrastructure. Once the remediation workflow reaches the execution stage, GitHub Actions authenticates with AWS using securely stored GitHub Secrets and triggers AWS Systems Manager to execute the remediation script on the EC2 environment. The execution logs are stored in Amazon S3, and if the remediation fails, those logs are provided to the AI agent for analysis and self-correction. After successful validation, the workflow pauses for Human-in-the-Loop approval, and only after the Pull Request is approved and merged is the validated remediation executed on the original target VM.”

Component 6 — MCP (Model Context Protocol)

1. What is MCP?

Interviewer: **“What is MCP?”**

Tum bolo:

“MCP stands for Model Context Protocol. It is a standardized way for AI agents to interact with external tools and systems. Instead of building a separate integration mechanism for every tool, MCP provides a common interface through which an agent can discover and use available tools.”

Simple language:

MCP = AI Agent aur external tools ke beech standardized communication layer.

2. Why did you use MCP in your project?

★ Very important.

“We used MCP because our remediation agents need to interact with external systems such as GitHub and other enterprise tools. MCP provides a standardized interface for tool integration and action execution, so the agent can perform operations without tightly coupling the agent logic with every individual API.”

Tumhare documentation mein MCP specifically **tool integration, context sharing and action execution** ke liye listed hai.

3. MCP ka actual role tumhare project mein kya tha?

Tumhare documentation mein **GitHub MCP Server** explicitly configured hai.

“In our project, MCP is used to allow the AI agent to interact with GitHub repositories. We configured a GitHub MCP Server and authenticated it using a fine-grained GitHub Personal Access Token. This allows the agent to perform the required repository operations through the MCP interface.”

Documentation confirms GitHub MCP Server ko repository access ke liye configure kiya gaya tha.

4. GitHub MCP Server kya karta hai?

“The GitHub MCP Server exposes GitHub operations as tools that the AI agent can use. Through the configured repository permissions, the agent can interact with repository contents, Pull Requests, Actions, workflows, and related GitHub operations.”

Tumhare documentation mein GitHub MCP token ko specifically in permissions ke saath configure kiya gaya:

- Actions
 - Commit Statuses
 - Contents
 - Pull Requests
 - Workflows
 - Metadata
-

5. MCP mein Client, Server aur Tool kya hote hain?

Ye **basic MCP interview question** hai.

MCP Client

“The MCP client is the component that connects the AI application or agent to an MCP server and requests the tools it needs.”

MCP Server

“The MCP server exposes a set of tools or capabilities from an external system in a standardized MCP interface.”

Tool

“A tool is a specific operation exposed by the MCP server that the agent can invoke.”

Tumhare project ke context mein:

AI / Remediation Agent

↓

MCP Client

↓

GitHub MCP Server

↓

GitHub Tools

↓

GitHub

6. MCP vs REST API

🔥 **Very likely question.**

Interviewer:

“If GitHub already provides APIs, why did you need MCP?”

Tum bolo:

“A REST API directly exposes the API endpoints of a particular service, whereas MCP provides a standardized tool interface for AI agents. In our case, instead of making the agent tightly dependent on GitHub-specific API details, MCP exposes the required GitHub operations as tools that the agent can discover and invoke.”

Easy difference:

REST API

Application → GitHub API

MCP

AI Agent → MCP → Tool → GitHub

7. MCP vs Function Calling

Possible advanced question.

“Function calling is a model capability where an LLM can request a function to be executed. MCP is an interoperability protocol that standardizes how tools and external systems can be exposed to and accessed by AI applications. So function calling can be part of an agent's tool-use mechanism, while MCP provides a standardized tool integration layer.”

Short mein:

Function Calling = Model ko tool call karne ki capability

MCP = Tools ko standardized way mein expose/connect karne ka protocol

8. How did you authenticate GitHub MCP?

Very important because documentation has exact implementation.

“We created a fine-grained GitHub Personal Access Token and restricted repository access to the required repository. We granted only the required permissions, such as Actions, Contents, Pull Requests and Workflows. The token was stored as the GITHUB_MCP_TOKEN environment variable instead of hardcoding it.”

9. Why Fine-Grained PAT?

“We used a fine-grained Personal Access Token so that access could be restricted to the required repository and specific permissions. This follows the principle of least privilege instead of giving the agent unrestricted access to all repositories.”

Documentation says repository access was limited to **only selected repositories** and specific permissions were granted.

10. What GitHub operations can your MCP integration perform?

According to your documentation, the configured permissions include:

“The MCP GitHub integration has permissions for repository contents, Pull Requests, Actions, commit statuses and workflows, along with metadata access.”

Don't overclaim a specific operation unless you actually used it.

For interview:

“The MCP integration provides the agent with the GitHub capabilities required by our remediation workflow, particularly around repository contents, Pull Requests, Actions and workflows.”

11. Why not directly integrate GitHub SDK/API into the agent?

“Direct API integration would tightly couple our agent logic with the GitHub-specific API implementation. MCP provides a standardized tool interface, which makes the agent architecture more modular and makes it easier to integrate additional external systems using the same tool-oriented approach.”

12. How does MCP fit into your LangGraph architecture?

 Important.

Tumhare project mein **LangGraph workflow orchestration karta hai**, while MCP provides external tool access.

“LangGraph manages the state and flow of our remediation workflow, while MCP provides the interface through which the agent can interact with external tools such as GitHub. So LangGraph decides which step of the workflow should execute, and MCP can provide the external tool required for that step.”

Tumhare documented stack mein LangGraph specifically **workflow state and multi-step decision making** handle karta hai.

Simple:

LangGraph

↓

Workflow Decision

↓

Agent needs GitHub operation

↓

MCP

↓

GitHub MCP Server

↓

GitHub

13. MCP aur GitHub Webhook mein difference?

Ye confuse mat karna.

MCP:

“MCP allows the AI agent to perform or access external operations through tools.”

Webhook:

“Webhook allows GitHub to send an event back to our backend when something happens, such as a Pull Request review or comment.”

Tumhare project mein webhook specifically **HITL workflow resume** karne ke liye use hua.

Flow:

MCP:

Agent → GitHub

Webhook:

GitHub → Backend

🔥 Ye difference interviewer ko explain karna aana chahiye.

14. What happens during Human-in-the-Loop?

Interviewer:

“How does MCP relate to your PR approval workflow?”

Carefully answer:

“MCP provides the agent with the GitHub integration required to work with the repository and Pull Request workflow. Once the remediation is validated, the

workflow waits for human review. The actual human feedback is received through GitHub Webhooks, which notify our FastAPI backend and allow the paused workflow to resume.”

So:

MCP → Agent ↔ GitHub interaction

Webhook → GitHub → Backend event

Documentation confirms this event-driven HITL design.

15. What happens if MCP/GitHub operation fails?

“The operation failure should be captured by the workflow rather than allowing the remediation process to incorrectly proceed. Since our workflow maintains state, the task can be kept in its current state and retried or marked as failed depending on the error.”

Your architecture uses SQLite as a LangGraph checkpoint for active workflows, which supports preserving workflow state.

Most Important Question

“Explain how you used MCP in your project.”

Ye answer prepare karo:

“In our project, we used Model Context Protocol to provide a standardized tool interface between our AI agents and external systems. One of the main integrations was the GitHub MCP Server. We configured a fine-grained GitHub Personal Access Token with the required repository permissions, and the token was stored securely as an environment variable. Through the MCP integration, our agent can interact with the GitHub repository and Pull Request-related operations required by the remediation workflow. LangGraph manages the overall workflow and state, while MCP provides the external tool integration layer.”

Component 7 — Tavily Search API

1. What is Tavily?

Interviewer: “What is Tavily?”

Tum bolo:

“Tavily is a search API designed for AI applications and agents. It allows an AI agent to perform web searches and retrieve relevant information that can be used as context for reasoning and decision-making.”

2. Why did you use Tavily in your project?

★ Very important.

“We used Tavily because vulnerability remediation sometimes requires additional information beyond the scanner output. Our remediation agent performs automated research using web search to find trusted security advisories, vendor documentation, CVE references, patch information, and remediation guidance. This additional context helps the LLM understand the vulnerability and generate an appropriate remediation approach.”

Your documentation explicitly describes this research stage before script generation.

3. What exactly does Tavily search for?

Interviewer: **“What information are you retrieving through Tavily?”**

“The agent uses web search to gather information related to the vulnerability, such as security advisories, vendor documentation, CVE references, patch notes, root-cause information, affected versions, and possible remediation approaches.”

4. Explain Tavily's role in the remediation workflow.

This is the **most important answer**.

“After a vulnerability is selected for remediation, the agent first researches the vulnerability using web search. Tavily helps retrieve relevant security advisories, vendor documentation, CVE references, and patch information. This information is combined with the original vulnerability data and then provided to the LLM. Based on this context, the LLM generates the precheck, remediation, and validation scripts.”

Flow:

Vulnerability

↓

Research

↓

Tavily Search

↓

Security Advisories

Vendor Docs

CVE References

Patch Notes

↓

LLM

↓

Precheck Script

Remediation Script

Validation Script

5. Why not rely only on NVD?

🔥 **Very likely question.**

Tumhare project mein NVD aur other intelligence sources are used for **structured vulnerability intelligence**, while Tavily is used for **additional research/remediation context**.

Answer:

“NVD provides structured vulnerability information, but remediation may require more contextual information such as vendor-specific fixes, patch notes, configuration guidance, or detailed remediation references. Tavily allows the agent to perform targeted web research and collect that additional context before generating the remediation.”

Your documentation specifically describes the web research step as gathering vendor documentation, advisories, CVE references and patch notes.

6. Tavily vs NVD

Remember this distinction:

NVD

Tavily

Structured vulnerability database Web search API

CVE information

Broader research

CVSS/details

Vendor docs/advisories

Affected products

Patch/remediation references

Structured data

Search results/context

Interview answer:

“NVD gives us structured vulnerability intelligence, whereas Tavily helps the agent perform broader web research to find remediation-specific context.”

7. Tavily vs Google Search

Possible interviewer question:

“Why did you use Tavily instead of Google?”

“Our requirement was not simply to provide a search engine to a user. We needed search results that could be consumed programmatically by an AI agent as part of an automated reasoning workflow. Tavily provides an API-oriented search mechanism designed for AI applications, which fits better with our agentic workflow.”

⚠ This is a conceptual answer. Don't claim specific Tavily features that you didn't actually implement.

8. How does the LLM use Tavily's results?

“The search results provide additional context about the vulnerability. The LLM combines this research with the original vulnerability information and uses it to understand the root cause, affected versions and possible remediation approaches. It then generates the required remediation scripts.”

Your documentation explicitly says the LLM uses **original vulnerability information + collected research** to generate the scripts.

9. What scripts does the LLM generate after research?

🔥 Good follow-up.

Your project generates **three scripts**:

1. Precheck Script

Collects the current system state before making changes.

2. Remediation Script

Applies the security patch or configuration change.

3. Validation Script

Confirms whether the vulnerability has actually been remediated.

Interview answer:

“The research isn't directly used to execute a fix. It first provides context to the LLM, which generates three scripts: a precheck script, a remediation script, and a validation script.”

10. What if Tavily doesn't return useful information?

“The system shouldn't blindly generate a remediation based on incomplete search results. The agent can rely on the available vulnerability intelligence and other research sources, and the resulting remediation still goes through sandbox execution and validation before production. So the generated script is not trusted solely because the search succeeded.”

This answer is important because **Tavily is a research source, not the final authority.**

11. What if Tavily gives incorrect information?

🔥 Security-focused interviewer may ask this.

“We don't treat web search results as automatically trustworthy instructions. The information is used as additional context for the LLM. The generated remediation is then tested in a sandboxed environment and validated using security scanners before it can reach production. Human approval is also required before production deployment.”

Your documented workflow confirms sandbox validation followed by HITL approval.

12. What was one challenge you faced with Tavily?

🔥 **Very important because this is documented in your project.**

Documentation says:

“The LLM got stuck in a 3-minute tool-calling loop, spiking costs and latency, requiring intensive prompt engineering to minimize Tavily API calls.”

Interview-ready answer:

“One challenge we faced was excessive tool calling. In one case, the LLM got stuck in a tool-calling loop for around three minutes, which increased both latency and API cost. We addressed this through prompt engineering and by reducing unnecessary Tavily calls.”

🔥 This is a **real project challenge**, so it sounds much stronger than a generic answer.

13. Why is reducing Tavily calls important?

“Because every unnecessary external search adds latency and potentially increases API cost. In an automated remediation pipeline, excessive tool calls can significantly slow down the workflow. So we optimized the agent's prompts and tool usage to make searches more targeted.”

14. Does Tavily directly generate the remediation script?

✗ **No.**

This is important.

“No. Tavily only provides external research and context. The LLM uses that information to reason about the vulnerability and generate the remediation scripts.”

Remember:

Tavily → Research

LLM → Reasoning + Script Generation

EC2 → Execution

Scanner → Validation

15. Tavily vs OpenAI

Very common cross-question.

“They have different responsibilities. Tavily provides external information through web search, while OpenAI provides the language model capabilities used for reasoning, vulnerability analysis, remediation generation, and log analysis.”

Your component documentation describes OpenAI as powering vulnerability analysis, remediation generation, root-cause identification and log analysis.

Simple:

Tavily



Information

OpenAI



Reasoning

Together



Remediation Decision

 Most Important Question

“Explain why Tavily is used in your project.”

Directly ye bolo:

“We use Tavily as an external research source during the remediation stage. When a vulnerability needs to be remediated, the agent performs targeted web research to gather security advisories, vendor documentation, CVE references, patch notes, and remediation guidance. This information is combined with the original vulnerability data and provided to the LLM. The LLM then uses this context to understand the vulnerability and generate the precheck, remediation, and validation scripts. So Tavily provides the external knowledge, while the LLM performs the reasoning and script generation.”

Technology 1 — Python

Interviewer: Why did you use Python?

Tum bolo:

“We used Python as the primary programming language because most of our AI, vulnerability analysis, automation, and remediation logic is implemented in Python. It also has a strong ecosystem for LLM integration, workflow orchestration, security tooling, API development, and data processing, which made it suitable for our project.”

Interviewer: What exactly did you implement using Python?

“Python is used across the backend and automation layer of our project. We use it for vulnerability processing, AI-agent logic, remediation workflows, database interaction, API development through FastAPI, and integration with security and cloud services.”

Interviewer: Why Python instead of JavaScript/Node.js?

“Our project is heavily focused on AI agents, vulnerability analysis, automation, and security tooling. Python provides a strong ecosystem for these areas and integrates naturally with frameworks such as LangChain and LangGraph. So Python was a better fit for our backend and agentic workloads.”

Interviewer: What are the advantages of Python for this project?

“The main advantages are its rich ecosystem, ease of integration with AI and security tools, rapid development, and good support for automation and data processing. It allowed us to integrate LLMs, databases, AWS services, GitHub, Jira, and security scanners within the same backend ecosystem.”

Interviewer: Is Python only used for AI?

“No. In our project Python is not limited to AI. It is also used for backend APIs, database operations, vulnerability processing, automation, remediation logic, and integration with external services.”

 **Most Important Python Answer**

“Explain Python's role in your project.”

“Python is the primary programming language of our platform. We use it to implement the backend, vulnerability processing, AI-agent logic, remediation workflows, database interactions, and integrations with external services. Since our project combines AI, security automation, and infrastructure operations, Python provided a suitable ecosystem to bring these different components together.”

Python — Remember This

Python = Core implementation language

And:

Python

- |— FastAPI → Backend APIs
- |— LangChain → LLM + Tools
- |— LangGraph → Agent Workflows
- |— PostgreSQL/SQLite → Database
- |— AWS/GitHub/Jira → Integrations
- |— Security Tools → Vulnerability Processing

One tricky question

“If LangChain and LangGraph are doing the AI work, why do you need Python?”

“LangChain and LangGraph are frameworks used within our Python application. They provide abstractions for LLM integration and workflow orchestration, but Python is the underlying programming language in which we implement the application logic and integrate these frameworks with our databases, APIs, and external services.”

Technology 2 — FastAPI

Tumhare project ke according **FastAPI backend layer** hai. Ye frontend, AI agents aur databases ke beech communication handle karta hai.

1. What is FastAPI?

Interviewer: “What is FastAPI?”

“FastAPI is a modern Python web framework used for building APIs. In our project, we use it as the backend framework to expose API endpoints and connect the frontend dashboard with our AI agents, databases, and other system components.”

Tumhari technology-stack documentation bhi FastAPI ko **backend framework for API requests and communication between frontend, AI agents, and databases** define karti hai.

2. Why did you use FastAPI?

★ Important.

“We chose FastAPI because our entire backend and AI workflow is primarily implemented in Python. It provides a lightweight and fast way to expose APIs and integrate our frontend, AI agents, databases, and external services within the same Python ecosystem.”

3. What exactly does FastAPI do in your project?

“FastAPI acts as the communication layer of our platform. It receives requests from the frontend, handles file uploads and webhook events, interacts with the backend services and databases, and provides the required data back to the frontend. It also acts as the entry point for events coming from external systems such as GitHub.”

Tumhare architecture mein important endpoints include:

POST /api/v1/upload

POST /github-webhook

The first handles scanner file uploads, while the webhook endpoint receives GitHub events.

4. Explain FastAPI's role in the complete architecture.

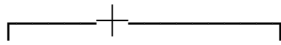
“The frontend dashboard communicates with our FastAPI backend through APIs. When a scanner CSV is uploaded, FastAPI receives the file and starts the preprocessing pipeline. It also receives GitHub webhook events related to Pull Requests and human reviews. From the backend, the appropriate workflow or agent is triggered, while databases maintain the required workflow and vulnerability state.”

Simple architecture:

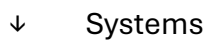
Next.js Dashboard



FastAPI



Agents Database External



PostgreSQL

SQLite

5. Why FastAPI instead of Flask?

🔥 Common question.

“Both FastAPI and Flask can be used to build APIs, but FastAPI provides strong support for asynchronous programming, automatic request validation through type hints, and automatic API documentation. Since our system involves API requests, webhooks, background workflows, and AI-agent operations, FastAPI was a good fit for our backend.”

6. What is an API endpoint?

“An API endpoint is a specific URL through which a client can communicate with a backend service for a particular operation. For example, our upload endpoint receives scanner files, while our GitHub webhook endpoint receives events from GitHub.”

Example:

POST /api/v1/upload

means:

Client → FastAPI → Upload scanner file

And:

POST /github-webhook

means:

GitHub → FastAPI → Process GitHub event

7. Why POST for file upload?

“We use POST because the client is sending data to the server and the operation results in processing or creating a new resource. In our case, the scanner output file is uploaded to the backend for processing.”

8. What is asynchronous programming in FastAPI?

Possible technical question:

“Asynchronous programming allows the application to handle operations without unnecessarily blocking the execution thread while waiting for I/O operations such as network requests or database operations. This is useful for a system like ours because it communicates with multiple external services and handles long-running workflows.”

⚠️ But don't say that **your entire remediation workflow runs inside the API request**. Your architecture separates the long-running background remediation process.

9. Why shouldn't the FastAPI request wait for the entire remediation?

🔥 Very important.

“Vulnerability remediation can be a long-running process involving script generation, execution, retries, validation, and human approval. If the API request waited for the complete workflow, it would unnecessarily block the request. Therefore, FastAPI handles the request and triggers or communicates with the background workflow, while the remediation process runs independently.”

10. How does FastAPI handle GitHub Webhooks?

★ Important for your project.

“GitHub sends events such as Pull Request reviews or comments to our FastAPI webhook endpoint. FastAPI receives the event, validates and processes the

payload, identifies the corresponding remediation workflow using the stored mapping, and resumes the appropriate workflow based on the human action.”

Your architecture specifically uses GitHub Webhooks for the event-driven HITL mechanism.

11. What happens when a user uploads a CSV?

Interview-ready:

“The user uploads the scanner CSV through the frontend. The request reaches our FastAPI upload endpoint. FastAPI receives the raw file and initiates the preprocessing pipeline. The file then goes through parsing, normalization, and prioritization before the vulnerabilities enter the remediation workflow.”

Flow:

User

↓

Next.js

↓

FastAPI Upload API

↓

Parser

↓

Normalization

↓

Prioritization

↓

Remediation Queue

12. How does FastAPI communicate with databases?

“FastAPI acts as the backend layer through which application logic interacts with our databases. PostgreSQL stores persistent vulnerability and remediation

information, while SQLite is used for active workflow state and temporary workflow-related data.”

Remember this distinction from your project architecture:

PostgreSQL → persistent/historical data

SQLite → active workflow state

13. FastAPI vs Next.js

🔥 Important because both can technically expose APIs.

“Next.js is primarily used for our frontend dashboard and user interface, while FastAPI is our dedicated Python backend. Next.js handles the presentation layer, and FastAPI handles API requests, backend logic, agent integration, and database communication.”

Easy:

Next.js → Frontend

FastAPI → Backend

14. FastAPI vs WebSocket

Don't confuse these.

“FastAPI handles our HTTP-based API communication and also provides the backend endpoint for WebSocket communication. WebSocket is a communication protocol that we use when we need continuous real-time communication between the frontend and backend.”

15. What happens if the backend receives a GitHub webhook while another workflow is running?

“The webhook event is associated with a specific remediation workflow using the stored PR-to-workflow mapping. The backend can then resume or update that particular workflow without confusing it with other remediation tasks.”

Your architecture stores **PR number → workflow thread ID** mapping specifically for this purpose.

🔥 **Most Important FastAPI Answer**

Interviewer: “Explain the role of FastAPI in your project.”

Say:

“FastAPI acts as the backend communication layer of our vulnerability remediation platform. Our Next.js dashboard communicates with FastAPI through APIs, and FastAPI handles operations such as scanner file uploads and GitHub webhook events. It connects the frontend with our AI-agent workflows and databases. For example, when a scanner CSV is uploaded, FastAPI receives it and initiates the preprocessing pipeline, while GitHub webhook events are received by FastAPI to resume the appropriate remediation workflow after human actions. So overall, FastAPI connects the user interface, agentic workflow, databases, and external integrations.”

Technology 3 — PostgreSQL

1. What is PostgreSQL?

Interviewer: “What is PostgreSQL?”

“PostgreSQL is an open-source relational database management system. It stores structured data in tables and supports SQL for querying and managing that data. In our project, we use PostgreSQL as the primary persistent database for vulnerability, asset, remediation, and historical system data.”

2. Why did you use PostgreSQL?

★ Important.

“We used PostgreSQL because our system needs reliable and persistent storage for vulnerability records, asset information, remediation history, and workflow-related data. PostgreSQL provides a structured relational model and is suitable for handling the historical data and analytics required by our vulnerability management platform.”

3. What exactly do you store in PostgreSQL?

Tumhare project ke context mein:

“PostgreSQL stores persistent vulnerability records, asset information, remediation history, and other long-term system data. It also supports the dashboard and historical reporting requirements of the platform.”

Simple:

PostgreSQL

- └─ Vulnerability Records
 - └─ Asset Information
 - └─ Remediation History
 - └─ Historical Data
 - └─ Dashboard / Analytics Data
-

4. Why didn't you use SQLite for everything?

🔥 Very important question.

“We use SQLite and PostgreSQL for different purposes. SQLite is used for lightweight local workflow state, temporary data, and LangGraph checkpointing, while PostgreSQL is used for persistent and historical system data. So SQLite supports active workflow execution, whereas PostgreSQL provides long-term storage.”

Remember:

SQLite → Active workflow

PostgreSQL → Persistent data

5. Explain PostgreSQL vs SQLite in your project.

PostgreSQL	SQLite
Primary persistent database	Lightweight local database
Long-term system data	Active workflow/temporary state
Vulnerability records	Workflow state
Remediation history	Agent/checkpoint state
Asset information	Local execution support
Dashboard/history	LangGraph workflow support

Interview answer:

“PostgreSQL is our long-term system of record, while SQLite is mainly used to support the active agentic workflow locally.”

6. Why not use MongoDB?

“We chose PostgreSQL because our vulnerability management data has a structured and relational nature. We have entities such as vulnerabilities, assets, remediation records, and historical information that need to be stored persistently and queried in a structured way. PostgreSQL fits this requirement well as a relational database and serves as our long-term persistent data store. We use SQLite separately for active workflow state and checkpointing.”

If interviewer pushes: “But MongoDB can also store this data, right?”

“Yes, MongoDB could also be used. The reason PostgreSQL was a better fit for our documented architecture is that our persistent data is structured and has relationships between different entities. A relational database gives us a natural way to represent and query those relationships using SQL.”

7. Why is PostgreSQL suitable for your dashboard?

“The dashboard needs persistent vulnerability and remediation information for monitoring and historical analysis. PostgreSQL provides structured querying over this data, allowing the backend to retrieve information required for dashboard views, reporting, and analytics.”

8. How does FastAPI interact with PostgreSQL?

“FastAPI acts as the backend layer between the frontend and PostgreSQL. When the dashboard requests vulnerability or remediation information, FastAPI retrieves the required data from PostgreSQL and sends it to the frontend through API responses.”

Flow:

Next.js Dashboard

↓

FastAPI

↓

PostgreSQL



Vulnerability /

Remediation Data

9. How does PostgreSQL fit into the complete architecture?

“PostgreSQL is the persistent data layer of our platform. The processing and agent workflows generate vulnerability and remediation information, which is persisted in PostgreSQL. FastAPI provides access to this data for the frontend dashboard, reporting, and historical analysis.”

10. Why is persistent storage important in your project?

“Vulnerability remediation is not just a real-time operation. We need to maintain historical information about vulnerabilities, assets, remediation attempts, and their outcomes. Persistent storage allows us to retain this information even after an individual workflow or execution environment has finished.”

11. What is a relational database?

“A relational database stores structured data in tables consisting of rows and columns. Relationships between different entities can be represented using keys, and SQL is used to query and manipulate the data.”

Example in your project conceptually:

Vulnerabilities



Asset



Remediation



History

12. What is a primary key?

“A primary key is a column or set of columns that uniquely identifies each record in a table.”

If interviewer asks:

“Why is it important in your project?”

“It allows us to uniquely identify records such as vulnerabilities, assets, or remediation entries and maintain relationships between different pieces of data.”

13. What is a foreign key?

“A foreign key is a field that references a key in another table. It is used to establish relationships between tables and maintain referential integrity.”

14. How would you handle thousands of vulnerabilities?

“PostgreSQL is designed to handle large volumes of structured data. We would also use appropriate indexing, efficient queries, and proper database design so that frequently accessed fields such as vulnerability IDs, asset identifiers, and status can be queried efficiently.”

⚠ Don't claim specific indexes unless you actually implemented them.

15. How do you prevent duplicate vulnerability records?

Possible question.

“We can use appropriate uniqueness constraints based on the identifiers that define a vulnerability record, along with deduplication during the normalization stage. The exact uniqueness criteria depend on how the vulnerability and affected asset are identified in the system.”

This is better than blindly saying “CVE ID is always unique”, because the same CVE can affect multiple assets.

16. What happens if PostgreSQL becomes unavailable?

Scenario question:

“The persistent data operations would fail, so the system should not incorrectly report that the remediation has been completed. The backend should handle the database failure appropriately, such as returning an error or retrying the operation

depending on the operation. The active workflow state is separately handled through the workflow-state mechanism, so we can distinguish database persistence failures from workflow execution failures.”

Most Important PostgreSQL Answer

Interviewer: “Explain the role of PostgreSQL in your project.”

Say:

“PostgreSQL is our primary persistent database. We use it to store long-term vulnerability records, asset information, remediation history, and other system data required by the platform. The FastAPI backend communicates with PostgreSQL and provides this information to the Next.js dashboard for monitoring and historical analysis. We use SQLite separately for lightweight active workflow state and LangGraph checkpointing, so PostgreSQL acts as the long-term system of record while SQLite supports the running agent workflows.”

Technology 4 — SQLite

Ab tumhare project ke SQLite ko prepare karte hain. Ye PostgreSQL se confuse nahi hona chahiye, because dono ka role different hai.

1. What is SQLite?

“SQLite is a lightweight, serverless relational database that stores the database directly as a local file. In our project, we use SQLite mainly for active workflow state, temporary workflow data, and LangGraph checkpointing.”

2. Why did you use SQLite?

 Important:

“We used SQLite for the active workflow layer because our remediation workflows need lightweight and fast local state management. It is easy to deploy because it doesn't require a separate database server, and it works well for storing LangGraph checkpoints and the current state of active remediation workflows.”

3. What exactly do you store in SQLite?

Tumhare project mein important data:

“SQLite stores the active remediation workflow state, LangGraph checkpoints, workflow status, and mappings required to resume workflows. For example, we maintain the relationship between a Pull Request and its corresponding workflow thread so that when a GitHub webhook arrives, we know exactly which workflow needs to be resumed.”

Tumhare documentation mein state_db.sqlite mein specifically workflow state aur PR-to-thread mapping use hoti hai.

4. Why not use PostgreSQL for everything?

Most important SQLite question

“We separated the two responsibilities. PostgreSQL is our persistent database for long-term vulnerability, asset, and remediation data, while SQLite is used for active workflow state and checkpointing. This keeps the running LangGraph environment lightweight and avoids mixing temporary execution state with long-term business data.”

Remember:

PostgreSQL



Long-term / Persistent Data

SQLite



Active Workflow / Checkpoints

5. What is LangGraph checkpointing?

Very important.

“Checkpointing means saving the current state of a workflow so that it can be resumed later instead of starting from the beginning. In our project, this is important because the remediation workflow can pause during the Human-in-the-Loop stage and later resume after receiving feedback through a GitHub webhook.”

Example:

Remediation

↓

Validation

↓

WAITING_FOR_HUMAN_APPROVAL

↓

[Workflow State Saved]

↓

Human approves/rejects

↓

Webhook

↓

Resume Workflow

6. Why is checkpointing important in your project?

“Our remediation workflow is long-running and can involve retries and human approval. If we didn't persist the workflow state, we would have to restart the workflow from the beginning after every pause or interruption. Checkpointing allows us to preserve the current state and resume from the appropriate point.”

7. What is a thread ID?

Possible interviewer question.

“A thread ID uniquely identifies a particular workflow execution. We use it to associate the workflow state and related information with a specific remediation task.”

For example:

Thread ID

↓

Vulnerability

↓

Remediation Workflow

↓

GitHub PR

8. How do you connect a GitHub PR with a workflow?

🔥 Very relevant to your project.

“When the Pull Request is created, we store the PR number along with the corresponding workflow thread ID in the state database. Later, when GitHub sends a webhook for that PR, we use this mapping to identify the correct workflow and resume it.”

Flow:

Workflow

↓

Create PR

↓

PR #123

↓

Store:

PR #123 → Thread ID

↓

Human Review

↓

GitHub Webhook

↓

Find Thread ID

↓

Resume Workflow

9. What happens when human approval is pending?

This is a very strong project-specific question.

“The workflow enters the `WAITING_FOR_HUMAN_APPROVAL` state and its current state is persisted. The agent doesn't continuously execute or poll GitHub. It waits for the relevant GitHub webhook event. Once the approval or reviewer feedback arrives, the system uses the stored PR-to-thread mapping to resume the correct workflow.”

This is an important part of your architecture.

10. SQLite vs PostgreSQL

Interviewer: “Explain the difference in your project.”

“PostgreSQL is used as the long-term persistent data store for our vulnerability management platform, while SQLite is used for active workflow execution and checkpointing. PostgreSQL contains persistent business and historical data, whereas SQLite contains the state required to run and resume the agent workflows.”

Table:

PostgreSQL	SQLite
Persistent database	Active workflow database
Long-term data	Current workflow state
Vulnerabilities	Workflow checkpoints
Assets	Thread/workflow state
Remediation history	PR-to-thread mapping
Dashboard/history	LangGraph state

11. Why not use Redis for workflow state?

Possible advanced question.

Don't say Redis is incapable.

“Redis could also be used for fast temporary state, but our workflow requires persistent checkpointing and a relational structure for the state we maintain.

SQLite provided a lightweight local solution without requiring another external infrastructure component.”

12. What happens if SQLite is deleted?

Scenario question:

“If the active workflow database is lost, the system could lose the checkpoint and mapping information required to resume active workflows. That's why this state is important during execution. Long-term vulnerability and remediation records are maintained separately in PostgreSQL.”

13. Why is SQLite suitable for LangGraph?

“SQLite is lightweight and works well for local state persistence. Since LangGraph needs to persist workflow checkpoints and resume stateful executions, SQLite provides a simple way to maintain that state without introducing another database server just for workflow execution.”

Most Important Interview Answer

“Explain why you used SQLite in your project.”

Say:

“We use SQLite specifically for active workflow state and LangGraph checkpointing. Our remediation workflows are stateful and can pause during Human-in-the-Loop approval or retry after failures. SQLite stores the current workflow state and mappings such as the Pull Request to workflow thread ID, allowing us to resume the correct workflow when a GitHub webhook arrives. We use PostgreSQL separately for long-term vulnerability, asset, and remediation data, so we keep workflow execution state separate from persistent business data.”

Technology 5 — LangChain

Ab LangChain ko tumhare project ke actual documentation ke according prepare karte hain.

Tumhare stack mein LangChain LLM integration aur tool integration ke liye hai, jabki LangGraph workflow orchestration aur state management handle karta hai. Is distinction ko interview mein clear rakhna.

1. What is LangChain?

Interviewer: “What is LangChain?”

“LangChain is a framework for building applications powered by Large Language Models. It provides abstractions for connecting an LLM with prompts, tools, external data sources, and application logic. In our project, we use LangChain as part of the AI-agent layer for LLM integration and tool interaction.”

2. Why did you use LangChain?

★ Important.

“We used LangChain because our remediation system is not just a simple LLM application. The agents need to use external tools, process vulnerability information, perform research, analyze logs, and generate remediation scripts. LangChain provides the integration layer between the LLM and these tools and application components.”

Your documented technology stack identifies LangChain as part of the LLM integration and agent tooling layer.

3. What exactly does LangChain do in your project?

“LangChain is used to connect our LLM with the tools and context required by the vulnerability remediation workflow. For example, the remediation agent can use vulnerability information, external research, execution logs, and available tools as context for the LLM to analyze the problem and generate remediation actions.”

Your documentation describes the remediation agent as doing vulnerability research, script generation, GitHub automation, sandbox validation, and self-correction.

4. Is LangChain an LLM?

🔥 Very common question.

“No. LangChain is not an LLM. It is a framework used to build applications around LLMs. The LLM provides the reasoning or generation capability, while LangChain provides the application-level integration with prompts, tools, context, and other components.”

Remember:

OpenAI / Gemini → LLM

LangChain → LLM application framework

5. LangChain vs LangGraph

🔥 🔥 Extremely important for your project.

Interviewer:

“If you're already using LangChain, why do you need LangGraph?”

Tum bolo:

“LangChain mainly provides the building blocks for integrating the LLM with tools and application logic, whereas LangGraph is used to orchestrate the stateful, multi-step workflow. In our project, LangChain helps with the LLM and tool integration, while LangGraph manages the remediation workflow, state transitions, retries, and pauses for human approval.”

Simple:

LangChain

↓

LLM + Tools + Context

LangGraph

↓

Workflow + State + Decisions

This distinction aligns with your documented stack: LangGraph is specifically used for workflow state and multi-step decision making.

6. What is a Tool in LangChain?

“A tool is an external function or capability that an agent can invoke to perform an action or retrieve information. For example, a tool could perform a search, interact with a repository, access a database, or execute an infrastructure operation.”

In your project, examples of external capabilities include:

- vulnerability research

- **GitHub operations**
- **AWS operations**
- **Jira operations**

But: only claim a specific operation as a LangChain tool if that is how your actual implementation exposed it. Your documentation establishes these integrations, but does not document every one as a specific LangChain tool.

7. What is an AI Agent in LangChain?

“An agent is a system where the LLM can reason about a task and decide which available tool or action should be used to accomplish it. In our project, this approach is useful because remediation isn't a fixed single-step operation—it can require research, script generation, execution, error analysis, and retry.”

8. Why not just call OpenAI API directly?

 **Good interview question.**

“We could call the model directly, but our workflow requires more than text generation. The agent needs to interact with tools and external systems and use their results as context. LangChain provides abstractions that make this integration more structured and easier to manage.”

9. How does LangChain use external information?

For your project:

“The agent can combine the original vulnerability information with additional context retrieved from external sources. For example, during remediation research, web search results provide information about advisories, affected versions, patches, and remediation approaches. That context can then be passed to the LLM for analysis and script generation.”

Your documentation explicitly describes original vulnerability information + collected research → LLM → three remediation scripts.

10. How does LangChain help with self-correction?

 **Important.**

“When a remediation attempt fails, the execution logs are provided as feedback to the LLM. The agent can analyze those logs, identify the likely cause of failure, and generate an improved remediation. The updated scripts are then tested again. LangChain provides the LLM and tool integration needed for this kind of agentic interaction, while LangGraph controls the overall retry workflow.”

Your documentation specifically describes the failure → S3 logs → LLM feedback → improved remediation → new validation cycle.

11. What is Prompt Engineering and why is it important?

Your project actually faced a related challenge.

“Prompt engineering is the process of designing and refining instructions given to the LLM so that it produces the desired output reliably. In our project, it was important because the agent needed to generate structured remediation scripts and avoid unnecessary tool calls.”

Your documentation says the LLM once entered a three-minute tool-calling loop, increasing latency and cost, and prompt engineering was required to reduce unnecessary Tavily calls.

12. What challenge did you face with LangChain/LLM tool calling?

This is a great real-project answer:

“One challenge was that the LLM could make excessive tool calls. In one case, it got stuck in a tool-calling loop for around three minutes, which increased both latency and cost. We addressed this by improving the prompts and controlling unnecessary tool usage, particularly around Tavily searches.”

13. How do you prevent the agent from blindly trusting tool results?

“Tool results are treated as context rather than as an automatically trusted remediation instruction. The generated remediation is tested in a sandboxed environment, followed by vulnerability validation. Only after successful validation and human approval can the remediation reach production.”

Your documented pipeline explicitly follows research → script generation → sandbox validation → HITL approval → production execution.

14. LangChain vs OpenAI

Another common question:

“OpenAI provides the language model capability, while LangChain provides the framework and abstractions for integrating that model with tools, prompts, context, and application logic.”

Simple:

OpenAI = Brain

LangChain = Framework connecting the brain with tools/context

15. LangChain vs MCP

Since you use both, this can definitely come up.

“LangChain is an application framework for building LLM and agent-based applications, while MCP is a standardized protocol for exposing and connecting external tools to AI applications. In our architecture, LangChain is part of the agent/LLM integration layer, while MCP provides standardized access to external systems such as GitHub.”

Remember:

LLM

↓

LangChain

↓

Agent Logic

↓

MCP

↓

External Tools

Don't say LangChain = MCP. They solve different problems.

🔥 Most Important Interview Answer

“Explain how you used LangChain in your project.”

Say:

“We used LangChain as part of our AI-agent layer to integrate the LLM with the tools and contextual information required for vulnerability remediation. The agent can use vulnerability data, external research, and execution logs as context and interact with the required tools to perform the remediation workflow. For example, during remediation research, external security information is collected and provided to the LLM, which then generates the precheck, remediation, and validation scripts. If execution fails, the logs are fed back to the agent so it can generate an improved remediation. LangGraph then manages the overall stateful workflow, retries, and human approval stages.”

Technology 6 — LangGraph

Ye tumhare project ka one of the most important components hai. Interviewer yahan se deep questions pooch sakta hai because your system is a stateful, multi-step remediation workflow.

Tumhare documentation ke according LangGraph ka role specifically workflow state aur multi-step decision making hai.

1. What is LangGraph?

Interviewer: “What is LangGraph?”

Tum bolo:

“LangGraph is a framework for building stateful and multi-step agent workflows. It represents the workflow as a graph of nodes and transitions, where each node performs a particular operation and the state is passed between the nodes. In our project, we use LangGraph to orchestrate the vulnerability remediation workflow, including remediation, validation, retries, and Human-in-the-Loop approval.”

2. Why did you use LangGraph?

★ Very important

“Our remediation process is not a simple linear LLM call. It has multiple steps and conditional paths. For example, after generating a remediation script, it may succeed, fail and require self-correction, or pause for human approval. LangGraph allows us to represent these steps and transitions explicitly while maintaining the state of the workflow.”

Your workflow:

Research



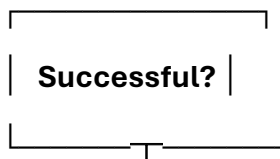
Script Generation



Execution



Validation



No | Yes



Self-Correction



Retry

Success



Human Approval



Production

3. Why not just use LangChain?

🔥 Very likely question

“LangChain provides the building blocks for LLM and tool integration, but our workflow requires explicit state management and conditional transitions. LangGraph is better suited for representing a stateful graph where the workflow can branch, retry, pause, and resume. So we use LangChain for the LLM and tool integration layer and LangGraph for workflow orchestration.”

Easy:

LangChain → Agent building blocks

LangGraph → Agent workflow

4. What is a Node in LangGraph?

“A node represents an individual operation in the workflow. For example, a node can perform vulnerability research, generate a remediation, execute a validation step, or handle human approval.”

Conceptually:

[Research Node]

↓

[Generate Script]

↓

[Execute]

↓

[Validate]

5. What is an Edge?

“An edge defines the transition from one node to another. It determines what step should execute next after a node finishes.”

Example:

Execute

↓

Validate

↓

Success → HITL

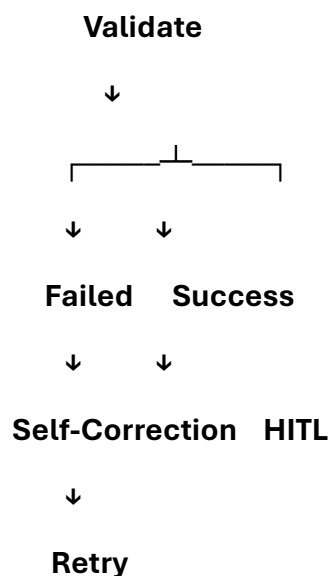
Failure → Self-Correction

6. What is Conditional Routing?

🔥 Important.

“Conditional routing means the next node is selected based on the current workflow state or result. In our project, for example, if remediation validation fails, the workflow goes to the self-correction path, while if validation succeeds, it moves toward the Human-in-the-Loop approval stage.”

Example:



7. What is State in LangGraph?

★ Very important

“State represents the information currently available to the workflow. It can contain things such as the vulnerability being processed, execution results, logs, remediation information, workflow status, and other information required by subsequent nodes.”

Simple:

“State is basically the shared information that the workflow carries from one step to another.”

8. Why is state important in your project?

“Because our remediation workflow is long-running and can have multiple branches, retries, and pauses. The next step needs to know what happened in the previous step. Maintaining state allows the workflow to make the correct decision and resume from the appropriate point.”

9. What is checkpointing?

🔥 Very important

“Checkpointing means persisting the current workflow state so that the workflow can be resumed later. In our project, this is particularly important during the Human-in-the-Loop stage. When the workflow waits for human approval, its state is persisted, and after the GitHub webhook provides the review result, the workflow can resume from the saved state.”

10. Why do you need Human-in-the-Loop in LangGraph?

“The remediation workflow can automatically generate and validate a fix, but we don’t want an AI-generated change to reach production without human review. So after successful sandbox validation, the workflow enters a waiting state for human approval. Once the reviewer approves or provides feedback, the workflow resumes accordingly.”

Your documented workflow explicitly enters `WAITING_FOR_HUMAN_APPROVAL`, persists the state, and resumes after GitHub events.

11. How does the workflow resume after human approval?

★ Excellent interviewer question

“When the Pull Request is created, we store its PR number along with the corresponding workflow thread ID. When GitHub sends a webhook for a review or comment, our backend uses that mapping to identify the correct workflow. The saved workflow state is then used to resume the appropriate LangGraph execution.”

Flow:

LangGraph Workflow

↓

Create PR



Save PR → Thread ID



WAITING_FOR_HUMAN_APPROVAL



GitHub Review



Webhook → FastAPI



Find Thread ID



Resume Workflow

12. What is a Thread ID?

“A thread ID identifies a particular workflow execution. It allows us to associate the persisted state and external events, such as a GitHub Pull Request, with the correct remediation workflow.”

13. How does LangGraph handle remediation failure?

🔥 Important.

“When validation or execution fails, the workflow doesn't simply terminate. The failure information and logs are made available to the remediation agent. The self-correction process analyzes the failure and generates an improved remediation. LangGraph then routes the workflow back into the remediation and validation cycle.”

Your documentation describes exactly this self-healing loop.

14. How does the self-correction loop work?

Interview answer:

“The workflow executes the generated remediation and then validates the result. If it fails, the execution logs are retrieved from S3 and provided as feedback to the LLM. The agent analyzes the error and generates an improved script. The updated scripts are pushed back through the GitHub workflow, and the validation cycle runs again. This continues until the remediation succeeds or the retry limit is reached.”

Flow:

Generate

↓

Execute

↓

Validate

↓

Failed?

↓ YES

Logs → S3

↓

LLM Analysis

↓

Improved Script

↓

Execute Again

15. What happens if the retry limit is reached?

“The workflow stops the self-healing loop rather than retrying indefinitely. The remediation remains unsuccessful and can be moved into an appropriate failure or unresolved state for further investigation.”

Your documentation explicitly says the loop continues until success or the retry limit is reached.

16. How does LangGraph interact with GitHub?

“LangGraph manages the remediation workflow state, while GitHub provides the repository and Pull Request lifecycle. The workflow creates or updates the remediation Pull Request, waits for human feedback, and resumes based on GitHub webhook events.”

17. How does LangGraph interact with FastAPI?

“FastAPI acts as the backend interface, while LangGraph manages the actual stateful remediation workflow. FastAPI receives requests and external events such as GitHub webhooks, and the appropriate LangGraph workflow is triggered or resumed based on those events.”

18. LangGraph vs traditional workflow?

Possible advanced question:

“A traditional workflow can handle predefined sequential steps, but our remediation workflow has dynamic decisions based on execution results, self-correction, retries, and human approval. LangGraph is useful because it represents these conditional and stateful transitions explicitly.”

19. Why is LangGraph suitable for your project?

“Because our remediation workflow is stateful, conditional, and long-running. It can move forward, branch into self-correction, retry failed remediation, pause for human approval, and later resume from the saved state. LangGraph provides a natural graph-based model for this kind of workflow.”

Most Important Question

Interviewer: “Explain exactly how you used LangGraph in your project.”

Tum bolo:

“We use LangGraph as the orchestration framework for our stateful vulnerability remediation workflow. The workflow is represented as multiple nodes connected through transitions, with the current state passed between them. After vulnerability research and remediation generation, the workflow executes and validates the generated scripts in a sandboxed VM. If validation fails, the workflow routes the failure information to the self-correction process and retries. If validation succeeds, the workflow moves to a Human-in-the-Loop state and waits

for the Pull Request review. The workflow state is persisted, and when a GitHub webhook arrives with the review result, the correct workflow is identified using the PR-to-thread mapping and resumed. So LangGraph gives us the state management, conditional routing, retries, and pause-resume capability required for our remediation pipeline.”

Technology 7 — Next.js

Ab Next.js ko tumhare project ke actual technology stack ke context mein prepare karte hain.

Tumhare project mein Next.js frontend/dashboard layer hai. Iska main purpose vulnerability management dashboard ko provide karna hai, jahan user vulnerabilities, assets, remediation status aur workflow information ko monitor kar sakta hai.

1. What is Next.js?

Interviewer: “What is Next.js?”

“Next.js is a React-based full-stack web framework. It provides features such as routing, server-side rendering, API integration, and optimized frontend development. In our project, we mainly use Next.js to build the frontend dashboard for monitoring vulnerabilities and remediation workflows.”

2. Why did you use Next.js?

“We used Next.js because we needed a structured frontend framework for building our vulnerability management dashboard. It provides routing and React-based component development, and it integrates well with our FastAPI backend for fetching and displaying vulnerability and remediation data.”

Simple:

Next.js → Dashboard/UI

FastAPI → Backend/API

3. What exactly does Next.js do in your project?

Tum bolo:

“Next.js provides the user interface for our vulnerability management platform. It allows users to view vulnerability data, prioritization results, remediation status,

asset information, and workflow progress. It communicates with our FastAPI backend to retrieve and update the required data.”

4. Explain the frontend-backend flow.

★ Very important.

User

↓

Next.js Dashboard

↓

HTTP Request

↓

FastAPI

↓

Agents / Database

↓

FastAPI Response

↓

Next.js

↓

UI Update

Interview answer:

“The user interacts with the Next.js dashboard. When the frontend needs data, it sends a request to the FastAPI backend. FastAPI processes the request and retrieves the required information from the agents or database, then returns the response to Next.js, which renders it on the dashboard.”

5. Why not use plain React?

🔥 Very likely question.

“React is primarily a UI library, whereas Next.js provides a more complete application framework around React. It gives us features such as routing and other application-level capabilities out of the box. Since our project required a structured dashboard application rather than only individual UI components, Next.js was a suitable choice.”

Remember:

React → UI library

Next.js → Framework built around React

6. Next.js vs React

React	Next.js
UI library	React-based framework
Component focused	Complete application structure
Routing requires additional solution	Built-in routing capabilities
More setup required	Provides application conventions
UI development	Full web application framework

Interview mein short:

“React helps us build the UI, while Next.js provides the framework and application structure around React.”

7. How does Next.js communicate with FastAPI?

“Next.js communicates with FastAPI through HTTP APIs. The frontend sends requests to the appropriate FastAPI endpoints and receives JSON responses containing vulnerability, asset, remediation, or workflow information.”

Example conceptual flow:

Next.js

|

| GET / API request

↓

FastAPI

|

↓

PostgreSQL / SQLite / Workflow

|

↓

JSON Response

|

↓

Next.js Dashboard

8. How do you show real-time workflow updates?

★ Important because your stack includes WebSocket.

“For real-time updates, our frontend can maintain a WebSocket connection with the backend. This allows the backend to push workflow or remediation status updates to the dashboard without requiring the frontend to repeatedly poll the server.”

Polling:

Frontend → Status?

Frontend → Status?

Frontend → Status?

WebSocket:

Frontend ↔ Backend

Persistent Connection

↓

Real-time Updates

9. Why WebSocket instead of polling?

“Polling requires the frontend to repeatedly send requests even when there is no change. WebSocket provides a persistent connection, allowing the backend to push updates when something actually changes. This is more suitable for displaying real-time remediation workflow progress.”

10. What kind of information can the dashboard show?

Based on your project's purpose:

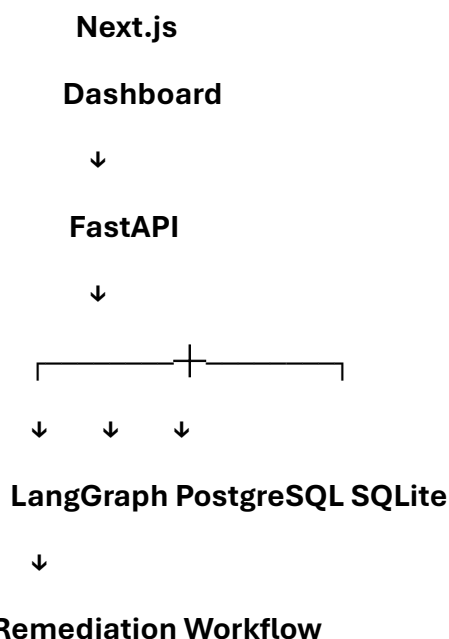
“The dashboard can provide visibility into vulnerabilities, their severity and priority, affected assets, remediation status, workflow progress, and historical remediation information.”

Don't claim a specific UI widget unless it is actually present in your implementation.

11. How does Next.js fit into the complete architecture?

“Next.js is the presentation layer of our platform. It communicates with FastAPI for backend operations, receives vulnerability and remediation information, and presents that information to the user. The actual vulnerability processing and agentic workflow happen in the backend through Python, LangChain, and LangGraph.”

Architecture:



12. What is SSR?

Possible interviewer question:

“SSR stands for Server-Side Rendering. It means the HTML for a page is generated on the server and then sent to the client. This can improve initial rendering and can be useful for applications where server-generated content is beneficial.”

⚠ Important: Your project document doesn't establish that you specifically used SSR for the dashboard. So if asked “Did you use SSR?”, answer based on your actual code rather than claiming it.

13. What is CSR?

“CSR stands for Client-Side Rendering. In CSR, the browser loads the application and renders or updates the UI on the client side using JavaScript. Interactive dashboards commonly use client-side rendering for dynamic updates.”

Again, don't claim a specific rendering strategy unless your implementation confirms it.

14. Why is Next.js suitable for your dashboard?

“Our dashboard is highly interactive because users need to monitor vulnerability and remediation states. Next.js provides a structured React-based environment for building this interface, while the backend APIs and WebSocket communication allow the dashboard to receive the required data and workflow updates.”

15. What happens when backend is unavailable?

Scenario:

“The frontend should handle the API failure gracefully rather than assuming that the request succeeded. It can show an appropriate error or unavailable state to the user. The backend and workflow state remain separate, so a temporary frontend/API failure doesn't mean that the remediation workflow itself has failed.”

This distinction is important:

UI failure ≠ Remediation failure

16. Next.js vs FastAPI

Must remember

Next.js:

“Frontend and dashboard layer.”

FastAPI:

“Backend API and integration layer.”

If interviewer asks:

“Why not build the backend in Next.js itself?”

Answer:

“Our core backend and AI-agent workflows are implemented in Python using LangChain and LangGraph. Using FastAPI allows us to keep the backend in the same Python ecosystem as those components, while Next.js focuses on the frontend dashboard.”

This is a strong architecture-based justification.

Most Important Interview Answer

“Explain the role of Next.js in your project.”

Tum bolo:

“We use Next.js as the frontend framework for our vulnerability management dashboard. It provides the user interface through which users can monitor vulnerabilities, assets, prioritization, remediation status, and workflow progress. The Next.js frontend communicates with our FastAPI backend through APIs, while real-time workflow updates can be delivered through WebSocket communication. The backend handles the actual AI-agent workflows and database operations, while Next.js is primarily responsible for presenting that information to the user.”

Technology 8 — WebSocket

Ab last technology WebSocket ko interview ke liye prepare karte hain.

Tumhare stack mein WebSocket ka purpose real-time communication between the frontend dashboard and backend hai.

1. What is WebSocket?

Interviewer: “What is WebSocket?”

“WebSocket is a communication protocol that provides a persistent, two-way communication channel between a client and a server. Unlike normal HTTP request-response communication, once the WebSocket connection is established, both the client and server can send messages to each other whenever required.”

Simple:

HTTP:

Client → Request → Server

Client ← Response ← Server

WebSocket:

Client ←————→ Server

Persistent

Connection

2. Why did you use WebSocket in your project?

★ Important.

“We use WebSocket to provide real-time updates to the vulnerability management dashboard. Remediation workflows can run for some time and their status can change during execution. Instead of repeatedly polling the backend, the frontend can maintain a WebSocket connection and receive workflow updates from the backend as they occur.”

3. What problem does WebSocket solve?

“The main problem is real-time status visibility. Without WebSocket, the frontend would need to repeatedly ask the backend whether the remediation status has changed. WebSocket allows the backend to push an update to the frontend when there is a relevant change.”

4. WebSocket vs HTTP

🔥 Very common interview question.

HTTP	WebSocket
Request-response model	Two-way communication
Usually short-lived requests	Persistent connection
Client generally initiates request	Either side can send messages
Polling may be required for updates	Server can push updates
Good for normal APIs	Good for real-time communication

Interview answer:

“HTTP is primarily request-response based, while WebSocket provides a persistent bidirectional connection. We use HTTP APIs for normal backend operations and WebSocket when the dashboard needs real-time workflow updates.”

5. Why not use polling?

🔥 Very likely.

Suppose remediation takes several minutes.

With polling:

Frontend

↓

Status?

↓

Backend

Frontend

↓

Status?

↓

Backend

Frontend

↓

Status?

↓

Backend

This creates unnecessary requests.

WebSocket:

Frontend ←————→ Backend

↓

Status Changes

↓

Push Update

Interview answer:

“Polling requires the client to repeatedly send requests even when there is no change. With WebSocket, we maintain a persistent connection and the backend can push updates when the workflow state changes. This reduces unnecessary requests and gives the dashboard more immediate updates.”

6. How does WebSocket work?

★ **Basic technical question.**

There are generally two stages:

Step 1 — Handshake

The client initially connects to the WebSocket endpoint.

Step 2 — Persistent connection

After the connection is established, both sides can exchange messages.

Interview answer:

“A WebSocket connection begins with a handshake. Once the handshake succeeds, the connection is upgraded to a persistent WebSocket connection. After that, the client and server can exchange messages in both directions without creating a new HTTP request for every update.”

7. How does WebSocket fit into your architecture?

Your architecture can be explained like this:

User

↓

Next.js Dashboard

↓

WebSocket Connection

↓

FastAPI Backend

↓

Remediation Workflow

↓

LangGraph

Tum bolo:

“The Next.js dashboard establishes a WebSocket connection with our FastAPI backend. As the remediation workflow progresses, the backend can send relevant status updates through that connection, allowing the dashboard to reflect the current workflow state in real time.”

8. What kind of updates would you send?

Project context mein:

“The WebSocket channel can be used for workflow-related status updates, such as remediation progress, execution status, validation status, or other relevant changes that need to be reflected on the dashboard.”

⚠️ Exact event names ya message structure tabhi batao agar interviewer code-level implementation pooche aur tumhare actual code mein defined ho.

9. Does WebSocket execute the remediation?

✗ No.

Very important distinction.

“No. WebSocket is only a communication mechanism for real-time updates. It does not execute the remediation itself. The remediation workflow is handled by LangGraph and the infrastructure execution layer such as AWS SSM and EC2.”

Remember:

WebSocket → Communication

LangGraph → Workflow

SSM → Remote execution

EC2 → Execution environment

10. WebSocket vs FastAPI

Interviewer may ask:

“Is WebSocket a replacement for FastAPI?”

Answer:

“No. FastAPI is our backend framework, while WebSocket is a communication protocol. FastAPI can expose WebSocket endpoints and handle the WebSocket connection. So in our architecture, FastAPI provides the backend and WebSocket provides the real-time communication mechanism.”

 **This is important.**

11. WebSocket vs REST API

“REST APIs are suitable for normal request-response operations such as retrieving vulnerability data or uploading scanner files. WebSocket is useful when the server needs to continuously push updates to the client, such as remediation workflow progress.”

Your architecture:

Normal Data

Next.js

↓ HTTP/REST

FastAPI

↓

Database

Real-Time Updates

Next.js

↕ **WebSocket**

FastAPI

↓

Workflow Updates

12. What happens if WebSocket connection disconnects?

🔥 **Scenario question.**

“The frontend should detect the disconnection and attempt to reconnect. The important point is that the remediation workflow itself should not depend on the WebSocket connection. WebSocket is only used for displaying updates, while the actual workflow state is maintained separately by the backend and workflow persistence layer.”

This is a very strong architectural answer.

Remember:

Dashboard connection lost ≠ Remediation workflow lost

13. What if the frontend disconnects while remediation is running?

“The remediation should continue independently because the workflow is managed by the backend. When the frontend reconnects, it can retrieve the current state through the backend and continue receiving real-time updates.”

14. Why is this important for your project?

“Our remediation workflow is long-running and has multiple stages such as execution, validation, retries, and human approval. Real-time communication gives the user visibility into what the system is currently doing without manually refreshing the dashboard.”

15. Can WebSocket be used for Human-in-the-Loop?

Possible tricky question.

“WebSocket can be used to display the current HITL status to the frontend, but in our architecture the actual GitHub human-review event is received through GitHub Webhooks. The webhook notifies the backend about the review action, while WebSocket can be used to reflect the resulting status on the dashboard.”

🔥 This distinction is very important.

Don't confuse:

GitHub Review

↓

GitHub Webhook

↓

FastAPI

↓

LangGraph Resume

↓

WebSocket

↓

Dashboard Update

16. WebSocket vs GitHub Webhook

Interviewer may specifically ask this.

WebSocket:

“Used for real-time communication between our frontend and backend.”

GitHub Webhook:

“Used by GitHub to notify our backend when an external GitHub event occurs.”

One-line difference:

“WebSocket is our frontend-backend real-time communication channel, while GitHub Webhook is an event notification mechanism from GitHub to our backend.”

Most Important WebSocket Answer

Interviewer: “Explain how you use WebSocket in your project.”

Tum bolo:

“We use WebSocket to provide real-time workflow updates to our Next.js dashboard. Since vulnerability remediation is a long-running process involving execution, validation, retries, and human approval, the user needs to see the current status without repeatedly polling the backend. The frontend establishes a persistent WebSocket connection with our FastAPI backend, and the backend can push relevant workflow status updates as the remediation progresses. The WebSocket is only responsible for communication and UI updates; the actual workflow is managed by LangGraph and the remediation is executed through the infrastructure layer.”

Parser Agent — Interview Preparation

1. First: Parser Agent kya hai?

Interviewer: “What is the Parser Agent?”

Tum bolo:

“Parser Agent is an AI-driven component of our vulnerability management system that converts heterogeneous security scanner outputs into a standardized format. Different scanners generate reports in different JSON structures, so instead of writing a separate static parser for every scanner, our Parser Agent analyzes the scanner output and dynamically generates the required parsing logic using an LLM. It then validates the extracted data and automatically retries if parsing fails.”

Short version:

“Parser Agent converts different scanner outputs into one standardized vulnerability format using dynamically generated parsing logic.”

2. Why did you need a Parser Agent?

Very important question

Interviewer:

“Why can't you simply write parsers manually?”

Answer:

“Because different security scanners produce different report structures and schemas. If we use static parsers, we would have to manually develop and maintain a separate parser for every scanner. Whenever a scanner changes its output structure or a new scanner is added, the parser would also need to be updated. Our AI-driven Parser Agent solves this by analyzing the input structure and generating scanner-specific parsing logic dynamically.”

Example:

Trivy → JSON Structure A

Snyk → JSON Structure B

Grype → JSON Structure C

OWASP → JSON Structure D

ZAP → JSON Structure E

Without Parser Agent:

Trivy Parser

Snyk Parser

Grype Parser

OWASP Parser

ZAP Parser

...

With Parser Agent:

Scanner JSON

↓

Parser Agent

↓

Analyze Structure

↓

Generate Parser

↓

Standardized Output

3. What problem does Parser Agent solve?

“The main problem is heterogeneity of scanner outputs. Different scanners use different JSON structures, field names, nesting levels, and vulnerability representations. The Parser Agent provides an adaptive parsing mechanism that converts these different formats into a common structure that downstream agents can process.”

Remember:

Problem → Heterogeneous scanner output

Solution → Adaptive AI-based parser

Output → Standardized vulnerability data

4. Which scanners can your Parser Agent handle?

According to your documentation:

“The Parser Agent accepts vulnerability reports from scanners such as Trivy, Snyk, Gripe, OWASP, ZAP, and other supported scanner outputs.”

Don't say **“it supports every scanner automatically”** because your document says *other industry supported scanner outputs*, not unlimited arbitrary scanners.

5. Explain the complete Parser Agent workflow.

★ **This is probably the most important question.**

Interviewer:

“Can you explain the Parser Agent from input to output?”

Tum bolo:

“First, the Parser Agent discovers the available scanner output files. It loads the raw JSON report and analyzes its structure to identify where the vulnerability information is located. It then uses an LLM to generate scanner-specific parsing logic. The generated parser is executed and the extracted data is validated. If the parser fails, produces empty results, or encounters a runtime error, the error

context is sent back to the LLM, which generates an improved parser. This self-correction cycle continues until successful extraction or until the retry threshold is reached. Finally, the normalized vulnerability data is exported into CSV format for downstream processing.”

Full flow:

Raw Scanner Reports



Discover Files



Load JSON



Analyze Structure



LLM



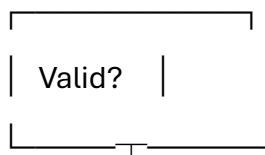
Generate Parser



Execute Parser



Validate Output



Yes | No



Export CSV



↓

Downstream Agents

Failure path:

Parser Failure

↓

Capture Error

↓

LLM

↓

Generate New Parser

↓

Execute

↓

Validate Again

6. What is the input of Parser Agent?

“The input is raw vulnerability scanner output, primarily in JSON format. These reports can come from scanners such as Trivy, Snyk, Grype, OWASP, ZAP, and other supported scanners.”

7. What is the output?

“The Parser Agent produces a normalized CSV dataset containing vulnerability information in a standardized tabular structure. Example fields include CVE ID, severity, package name, installed version, fixed version, CVSS score, and description.”

8. Why CSV as output?

Interviewer:

“Why did you convert JSON to CSV?”

Turn carefully bold:

“The purpose of the CSV is to provide a standardized tabular representation that can be easily consumed by downstream processing and analysis stages. Regardless of the original scanner's JSON structure, the downstream components receive the same normalized representation.”

Don't claim CSV is inherently better than JSON. The key point is **standardization for your downstream pipeline.**

9. What exactly does “normalization” mean here?

“Normalization means mapping the different scanner-specific fields and structures into a common set of vulnerability fields. For example, regardless of how a scanner represents severity, package name, installed version, or CVE information, the Parser Agent extracts those values into our standardized output fields.”

Example:

Scanner A:

"package": "openssl"

Scanner B:

"PackageName": "openssl"

Scanner C:

"artifact": "openssl"

After parsing:

Package Name

↓

openssl

10. How does the LLM generate the parser?

 **Very likely technical question**

Tum bolo:

“The agent first analyzes the structure of the raw scanner report and identifies the relevant vulnerability entry points. This structural information is provided to the LLM along with the parsing requirement. The LLM then generates scanner-specific parsing logic that extracts the required fields from that particular report structure.”

Important distinction:

LLM doesn't directly parse the entire file and magically output CSV.

It:

Analyzes structure → Generates parsing logic → Executes logic → Validates result

11. Why not ask the LLM to directly convert JSON to CSV?

🔥 Excellent follow-up question.

“We could ask the LLM to directly transform the data, but generating executable parsing logic provides a more structured and repeatable approach. The generated parser can be executed against the input and its output can be validated. If it fails, the error information can be fed back to the LLM and the parser can be regenerated.”

This also makes your **self-healing mechanism** easier to explain.

12. What happens if the generated parser fails?

★ **Must know**

“The parser execution failure is captured along with the relevant error context. That information is provided back to the LLM. The LLM then generates a revised parser, which is executed and validated again. So instead of immediately failing the entire pipeline, the Parser Agent has a self-correction loop.”

Flow:

Generate Parser

↓

Execute

↓

Error

↓

Capture Error

↓

LLM Analysis

↓

New Parser

↓

Execute Again

13. What if parser executes successfully but produces empty output?

This is a **very good interviewer question**.

Your documentation explicitly says this case is handled.

“Execution success alone is not sufficient. We also validate the extracted data. If the generated parser produces empty results, that is considered a parsing failure. The failure context is sent back to the LLM and a new parser is generated.”

🔥 Remember this distinction:

Code execution successful ≠ Parsing successful

14. How do you validate the parser output?

Based on your documentation:

“We validate whether the generated parser successfully extracted the expected vulnerability data. Empty results or runtime errors indicate that the parser has failed and should enter the self-correction cycle.”

Don't invent specific validation rules such as exact schema libraries or row-count thresholds unless they're actually documented/implemented.

15. What happens if the parser keeps failing?

“The system retries parser generation and validation, but the loop is bounded by a retry threshold. If the threshold is reached without successful extraction, the parsing task is considered unsuccessful instead of retrying indefinitely.”

This is important because otherwise interviewer can ask:

“Can your agent get stuck forever?”

Answer:

“No. We use a retry threshold to prevent an infinite self-correction loop.”

16. Why is self-healing important for Parser Agent?

“Because scanner formats can be complex and the generated parsing logic may not be correct on the first attempt. A self-correction loop allows the system to use actual runtime errors or validation failures as feedback and automatically improve the parser instead of requiring a developer to manually debug every new scanner format.”

17. What is the advantage over static parsers?

Interviewer: “Compare your approach with traditional parsing.”

Static Parser	Parser Agent
Manually written	Dynamically generated
Scanner-specific code	Adaptive
New scanner → new parser	Agent analyzes new structure
Failure requires developer intervention	Self-correction
Maintenance overhead	Reduced manual parser maintenance

Answer:

“The main advantage is adaptability. A static approach requires manually maintaining parser logic for every scanner, whereas our Parser Agent dynamically generates parsing logic based on the actual input structure and can self-correct when the first generated parser fails.”

18. Is the Parser Agent fully autonomous?

Careful answer:

“The parsing workflow is designed to be automated, including parser generation, execution, validation, and retry. However, it is bounded by validation rules and a retry threshold, so it doesn't continue indefinitely.”

19. What happens after Parser Agent finishes?

★ Important architecture question.

“Once the scanner data has been successfully normalized and exported, the standardized vulnerability dataset becomes the input for the downstream vulnerability management pipeline, where vulnerabilities can be normalized further if required, enriched, prioritized, and eventually sent to the remediation workflow.”

High-level:

Scanner

↓

Parser Agent

↓

Normalized CSV

↓

Normalization / Enrichment

↓

Prioritization

↓

Remediation Queue

🔥 20. Why is Parser Agent important to the overall system?

“Parser Agent is the entry point that makes heterogeneous scanner data usable by the rest of the system. Without standardization, every downstream agent would have to understand every scanner's unique format. By converting all scanner outputs into a common structure, the downstream normalization, enrichment, prioritization, and remediation components can operate consistently.”

🔥 This is a **very strong architecture-level answer**.

21. What if a scanner changes its JSON structure?

This is exactly where your approach shines.

“A static parser might break and require manual code changes. With our adaptive Parser Agent, the new report structure can be analyzed again and the LLM can generate parsing logic based on the updated structure. The generated parser is then validated before its output is accepted.”

22. What if a completely new scanner is introduced?

“The system is designed to reduce the need for manually writing a new parser for every scanner. The agent analyzes the new scanner's report structure, identifies the vulnerability entry points, generates the required parsing logic, executes it, and validates the output. If parsing fails, it uses the failure information for self-correction.”

Don't say **“zero code changes are guaranteed”**. Your document says the approach enables support for multiple scanners without manually written parsers for each tool.

23. Most important question — Explain Parser Agent in 1 minute

Ye answer practice karna:

“The Parser Agent is responsible for converting heterogeneous security scanner reports into a standardized format. Different scanners like Trivy, Snyk, Gype, OWASP and ZAP produce different JSON structures, so maintaining a static parser for every scanner would create significant maintenance overhead. Our Parser Agent first discovers and loads the scanner report, analyzes its structure to identify the vulnerability entry points, and uses an LLM to generate scanner-specific parsing logic. The generated parser is then executed and its output is validated. If it produces empty results or encounters a runtime error, the error context is sent back to the LLM, which generates an improved parser and the process is retried. Once successful, the extracted vulnerability information is exported into a standardized CSV format that can be consumed by downstream agents.”

24. Interviewer: “What is the innovative part?”

Very important for a project like yours.

“The key innovation is the adaptive and self-healing parsing mechanism. Instead of maintaining a manually coded parser for every scanner, the system dynamically generates scanner-specific parsing logic based on the actual report structure. More importantly, it validates the generated parser and uses runtime failures or empty output as feedback to automatically regenerate the parser.”

25. Interviewer: “What are the risks of using an LLM for parsing?”

Excellent advanced question.

“The main risk is that an LLM-generated parser may incorrectly understand the structure or extract the wrong fields. That's why we don't directly trust the generated parser. We execute it, validate its output, and use failures as feedback. We also have a retry threshold so that the agent doesn't continue indefinitely.”

26. Interviewer: “Does Parser Agent use AI for every file?”

Be careful.

Your documentation says it **discovers files and loads raw JSON reports**, but doesn't specify exactly whether one LLM call is made per file or whether files are batched.

So say:

“The agent discovers the available scanner output files and processes their structures to generate the required parsing logic. The exact batching or LLM-call strategy depends on the implementation.”

Don't invent.

27. Interviewer: “What happens if the input JSON itself is corrupted?”

The document doesn't explicitly specify corrupted JSON handling.

So interview-safe answer:

“A corrupted or unreadable input would need to be treated as an input-level failure rather than a normal parser failure. The documented self-correction loop specifically handles generated-parser runtime errors and empty extraction results. For corrupted input, the appropriate behavior would be to capture the input error and flag that scanner report for investigation.”

This is a **better answer** than pretending your document implemented something it doesn't say.

28. Interviewer: “Why use an LLM instead of a generic JSON parser?”

“A generic JSON parser can read JSON syntax, but it doesn't know which nested structures represent the vulnerability information across different scanner schemas. Our LLM-based approach is used to analyze the scanner-specific structure and generate the extraction logic needed for that schema.”

Normalization Agent

1. What is the Normalization Agent?

Interviewer: “What is the Normalization Agent?”

Tum bolo:

“The Normalization Agent is responsible for converting parsed vulnerability data from different scanners into a common enterprise-wide schema. Although the Parser Agent extracts the vulnerability information successfully, different scanners can still use different column names, severity formats, data types, and metadata structures. The Normalization Agent standardizes these differences so that downstream components such as prioritization, remediation, validation, and reporting can work with a consistent dataset.”

Short answer:

“Parser Agent extracts the data; Normalization Agent standardizes the extracted data into a common schema.”

2. Why do we need a Normalization Agent if Parser Agent already exists?

 **Very important question**

Interviewer:

“If Parser Agent already gives you CSV, why do you need another agent?”

Answer:

“The Parser Agent solves the extraction problem, not the standardization problem. After parsing, different scanners can still produce different column names, severity labels, formats, and metadata. For example, one scanner may call a field package, another may use component, and another may use a different name. The

Normalization Agent maps these scanner-specific representations into our common enterprise schema.”

Remember:

Parser Agent

↓

Extract vulnerability information

↓

Scanner-specific CSV

↓

Normalization Agent

↓

Common Enterprise Schema

3. Parser Agent vs Normalization Agent

🔥 🔥 **Must prepare**

Parser Agent

Normalization Agent

Raw scanner output → structured data Parsed data → common schema

Handles JSON structures

Handles schema differences

Identifies vulnerability entry points

Maps fields

Generates parser logic

Generates normalization logic

Output is parsed CSV

Output is standardized CSV

Solves extraction

Solves consistency

Interview answer:

“Parser Agent answers: ‘Where is the vulnerability data in this scanner's report?’

Normalization Agent answers: ‘How should that data be represented in our enterprise schema?’”

This is probably the **best one-line distinction**.

4. What is the input of Normalization Agent?

“The input is the parsed CSV files generated by the Parser Agent.”

So:

Trivy JSON

↓

Parser

↓

Trivy Parsed CSV

↓

Normalization Agent

Same for Snyk, Gype, OWASP, ZAP, etc.

5. What is the output?

“The output is standardized normalized CSV files, which are then merged into a unified working dataset that can be consumed by downstream prioritization and remediation components.”

Flow:

Normalized Trivy CSV

+

Normalized Snyk CSV

+

Normalized Gype CSV

+

...

↓

Unified Working Dataset

↓

Prioritization

6. Explain the complete workflow.

★ Most important question

Interviewer:

“How does your Normalization Agent work?”

Tum bolo:

“First, the agent discovers the parsed CSV files generated by the Parser Agent. It analyzes their column names, data types, and sample records to understand how the scanner represents each field. Then it uses an LLM to dynamically generate normalization logic that maps scanner-specific fields into our standardized schema. The generated logic is executed and the output is validated. If the output is invalid, empty, or the normalization code encounters a runtime error, the failure context is provided back to the LLM, which generates improved normalization logic and retries. After successful normalization, the individual normalized datasets are exported and merged into a unified working dataset.”

Complete flow:

Parsed CSV Files

↓

Discover Files

↓

Analyze:

- Column Names
- Data Types
- Sample Records

↓

LLM

↓

Generate Normalization Logic

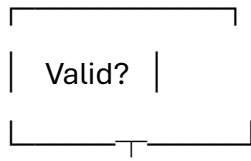
↓

Transform Fields

↓

Validate Output

↓



Yes | No

|

↓

Export

|

↓

Merge Datasets

|

↓

Unified Dataset

Failure:

Normalization Failure

↓

Capture Error

↓

LLM

↓

Improved Logic

↓

Retry

7. What does “standardization” actually mean?

“Standardization means converting scanner-specific representations into the same field names, formats, and expected data types defined by our enterprise schema.”

For example:

Scanner A:

package → openssl

Scanner B:

component_name → openssl

Scanner C:

artifact → openssl

↓

Standard Schema:

component → openssl

Similarly, severity could be represented differently by different scanners, but the normalized dataset provides a consistent field such as:

severity_raw

8. What is your standardized schema?

 **Very important — interviewer may ask directly.**

Your core schema contains:

Field	Meaning
vuln_id	CVE ID or scanner-specific identifier
severity_raw	Original scanner severity
ip	Affected IP

Field	Meaning
hostname	Target hostname
port	Affected port
instance_id	Cloud instance identifier
component	Vulnerable software/package
affected_version	Installed vulnerable version
cvss_score	CVSS score
cvss_vector	CVSS vector
description	Vulnerability description
references	Related references/advisories

If interviewer asks:

“Why keep severity_raw instead of simply converting everything to High/Medium/Low?”

“We preserve the original scanner severity as severity_raw so that the source information is not lost during normalization. Downstream prioritization can then use this raw value along with other enrichment and scoring information.”

9. Why is vuln_id not always just CVE?

This is a **good technical question**.

Your schema explicitly says:

vuln_id = **CVE ID or scanner-specific identifier**

So answer:

“Not every scanner finding necessarily has a CVE identifier. Therefore, our schema allows vuln_id to contain either a CVE ID or a scanner-specific vulnerability identifier. This prevents us from losing findings that don't map directly to a CVE.”

This is important for your architecture.

10. Why do you keep both CVSS score and severity?

“Severity and CVSS score provide different levels of information. severity_raw preserves the scanner's original severity classification, while cvss_score provides the numerical CVSS value when available. Keeping both allows downstream prioritization to use the available information without losing the original scanner context.”

11. Why do you keep cvss_vector?

“The CVSS vector provides the detailed characteristics behind the CVSS score. Keeping it preserves the underlying scoring information and provides additional context for vulnerability analysis and prioritization.”

12. Why are IP, hostname, port and instance ID all needed?

 Good architecture question.

“These fields identify the affected asset and its context. An IP identifies the network address, hostname provides the system identity, port identifies the affected network service, and instance ID can uniquely identify a cloud instance. Keeping these fields helps correlate a vulnerability with the actual asset that needs remediation.”

13. Why use an LLM for normalization?

“Because the mapping between scanner-specific fields is not always fixed or identical. Different scanners can use different names, structures, and representations. The LLM can analyze the column names, data types, and sample records and dynamically generate the transformation logic required for that particular input.”

14. Why not hardcode the field mappings?

 Very likely.

“Hardcoded mappings would work for known scanner formats, but they increase maintenance when a scanner changes its output schema or a new scanner is introduced. Our dynamic approach analyzes the actual input and generates the required normalization logic, reducing the need to manually maintain mappings for every scanner.”

Static:

if scanner == Trivy:

map A → B

if scanner == Snyk:

map C → D

Dynamic:

CSV

↓

Analyze columns + samples

↓

LLM generates mapping

↓

Validate

15. How does the LLM know which fields to map?

“The agent provides the LLM with information about the input CSV, including column names, data types, and sample records, along with the target standardized schema. The LLM uses that information to determine how the source fields correspond to the target fields and generates the normalization logic.”

This directly follows your documented working principle.

16. What happens if the LLM maps a field incorrectly?

 Important.

“The generated normalization logic is not accepted blindly. Its output is validated. If the output is invalid or empty, or if the generated logic produces a runtime error, the failure context is sent back to the LLM and improved normalization logic is generated.”

17. What if normalization produces an empty dataset?

“An empty dataset is considered a normalization failure. The system captures that failure, provides the context to the LLM, generates improved normalization logic, and retries until successful normalization or until the retry threshold is reached.”

🔥 Remember:

Empty output = Failure

18. What if the generated normalization code has a runtime error?

“The runtime error is captured and passed back as feedback to the LLM. The LLM generates a revised normalization logic, which is executed and validated again.”

19. How do you prevent infinite self-correction?

“We use a retry threshold. The agent can automatically retry normalization when validation fails, but it does not retry indefinitely. Once the retry threshold is reached, the normalization process is considered unsuccessful.”

20. Why merge all normalized datasets?

★ Important.

Suppose:

Trivy → normalized_trivy.csv

Snyk → normalized_snyk.csv

Grype → normalized_grype.csv

Then:

↓

Unified Working Dataset

↓

Prioritization Agent

Answer:

“The purpose of merging is to provide downstream components with one consistent working dataset instead of requiring them to process each scanner

separately. Once all findings follow the same schema, prioritization and remediation logic can operate uniformly across all scanner sources.”

21. What happens after Normalization Agent?

“The normalized dataset becomes the input for downstream vulnerability enrichment and prioritization. The system can then enrich findings with additional intelligence and determine which vulnerabilities should enter the remediation queue.”

22. What is the biggest advantage of your Normalization Agent?

“The biggest advantage is consistency. Regardless of which scanner produced the finding, downstream components receive the same schema and can process the vulnerability in the same way.”

23. What is the difference between parsing and normalization?

🔥 🔥 Interviewer definitely can ask this.

Best answer:

“Parsing is about extracting information from the scanner's original structure, while normalization is about representing that extracted information consistently. For example, the Parser Agent may extract a package name from a nested JSON object. The Normalization Agent then decides that this value belongs in the standard component field.”

Remember:

Parsing = Extract

Normalization = Standardize

24. Can Normalization Agent work without Parser Agent?

Based on your documented architecture:

“In our defined workflow, the Normalization Agent consumes the parsed CSV files generated by the Parser Agent, so Parser Agent is the upstream component. The Normalization Agent is designed around that standardized intermediate input.”

Don't claim that it independently accepts raw JSON unless your implementation actually supports it.

25. What if two scanners report the same vulnerability?

This is where you need to be careful.

The document you've provided explains **normalization**, but it does **not explicitly define deduplication logic inside the Normalization Agent**.

So interview-safe answer:

“Normalization ensures that the records follow the common schema. Deduplication is a separate concern unless it is explicitly implemented as part of the downstream pipeline. I would not consider two identical normalized records automatically deduplicated just because they share the same schema.”

🔥 This is a technically mature answer.

26. What if a scanner doesn't provide CVSS?

Another good question.

Your schema defines `cvss_score`, but the provided documentation doesn't specify the exact missing-value strategy.

This will handle by enrichment Agent.

🔥 27. Most Important: Explain Normalization Agent in 1 minute

Practice this:

“The Normalization Agent takes the parsed CSV files generated by our Parser Agent and converts them into a common enterprise-wide schema. The reason we need this stage is that even after parsing, different scanners can use different column names, severity labels, data types, and metadata structures. The agent first analyzes the input columns, data types, and sample records, and then uses an LLM to dynamically generate normalization logic that maps those fields to our standardized schema. The generated logic is executed and validated. If it produces invalid or empty output or encounters a runtime error, the failure context is sent back to the LLM, which generates improved logic and retries. Once successful, the normalized datasets are exported and merged into a unified working dataset for downstream prioritization and remediation.”

🔥 28. “What is innovative about your Normalization Agent?”

“The key advantage is adaptive schema mapping. Instead of maintaining hardcoded field mappings for every scanner, the agent analyzes the actual input structure and dynamically generates the normalization logic. Combined with validation and self-correction, this makes the normalization process more adaptable to different scanner formats.”

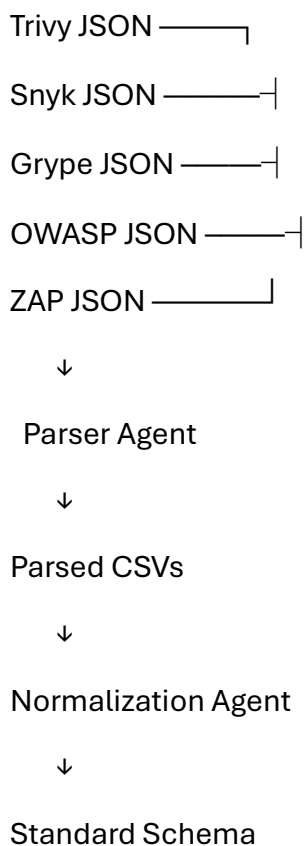
🧠 Parser + Normalization — VERY IMPORTANT

Interviewer may ask you to explain **both together**.

Tum ye bolo:

“Parser and Normalization are two separate stages. The Parser Agent deals with scanner-specific report structures and extracts the vulnerability information. The Normalization Agent then takes those parsed outputs and maps the scanner-specific fields into our enterprise-wide schema. So Parser solves the extraction problem, while Normalization solves the consistency problem.”

Full pipeline:



↓

Unified Dataset

↓

Prioritization

↓

Remediation

★ Golden line:

“Parser tells us what the scanner found; Normalization tells the rest of the system how to represent that finding consistently.”

Vulnerability Enrichment Agent – Part 1

1. Interviewer: What is the Vulnerability Enrichment Agent?

Tum bolo:

“The Vulnerability Enrichment Agent is responsible for validating and enriching the vulnerabilities received from the scanner pipeline. Its first job is to make sure that each finding has a valid standard CVE identifier. After that, it enriches the vulnerability with authoritative NIST-based information such as CVSS score, CVSS vector, fix version, and published date.”

One-line version:

“It converts scanner findings into CVE-based, information-rich vulnerability records.”

2. Why do we need this agent?

★ Very important.

“The scanner output is not always complete or standardized. Some findings may have missing, blank, or proprietary vulnerability identifiers. If we don't have a valid CVE, it becomes difficult to consistently enrich and prioritize the vulnerability. So this agent first resolves the vulnerability identity and then adds authoritative vulnerability metrics.”

The documented workflow explicitly checks for missing, blank, or non-standard IDs before enrichment.

3. Explain the complete workflow

★ Most important question.

Tum bolo:

***“First, the agent reads the normalized CSV and checks whether each vulnerability has a valid CVE. If the CVE is missing or invalid, the finding is sent to Gemini, which uses the vulnerability title and description to infer the likely CVE. If the CVE cannot be recovered, the record is moved to a rejected records file.

For valid CVEs, the agent first checks our local PostgreSQL cache. This database contains synchronized NVD information, so we can retrieve CVSS score, CVSS vector, fix version, and published date without calling the external API every time.

If a CVE is missing from the local cache, the agent batches those missing CVEs and makes concurrent requests to the NVD API. Finally, it merges the database and API results, writes the enriched information back to the CSV, and asynchronously batch-upserts newly retrieved records into PostgreSQL for future runs.”**

4. Explain the architecture diagram

Is diagram ko interviewer ke saamne **left-to-right story** mein explain karo:

Input CSV

↓

Read & Check CVE

↓

Missing CVE?

└— No → PostgreSQL Cache

↓

CVSS / Fix / Date

↓

└ Yes → Gemini

↓

CVE Inference

↓

Unrecoverable → Rejected



PostgreSQL Cache



Missing in Cache?

└─ No → Metrics

└─ Yes → NVD API



Missing Metrics



Merge + Batch Upsert



Enriched CSV

Interview mein:

“The agent follows a database-first strategy. We first validate the identity, then use the local database as the primary source for enrichment. The external NVD API is only used for CVEs that are missing from our cache. This reduces external API dependency and improves runtime performance.”

The database-first and NVD fallback behavior is explicitly documented.

5. Why do you check CVE first?

🔥 Likely question.

“Because CVE acts as the standardized vulnerability identifier that we use to retrieve authoritative vulnerability intelligence. Without a valid CVE, we cannot reliably perform the NVD lookup or consistently connect the finding to the rest of our vulnerability intelligence pipeline.”

6. What if CVE is missing?

This is a **very important part** of your architecture.

“We don't immediately discard the finding. We send it to Gemini as a fallback. Gemini reads the vulnerability title and description and attempts to infer the appropriate CVE. If it still cannot identify a reliable CVE, the finding is rejected and stored separately in the rejected records file.”

Flow:

Missing CVE



Gemini



Can identify CVE?



Yes No



Continue Reject



rejected_records

7. Why Gemini here?

Interviewer:

“Why did you use an LLM to identify CVEs?”

“Because a scanner finding may contain a vulnerability title and description even when the scanner doesn't provide a standard CVE identifier. An LLM can use that descriptive context to infer the likely CVE instead of immediately discarding the finding.”

Important:

Gemini is a recovery/fallback mechanism here, not the authoritative source.

The authoritative vulnerability metrics then come from the local database/NVD.

8. Can Gemini always identify the correct CVE?

“No. Gemini is only a fallback mechanism. If the agent cannot reliably recover the CVE, the record is rejected rather than forcing an uncertain mapping. This avoids introducing incorrect vulnerability identities into the downstream pipeline.”

This is a **good security-oriented answer**.

9. What is the role of PostgreSQL here?

★ Important.

“PostgreSQL acts as our local cache for vulnerability intelligence. It is kept updated by a backend synchronization process, so the enrichment agent can retrieve CVSS score, CVSS vector, fix version, and published date locally instead of making an external NVD request for every record.”

10. Why local PostgreSQL instead of calling NVD every time?

🔥 Very important

“Calling an external API for every vulnerability would add network latency, create dependency on API availability, and could also run into rate limits. By maintaining a local PostgreSQL cache, most lookups happen locally and much faster. We only call NVD for the records missing from our local cache.”

Your broader project documentation also explains that local threat-intelligence copies are maintained specifically to avoid runtime network dependency and latency.

11. What information do you get from NVD?

According to this agent's documented Part 1 flow:

“We retrieve base vulnerability metrics such as CVSS score, CVSS vector, fix version, and published date.”

Remember:

CVSS Score → severity metric

CVSS Vector → detailed representation of the CVSS calculation

Fix Version → version containing the fix

Published Date → vulnerability publication date

12. Why do you use a local cache and then NVD API?

This is called a **fallback strategy**.

“We use a database-first strategy. Most CVEs are served from the local cache, and only cache misses are sent to the NVD API. This gives us both performance and completeness.”

One line:

“Local DB for speed, NVD API for missing data.”

13. Why batch the missing CVEs?

🔥 Important technical question.

“Instead of making one network request for every missing CVE, we collect the missing CVEs and process them in batches. This reduces the number of network round trips and improves the overall efficiency of the enrichment stage.”

Your documentation specifically says missing CVEs are **dynamically batched** before concurrent NVD requests.

14. Why concurrent requests?

“The missing CVEs are independent lookup operations, so they can be requested concurrently rather than waiting for one request to finish before starting another. This reduces overall waiting time for the enrichment stage.”

The source describes the NVD request as concurrent.

15. What does “async batch upsert” mean?

🔥 Good technical question.

“After retrieving the missing data, instead of inserting each record individually, we update multiple records in a batch operation. This reduces database round trips and makes synchronization more efficient.”

Your documentation explicitly says the newly retrieved records are **asynchronously batch-upserted** into PostgreSQL.

“Upsert” means:

“If the record already exists, update it; if it doesn't exist, insert it.”

16. Why save the newly fetched NVD data back into PostgreSQL?

★ Important.

“Because the next time the same CVE appears, we can retrieve its information directly from the local cache instead of calling NVD again. This gradually improves the efficiency of future runs and reduces external API dependency.”

17. What happens to unrecoverable vulnerabilities?

“If Gemini cannot identify a reliable CVE, the finding is not silently dropped. It is saved to a rejected records file so that the system maintains visibility into what could not be processed.”

This is important because it supports **traceability**.

18. What is the difference between Parser Agent and Vulnerability Enrichment Agent?

🔥 Very likely.

“The Parser Agent is responsible for extracting vulnerability information from different scanner formats and converting it into a common structure. The Vulnerability Enrichment Agent takes that normalized vulnerability data and adds authoritative vulnerability intelligence such as CVE, CVSS, fix version, and publication information.”

Easy:

Parser Agent:

“What did the scanner report?”

Enrichment Agent:

“What does this vulnerability actually mean and what authoritative information do we know about it?”

19. What is the difference between Part 1 and Part 2?

This is **very important** because your documentation has two vulnerability enrichment stages.

Part 1:

Identity + NVD baseline metrics

Part 2:

Threat intelligence / exploit context

Part 2 uses local threat intelligence to retrieve **EPSS score, KEV flag and exploit counts**, with API fallback when necessary.

So if interviewer asks:

“Why two enrichment stages?”

Tum bolo:

“We separate baseline vulnerability information from dynamic threat intelligence. Part 1 establishes the vulnerability identity and authoritative NVD metrics, while Part 2 adds real-world exploit intelligence such as EPSS, KEV and exploit evidence. This separation keeps the pipeline modular and makes the two enrichment concerns easier to manage.”

20. Does this agent prioritize vulnerabilities?

✗ Not directly.

Very important.

“No. This agent enriches the vulnerability data. The enriched information is then consumed by the prioritization stage, where technical severity, exploit intelligence, and asset criticality are combined to calculate the remediation priority.”

The documentation identifies the prioritization agent as the component that mathematically combines technical data, threat intelligence and business context.

21. Most Important — Explain this agent in 1 minute

This is the answer you should **practice speaking**:

****“The Vulnerability Enrichment Agent is responsible for validating and enriching the normalized scanner findings. First, it checks whether each finding has a valid CVE identifier. If the CVE is missing or proprietary, we use Gemini as a fallback to infer the CVE from the vulnerability title and description. If we cannot recover it, we move the finding to a rejected records file.**

For valid CVEs, we follow a database-first approach. We query our local PostgreSQL cache, which provides metrics such as CVSS score, CVSS vector, fix version, and published date. If a CVE is missing from the local cache, we batch those missing CVEs and make concurrent requests to the NVD API. Finally, we merge the database and API results, write the enriched data back to the CSV, and asynchronously batch-upsert the newly fetched records into PostgreSQL so that future runs can use the local cache.”**

 22. “Why is this architecture better?”

“The architecture combines correctness, performance, and resilience. Gemini helps recover missing vulnerability identities instead of immediately discarding them. PostgreSQL provides fast local lookups for most records, while NVD is used only as a fallback. Batch and concurrent processing reduce network and database overhead, and rejected records are preserved for traceability.”

Full flow:

CSV

↓

CHECK CVE

↓

Missing?

└─ No ───────────────────┘

| ↓

└─ Yes → Gemini → CVE

↓

PostgreSQL Cache

↓

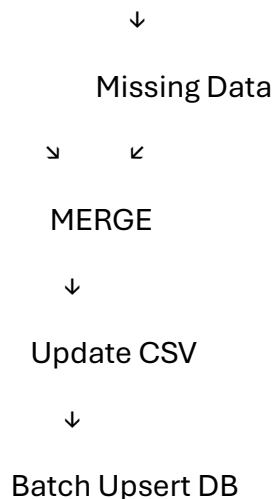
Cache Miss?

↙ ↘

No Yes

↓ ↓

Metrics NVD API



Vulnerability Intel Agent

1. What is the Vulnerability Intel Agent?

Interviewer: “What is the Vulnerability Intel Agent?”

Tum bolo:

“The Vulnerability Intel Agent is the threat intelligence layer of our vulnerability management system. It takes the already enriched vulnerabilities and adds real-world exploit intelligence, such as EPSS score, CISA KEV status, and exploit counts. This information is later used by the prioritization engine to determine which vulnerabilities are more urgent to remediate.”

Short version:

“It adds real-world exploit intelligence to vulnerabilities so that prioritization is based not only on severity, but also on actual exploit risk.”

2. Why did you need Part 2 after Part 1?

🔥 Very important.

“Part 1 gives us the baseline vulnerability information, such as CVSS, fix version, and publication details. But CVSS alone doesn't tell us whether attackers are actively exploiting the vulnerability. Part 2 adds dynamic threat intelligence like EPSS, KEV and exploit evidence, which gives us a better understanding of the actual risk.”

Remember this distinction:

Part 1 → What is the vulnerability?

Part 2 → How dangerous is it in the real world?

This separation is explicitly reflected in the documentation. Part 1 retrieves CVSS, CVSS vector, fix version and published date, while Part 2 adds EPSS, KEV and exploit evidence.

3. Explain the complete workflow

★ Most important question

Tum bolo:

“The process starts with an already enriched CSV. In the background, a nightly cron job maintains a local threat-intelligence mirror by downloading large datasets from sources such as FIRST EPSS, CISA KEV and ExploitDB into PostgreSQL. During runtime, the agent first queries this local cache and retrieves the EPSS score, KEV flag and exploit count for the vulnerabilities. If some records are missing from the cache, the agent fetches the missing information through batched API requests. It then merges the available intelligence, assigns the exploit evidence, writes the final values back to the CSV, and batch-upserts any newly retrieved records into PostgreSQL for future runs.”

Complete flow:

Enriched CSV

↓

Nightly Threat Intel Sync

↓

Local PostgreSQL Cache

↓

Query EPSS / KEV / Exploit Data

↓

Missing Data?

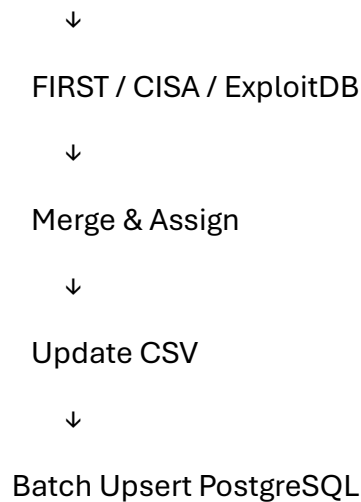
↙ ↘

No Yes

↓

↓

Use Cache Batched API Fetch



4. What is the Nightly Intel Cron Sync?

“The Nightly Intel Cron Sync is a background process that periodically downloads global threat-intelligence datasets and stores them in our local PostgreSQL cache. According to our documentation, it downloads more than 250,000 records covering the CISA KEV catalog, FIRST EPSS data, and ExploitDB repositories.”

Why is this important?

“It allows the runtime enrichment agent to query local data instead of downloading the same large datasets every time a vulnerability needs to be processed.”

5. Why don't you directly call EPSS and KEV APIs during runtime?

 **Very likely question**

“Calling external APIs for every vulnerability would introduce network latency, rate-limit dependency, and availability risks. So we maintain a local intelligence mirror in PostgreSQL and use external APIs only when a required record is missing from the cache.”

Your documentation specifically explains that local caching avoids runtime dependence on external services and provides much faster lookups.

Best one-liner:

“We do the heavy data download offline and keep runtime enrichment lightweight and local.”

6. What is EPSS?

Interviewer:

“What is EPSS?”

“EPSS stands for Exploit Prediction Scoring System. It provides a probability-based estimate of how likely a vulnerability is to be exploited. In our project, we use the EPSS score as one of the indicators of exploit likelihood.”

Project-specific:

“The agent retrieves the EPSS score and passes it into the later prioritization stage.”

Your documented Part 2 workflow explicitly retrieves EPSS values from the local intelligence mirror and uses the FIRST API as fallback.

7. What is CISA KEV?

“CISA KEV stands for Known Exploited Vulnerabilities. It is a catalog of vulnerabilities that are known to have been exploited in the wild. In our system, we represent the KEV presence as a flag, and a vulnerability being present in KEV is treated as strong exploit evidence.”

Your documentation says that if a vulnerability is flagged on the CISA KEV list, it receives the **highest exploit flags** in the evidence aggregation step.

8. What is ExploitDB data used for?

“ExploitDB provides exploit-related evidence. In our workflow, exploit counts are associated with the vulnerability and used as an additional signal when evaluating exploit risk.”

The documentation explicitly mentions ExploitDB as one of the datasets downloaded by the nightly intelligence sync and exploit counts being extracted during runtime.

9. Why use all three — EPSS, KEV and ExploitDB?

 Important.

“They provide different types of threat evidence. EPSS gives us exploit likelihood, KEV tells us whether the vulnerability is known to be exploited in the wild, and exploit counts provide additional exploit evidence. Combining these signals gives the prioritization engine more context than relying only on CVSS severity.”

Easy memory:

Signal	Tells us
CVSS	Technical severity
EPSS	Exploit likelihood
KEV	Known active exploitation
ExploitDB	Available exploit evidence

10. What happens if a vulnerability is missing from the local cache?

“The agent identifies the missing records and performs targeted batched API requests to retrieve the required intelligence. For EPSS, it can fetch missing values from FIRST.org, while CISA KEV information is also retrieved to fill the gaps. The returned information is then merged into the final dataset.”

11. Why batch the API requests?

“Instead of making one API request per vulnerability, we group missing vulnerabilities and fetch them in batches. This reduces the number of network round trips and makes the enrichment process more efficient.”

12. What does “Merge & Assign” mean?

 Good technical question.

“After retrieving the threat-intelligence signals from the local cache and any fallback APIs, the agent combines them into a single record for each vulnerability. It assigns the corresponding EPSS score, KEV flag, and exploit evidence so that the downstream prioritization stage can use them together.”

The documentation also states that KEV presence gets the highest exploit evidence flag.

13. Does the Vulnerability Intel Agent itself decide final priority?

 No.

Very important.

“No. The Intel Agent provides threat intelligence signals. The final priority is calculated later by the Prioritization Agent, which combines technical vulnerability information, exploit intelligence, and business context such as asset criticality.”

Your documentation describes the Prioritization Agent as the component that combines technical data, threat intelligence and business context into a final priority score.

14. How does this information help prioritization?

“Suppose two vulnerabilities have similar CVSS severity. If one has a very high EPSS score or is present in CISA KEV, it represents a higher real-world threat. The prioritization engine can therefore give it a higher remediation priority, especially when the affected asset is also business-critical.”

Ye example explain karne ke liye good hai, but exact formula तभी बताना जब तुम्हारे implementation ka formula ready ho.

15. What is the advantage of local intelligence cache?

🔥 Important architecture question.

“The local cache gives us faster runtime lookups, reduces dependency on external APIs, avoids repeatedly downloading large datasets, and improves resilience if external intelligence services are temporarily unavailable.”

Your documentation explicitly highlights speed, resilience and operational security benefits of keeping threat intelligence local.

16. What happens if external APIs are down?

“If the required intelligence is already present in the local PostgreSQL mirror, the runtime pipeline can continue using that cached data. Only missing records require external fallback requests. So the local cache reduces the impact of external service outages.”

17. Why is this called a “Threat Intelligence Engine”?

“Because it doesn't just store vulnerability metadata. It correlates vulnerabilities with real-world attack signals such as exploitation probability, known exploitation status, and exploit evidence. That transforms raw vulnerability data into actionable security intelligence.”

18. Difference between Part 1 and Part 2

Must know

Part 1 — Vulnerability Enrichment Part 2 — Vulnerability Intel

Validates CVE	Uses already normalized/enriched CVEs
Gemini fallback for missing CVE	Threat intelligence correlation
NVD baseline metrics	EPSS
CVSS	CISA KEV
Fix version	Exploit counts
Published date	Real-world exploit context

Interview-ready:

“Part 1 establishes the vulnerability identity and its authoritative baseline information. Part 2 adds dynamic exploit intelligence so that the system knows not only what the vulnerability is, but also how likely or proven it is to be exploited.”

19. What exactly is stored in PostgreSQL?

For Part 2, your schema documents fields such as:

- epss_score
- kev_flag
- exploit_exists
- exploit_count
- last_updated

These are maintained as vulnerability intelligence data.

Interview answer:

“We maintain threat-intelligence fields such as EPSS score, KEV flag, exploit information and update timestamps in the vulnerability-intelligence data store.”

20. Most Important — Explain Vulnerability Intel Agent in 1 minute

Ye answer practice karo:

“The Vulnerability Intel Agent is our threat intelligence engine. It takes the enriched vulnerability CSV and adds real-world exploit intelligence. First, a background cron job maintains a local PostgreSQL mirror of large intelligence datasets such as FIRST EPSS, CISA KEV and ExploitDB. During runtime, the agent queries this local cache to retrieve the EPSS score, KEV flag and exploit counts for each vulnerability. If some data is missing, it performs targeted batched API requests to retrieve the missing records. It then merges all the available intelligence, assigns the exploit evidence, updates the CSV, and batch-upserts newly retrieved records into PostgreSQL for future runs. This enriched threat intelligence is then consumed by the prioritization engine.”

 **21. Interviewer: “Why did you build a local threat-intelligence mirror?”**

Best answer:

“We wanted the runtime remediation pipeline to be fast and resilient. Instead of depending on external APIs for every vulnerability, we pre-sync large intelligence datasets into PostgreSQL. This allows the agents to perform local lookups quickly, reduces API latency and rate-limit dependency, and allows the workflow to continue using cached intelligence even when external endpoints are temporarily unavailable.”

Asset Enrichment Agent – Part 1

1. What is the Asset Enrichment Agent?

Interviewer: **“What is the Asset Enrichment Agent?”**

Tum bolo:

“The Asset Enrichment Agent is responsible for resolving the identity of the assets affected by vulnerabilities. It takes the normalized vulnerability CSV and maps each finding to an existing asset in the database using identifiers such as instance ID, hostname, or IP address. For existing assets, it enriches the CSV with stored asset metadata, and for new assets, it creates the required database record so that every vulnerability is associated with a valid asset ID.”

One-line:

“Its main job is to answer: which actual asset does this vulnerability belong to?”

2. Why did you split Asset Enrichment into two agents?

Very important

“We deliberately split asset enrichment into two agents because they have different responsibilities. Agent 1 is the data resolution layer—it identifies the asset and assigns the correct asset ID. Agent 2 is the scoring layer—it calculates the asset's criticality. This separation keeps each agent's responsibility clean and prevents asset identification logic from being mixed with business-criticality scoring.”

Remember:

Agent 1 → WHO is the asset?

Agent 2 → HOW IMPORTANT is the asset?

3. What is the input and output of Agent 1?

“The input is `working.csv`, which comes from the normalization stage and contains vulnerability findings along with asset identity fields such as instance ID, hostname, and IP. The output is an enriched CSV where every valid row has an `asset_id`, and existing assets are enriched with their known metadata.”

4. Explain the complete workflow.

Most important question

****“First, the agent loads and validates the CSV. It rejects rows where all three asset identity fields—instance ID, hostname, and IP—are missing. Then it performs bulk database lookups for instance IDs, hostnames, and IPs. Each row is matched using a strict priority order: instance ID first, hostname second, and IP last.**

For existing assets, the agent retrieves the stored asset metadata and writes it into the CSV. For new assets, it creates skeleton records with the basic identity information and assigns the authoritative asset ID returned by the database. It also updates the `last_seen` timestamp for matched assets. Finally, it saves the enriched CSV and passes it to Agent 2 for asset criticality scoring.”**

5. Why instance ID first?

 Very likely.

“We use instance ID as the first matching key because it is an immutable cloud identifier and is generally more reliable for identifying a specific cloud instance than hostname or IP.”

6. Why hostname second?

“Hostname is used after instance ID because it is generally more stable than an IP address. In infrastructure environments, IPs can change due to DHCP, cloud reprovisioning, or other infrastructure changes, while hostnames are often more stable.”

7. Why is IP address last?

 Important.

“An IP address is less reliable as a primary asset identity because it can change. The same logical asset can receive a different IP over time, so using IP as the first identifier could result in incorrect asset mapping.”

Memory trick:

Instance ID → most reliable

Hostname → relatively stable

IP → can change

8. What happens if multiple identifiers are available?

“We follow a strict priority order. The agent first tries instance ID. If that doesn't produce a match, it tries hostname, and if that also fails, it uses IP. This gives us deterministic and consistent asset resolution.”

9. Why use bulk queries instead of querying the database for every row?

 Very important technical question

“A per-row database query would create a large number of round trips and would become expensive when processing thousands of vulnerability findings. Instead, we collect all unique instance IDs, hostnames, and IP addresses and perform one bulk query for each identifier type. We then build in-memory maps and resolve each row locally.”

Example:

Bad:

Row 1 → DB

Row 2 → DB

Row 3 → DB

...

Row 10,000 → DB

Your design:

All Instance IDs → 1 DB query

All Hostnames → 1 DB query

All IPs → 1 DB query

↓

In-memory maps

🔥 This is an excellent optimization to explain.

10. What is the purpose of the in-memory maps?

“After the bulk queries, we build lookup maps in memory. This allows each CSV row to be resolved against the retrieved database records without making another database call.”

11. What happens with an existing asset?

“If the asset already exists in the database, we retrieve its stored information and populate the corresponding CSV row. That can include fields such as inferred role, exposure score, environment, cloud type, dependency information, and existing criticality score. Because the asset is already scored, Agent 2 can skip recomputing its criticality.”

12. What happens when a new asset is discovered?

🔥 Important

“For a new asset, Agent 1 creates a skeleton database record containing only the basic identity information, such as asset ID, hostname, instance ID, IP address,

first-seen and last-seen timestamps. The full asset profile and criticality calculation are handled later by Agent 2.”

13. Why don't Agent 1 and Agent 2 both calculate everything?

“Because we want a clean separation of responsibilities. Agent 1 is responsible for resolving identity and making sure every asset has a valid database record. Agent 2 is responsible for inference and criticality scoring. This reduces duplication and makes the workflow easier to maintain.”

14. Why generate a UUID for a new asset?

“A new asset needs a unique identifier before it can be represented in the database. So we generate a UUID locally as an initial candidate ID.”

But here's the important part:

“Do you trust the generated UUID?”

“Not directly. Because another process could insert the same asset between our initial lookup and insert operation, we don't assume the locally generated UUID is authoritative. After the batch insert, we query the database again using the asset identity fields and use the actual asset ID returned by the database as the authoritative ID.”

🔥 This is a very strong technical answer.

15. Why do you re-query the database after inserting?

“It protects us from race conditions. If another process inserts the same asset concurrently, our ON CONFLICT DO NOTHING insert may not use our locally generated UUID. Re-querying the database ensures that the CSV gets the actual authoritative asset ID stored in the database.”

This is one of the best implementation details in your Agent 1.

16. What does ON CONFLICT DO NOTHING achieve?

“It prevents a duplicate insert from failing the transaction when the same asset already exists due to a concurrent operation. Instead of creating a duplicate record, the insertion is skipped, after which we re-query the database to retrieve the existing authoritative asset ID.”

17. What fields are stored for a new skeleton?

Tum exact answer de sakte ho:

“The skeleton contains `asset_id`, `hostname`, instance ID, IP address, `first_seen`, and `last_seen`. The full asset profile, such as role, exposure, dependency and criticality information, is populated by Agent 2.”

18. What happens to rows where asset identity is completely missing?

“If instance ID, hostname, and IP are all missing, the row cannot be reliably mapped to an asset. So we reject it early and write it to `rejected_asset_records.csv` instead of allowing an orphan record to propagate further into the pipeline.”

🔥 This demonstrates **data quality control**.

19. Why reject these rows early?

“Because continuing with a row that has no asset identity would create an orphan vulnerability record or an unreliable asset mapping. Early rejection keeps downstream data clean and makes the failure visible through the rejected records file.”

20. What is `last_seen`?

“`last_seen` represents the most recent time when an asset was observed in the scanner data. For existing assets, we update this timestamp whenever the asset appears again in a scan.”

Why important?

“It helps us track whether an asset is still active or being observed by the scanning pipeline.”

21. Why is everything async?

🔥 Very likely question

“The agent is I/O-heavy because it reads CSV files, opens database connections, executes queries, and writes files. Asynchronous programming allows these I/O operations to be handled without blocking the event loop. Blocking operations

such as file operations and psycopg2 database calls are offloaded to a thread pool.”

22. Does async mean your six nodes run in parallel?

 **Important trap**

No.

“No. The pipeline is intentionally sequential. Async here means the I/O operations are non-blocking; it does not mean that the graph nodes execute in parallel. Each node waits for the previous node to complete before the next one starts.”

This answer is especially important because interviewers often confuse **async** with **parallel execution**.

Remember:

Async ≠ Parallel

23. How do you handle transient database errors?

“Database connections and operations are wrapped in a retry mechanism. For transient errors such as connection failures, operational errors, or timeouts, the agent retries up to three times with a five-second delay between attempts.”

24. Do you retry every database error?

 **No.**

“No. We distinguish transient and non-transient errors. Connection failures and timeouts are retried because they may recover. Errors such as bad SQL or constraint violations are considered non-transient and are surfaced immediately rather than repeatedly retrying them.”

 **Very good interview answer.**

25. What happens if Agent 1 fails?

“Each node updates the workflow state, and if a node fails, the routing logic checks the status and can short-circuit the graph to the END state. The failure is captured rather than allowing invalid data to continue through the pipeline.”

26. How does Agent 1 hand over to Agent 2?

“After Agent 1 saves the enriched CSV, its final node directly invokes the compiled LangGraph graph of Agent 2, passing the working CSV path and a fresh initial state. From that point, Agent 2 takes responsibility for asset criticality scoring.”

This is a very project-specific detail. Remember it.

27. Why use LangGraph for Agent 1 instead of a normal Python script?

🔥 Good question.

“We could implement the process as a flat script, but LangGraph gives us explicit state management and error isolation. Each operation is represented as a node, and failures can be routed based on the current state. This makes the workflow easier to control and integrate with the larger agentic architecture.”

28. What does Agent 1 NOT do?

This is useful because it prevents you from mixing Agent 1 and Agent 2.

“Agent 1 does not calculate asset criticality. Its responsibility ends after asset identity resolution, database enrichment, new-asset insertion, timestamp updates, and passing the enriched CSV to Agent 2.”

29. Difference between Asset Agent and Vulnerability Enrichment Agent

🔥 Very likely cross-question.

“The Vulnerability Enrichment Agent enriches the vulnerability itself—for example, CVE, CVSS and NVD information. The Asset Lookup Agent enriches the affected infrastructure asset by identifying which asset the vulnerability belongs to and retrieving its asset metadata.”

Easy:

Vulnerability Agent → What is wrong?

Asset Agent → Where is it happening?

🔥 30. Most Important — Explain Agent 1 in 1 minute

Ye answer **directly interviewer ko bolna:**

****“Asset Enrichment Agent Part 1 is responsible for resolving asset identity. It takes the normalized vulnerability CSV containing instance ID, hostname and IP, and first validates whether enough identity information is available. Rows with no usable identity are rejected and stored separately.**

For valid rows, we perform bulk database lookups for instance IDs, hostnames and IPs, and then match each row using the priority order instance ID, hostname and finally IP. Existing assets are enriched with their stored metadata and their last_seen timestamp is updated.

For new assets, we create skeleton records containing the basic identity information. Because of possible concurrent inserts, we don't blindly trust the locally generated UUID. After the batch insert, we re-query the database and use the actual asset ID returned by the database as authoritative.

Finally, we save the enriched CSV and hand it over to Agent 2, which is responsible for calculating asset criticality.”**

Asset Enrichment Agent – Part 2 / Asset Criticality Agent

1. What is Asset Criticality Agent?

Interviewer: “What is the Asset Criticality Agent?”

Tum bolo:

“The Asset Criticality Agent determines how important an infrastructure asset is from a business and operational perspective. It analyzes factors such as the asset's role, network exposure, environment, cloud type, and dependency level, and combines them into a single asset criticality score. This score is later used by the prioritization engine along with vulnerability severity and threat intelligence.”

One-line:

“It tells us how important the affected asset is, not just how severe the vulnerability is.”

2. Why do you need asset criticality?

🔥 Most important question

“Technical severity alone is not enough for vulnerability prioritization. The same vulnerability can have very different impact depending on which asset it affects. For example, a vulnerability on a production authentication server is much more

important than the same vulnerability on an isolated development machine. So we calculate asset criticality to bring business context into prioritization.”

Your database design explicitly recognizes that the same CVE can receive different priority depending on the affected asset—for example, a public-facing authentication server versus an isolated development machine.

3. Why did you split Asset Lookup and Asset Criticality?

“The two agents have separate responsibilities. Agent 1 resolves the asset identity and retrieves existing metadata. Agent 2 performs the inference and criticality scoring for new or incomplete assets. This separation avoids mixing identity resolution with business-risk scoring.”

Easy:

Agent 1 → Identify the asset

Agent 2 → Evaluate the asset

4. What is the input?

“Agent 2 receives the enriched working.csv produced by Agent 1. It contains asset identity and the metadata already resolved from the database.”

5. What happens to assets that are already scored?

★ Important optimization.

“We don't recalculate their criticality. If all required criticality fields are already populated, Agent 2 skips that asset and preserves the existing score. This avoids unnecessary computation and, more importantly, avoids unnecessary LLM calls.”

6. Why do you process unique asset IDs instead of every vulnerability row?

🔥 Excellent question.

Suppose one server has 50 vulnerabilities.

You **don't** want:

Asset A → calculate criticality 50 times

Instead:

Asset A → calculate criticality once



Apply result to all rows

Interview answer:

“Criticality is an asset-level property, not a vulnerability-level property. So if the same asset appears in 50 vulnerability rows, we evaluate that asset only once and reuse its criticality information. This significantly reduces duplicate computation and unnecessary LLM calls.”

7. Explain the complete inference pipeline.

★ Most important

Tum bolo:

“For every new asset, we first infer its role, then determine its network exposure, identify whether it is production or non-production, detect the cloud type, and calculate its dependency level. These signals are then combined into the final asset criticality score. The process is deterministic-first, meaning we try to infer as much information as possible using predefined rules before using an LLM.”

Pipeline:

New Asset



Role Inference



Exposure Inference



Environment



Cloud Detection



Dependency Scoring



8. What is Role Inference?

“Role inference tries to determine what kind of service or system the asset represents. We use a priority chain: first open ports are mapped to likely roles, then component keywords are checked, and finally hostname hints are considered.”

Examples from your documentation include:

- DATABASE
 - AUTH
 - WEB SERVER
 - API BACKEND
 - CI/CD
 - MANAGEMENT/BASTION
-

9. Why use open ports for role inference?

“Open ports provide a useful deterministic signal about what service an asset is running. For example, certain ports can strongly suggest a database, web service, SSH service, or other infrastructure role. So we use ports before weaker textual signals.”

10. Why component keywords and hostname hints?

“If port information doesn't provide enough evidence, we look at the vulnerable component or package name and then the hostname. This provides additional deterministic signals before we fall back to an LLM.”

11. What is Exposure Score?

“Exposure score represents how accessible the asset is from the network. The system assigns different values based on whether the asset has a public IP, a private but exposed service, a normal private address, or localhost.”

According to your implementation:

Condition	Exposure
Public IP	1.0
Private but internet-exposed service	0.8
Standard private IP	0.5
Localhost	0.2
Unknown/fallback	0.5

12. Why is exposure important?

🔥 Important.

“A vulnerability on an internet-exposed asset is generally more concerning than the same vulnerability on an isolated internal asset. So exposure adds network context to the asset's business risk.”

13. How do you infer Environment?

“We inspect the hostname, component information, and tags for indicators that suggest a production or non-production environment.”

For example conceptually:

prod / production → Production

dev / test / staging → Non-production

Don't claim exact keywords unless asked about your implementation.

14. Why is production environment important?

“A vulnerability affecting a production asset can have a greater operational impact than one affecting a development or test asset. Therefore, production status contributes to the dependency and criticality calculation.”

15. What is Cloud Detection?

“Cloud detection identifies whether the asset corresponds to a managed service or infrastructure associated with providers such as AWS, Azure, or GCP. This context can affect the dependency or blast-radius assessment.”

16. What is Dependency Score?

★ Important.

“Dependency score represents how much the organization or other systems depend on that asset. We start from a role-based baseline and then modify it using factors such as internet exposure, production status, high-value cloud service type, and production CI/CD usage.”

Your documented examples include:

AUTH / DATABASE → Critical baseline

ENDPOINT → Low baseline

Then modifiers can increase or decrease the score.

17. Explain Dependency Score with an example.

“For example, an authentication server or a database starts with a high dependency baseline because many systems may rely on it. If that asset is also production and internet-facing, its dependency score can increase further. On the other hand, if an asset is isolated or non-production, the score can be reduced.”

18. What is the final criticality formula?

★ Must memorize exactly

Your documentation gives:

Criticality Score = 0.50 × Exposure Score + 0.50 × Dependency Score

So interview mein:

“Our final asset criticality score is calculated by giving equal weight to exposure and dependency. The formula is 0.50 times the exposure score plus 0.50 times the dependency score.”

Formula:

Example:

Suppose:

Exposure = 1.0

Dependency = 0.8

Then:

Criticality

$= 0.5(1.0) + 0.5(0.8)$

$= 0.9$

19. Why 50%-50%?

Interviewer could ask:

“We wanted the final score to reflect both network exposure and business dependency equally. Exposure captures how reachable the asset is, while dependency captures how important the asset is to the organization's operations.”

⚠ The documentation establishes the formula but **doesn't document a formal experiment proving that 50/50 was optimal**. So don't say “we statistically proved 50/50 was best.”

20. When do you call the LLM?

🔥 **Very important**

A common mistake would be saying:

“We use Gemini for every asset.”

❌ **No.**

Your implementation is **deterministic-first**.

The LLM is called **only if all four conditions are true simultaneously**:

- Role = UNKNOWN
- Environment = UNKNOWN
- Exposure = default unknown 0.5
- Cloud type = NONE

So interview answer:

“We use the deterministic inference pipeline first. We invoke the LLM only when all major deterministic signals fail—role is unknown, environment is unknown, exposure remains at the default 0.5, and cloud type is none. This minimizes unnecessary LLM calls, latency, and cost.”

21. Why not use LLM for every asset?

 **Very likely**

“Using an LLM for every asset would increase latency and cost unnecessarily, especially when deterministic signals are already available. For example, if a port or hostname clearly identifies the asset's role, there is no reason to ask an LLM to infer it again. So we use the LLM only when deterministic inference cannot provide meaningful information.”

This is an excellent architectural decision to highlight.

22. Which LLM do you use here?

According to your implementation:

“For this fallback, we use Google Gemini 2.5 Flash asynchronously.”

The LLM is used only for genuinely ambiguous assets.

23. What does Gemini do here?

“Gemini receives the available asset context, such as hostname, IP, component information, and description, and tries to infer missing contextual fields. After the LLM resolves the missing information, the dependency and criticality scores are recalculated using the newly resolved values.”

24. Why recalculate the score after LLM inference?

 **Good technical question.**

“Because the criticality score depends on the inferred context. If the LLM identifies that an unknown asset is actually a production authentication server, the dependency score changes, which in turn changes the final criticality score. So after every successful LLM inference, we recompute the dependency and criticality values from scratch.”

25. Is the LLM directly calculating the final criticality score?

✗ No.

Important:

“No. The LLM is only used as a fallback for missing contextual signals. The final dependency and criticality calculations still follow our deterministic formula.”

🔥 This is a great way to demonstrate that you're not blindly relying on AI.

26. How do you process multiple new assets?

“All new assets are processed concurrently using `asyncio.gather`. So, if we have 20 new assets, their inference pipelines can run concurrently rather than waiting for one asset to finish before starting the next.”

But remember:

Agent 1 ≠ Agent 2

Agent 2 has **concurrency within the inference node**, whereas Agent 1's nodes themselves are sequential.

27. Does this mean you call the LLM 20 times simultaneously?

Careful answer:

“The inference pipelines for multiple assets can run concurrently. However, the LLM fallback condition is checked independently for each asset, so only assets that satisfy all fallback conditions require an LLM call.”

28. How do you avoid duplicate upserts?

“Before the batch upsert, we deduplicate the assets by `asset_id`, because one asset can appear in multiple vulnerability rows. Each asset is therefore written only once during the upsert.”

29. What is the DB upsert logic?

“We use a batch upsert with `ON CONFLICT (asset_id) DO UPDATE`. Existing assets are updated with newly computed fields, while new assets are inserted.”

Important additional behavior:

“first_seen is preserved, while last_seen is updated to the current timestamp.”

30. Why preserve first_seen?

“first_seen represents when we initially discovered the asset. Overwriting it on every scan would destroy historical information. So we preserve the original discovery timestamp and only update last_seen when we observe the asset again.”

This is a **good audit/history answer**.

31. Why update last_seen every time?

“It tells us the most recent time the asset was observed in the scanning pipeline, which is useful for asset lifecycle tracking and historical consistency.”

32. What is the output of Agent 2?

“The final output is a fully scored working.csv where every row has a valid asset ID, complete asset metadata, and a criticality profile. The computed asset information is also persisted to PostgreSQL. This CSV is then passed to the prioritization stage.”

33. What is the role of Asset Criticality Agent in prioritization?

 **Extremely important**

“The Asset Criticality Agent provides the business-context score that the prioritization engine uses alongside vulnerability severity and threat intelligence. This means the final remediation priority is not based only on CVSS or exploit likelihood; it also considers how important the affected asset is.”

Your documentation explicitly says the criticality score becomes a foundational input to the Prioritization Agent so that business context influences remediation priority.

34. Example — Why criticality matters

Interviewer:

“Give me a practical example.”

Tum bolo:

“Suppose the same high-severity vulnerability exists on two machines. One is an isolated development machine, while the other is a public-facing production authentication server. Their CVSS could be identical, but the second asset should receive a higher remediation priority because it has greater exposure and dependency. Our asset criticality score captures that business context.”

This matches the documented design where the same CVE can have very different priority depending on the affected asset.

35. Most Important — Explain Asset Criticality in 1 minute

Ye answer practice karo:

****“Asset Criticality Agent determines how important an infrastructure asset is from a business and operational perspective. Agent 2 receives the asset-enriched CSV and first skips assets whose criticality fields are already populated, because their scores don't need to be recomputed. For new assets, we use a deterministic inference pipeline that identifies the role, network exposure, environment, cloud type, and dependency level.**

The final criticality score is calculated as 50% exposure plus 50% dependency. We use deterministic rules first, and Gemini 2.5 Flash is invoked only when all major signals are unknown—role, environment, exposure and cloud type. After LLM inference, the dependency and criticality scores are recalculated. New assets are processed concurrently, deduplicated by asset ID, and written back to PostgreSQL using batch upserts. Finally, the fully enriched CSV is passed to the prioritization agent.”**

8.8 Prioritization Agent — Interview Preparation

1. What is the Prioritization Agent?

Interviewer: “What is the Prioritization Agent?”

Tum bolo:

“The Prioritization Agent is responsible for calculating the final remediation priority of each asset-vulnerability pair. Instead of relying only on technical severity like CVSS, it combines vulnerability severity, real-world exploit intelligence, asset criticality, fix availability, and vulnerability age. It produces a single priority score from 0 to 10 and maps it to a priority level such as Low, Medium, High, or Critical.”

One-line:

“It tells us which vulnerability should be remediated first and why.”

2. Why do we need a Prioritization Agent?

🔥 **Very important**

Interviewer:

“Why can't you simply prioritize by CVSS?”

Answer:

“CVSS only represents the technical severity of a vulnerability. It doesn't fully capture whether the vulnerability is being actively exploited, how important the affected asset is to the business, whether a fix is available, or how old the vulnerability is. So we combine these different factors to produce a more context-aware remediation priority.”

Remember:

CVSS → How severe is it technically?

EPSS/KEV → How exploitable is it?

Asset Criticality → How important is the affected asset?

Fix Availability/Age → How actionable and old is it?

3. Explain the complete workflow.

★ **Most important question**

Tum bolo:

****“First, the agent checks the database for an existing priority score using the Asset ID and Vulnerability ID pair. If that pair was already prioritized in a previous run, we reuse the existing score to avoid unnecessary recalculation.**

For new calculations, the agent takes the fully enriched dataset and extracts CVSS, EPSS, KEV, fix availability, vulnerability age, and asset criticality. These values are normalized to a 0 to 1 range and passed into our custom weighted formula.

The final result is converted into a 0 to 10 priority score and mapped to a priority tier such as Low, Medium, High, or Critical. Finally, the result is written back to the CSV and batch-upserted into the database for downstream remediation and Jira tracking.”**

4. Explain the diagram step-by-step

Your diagram shows:

Step 1 — Ingest Enriched Data

The input CSV already contains:

- vulnerability information
- threat intelligence
- business/asset context

So say:

“The Prioritization Agent receives the fully enriched dataset from the previous enrichment and asset-criticality stages.”

Step 2 — Execute Database Cache Check

“Before doing a new calculation, we check whether the same Asset ID and Vulnerability ID pair has already been prioritized. If the score already exists, we reuse it instead of doing the calculation again.”

Why?

“This reduces redundant processing and improves performance.”

Step 3 — Calculate 0–10 Risk Score

This is the core.

“For new calculations, we normalize the input factors to a 0-to-1 range and apply our custom weighted formula. The final weighted value is then converted into a 0-to-10 risk score.”

5. Explain the Priority Formula

  **This is the most important technical question.**

Your documented weights are:

Factor	Weight
Asset Criticality	25%
CVSS	25%

Factor	Weight
--------	--------

EPSS + KEV	40%
------------	-----

Fix Availability + Vulnerability Age	10%
--------------------------------------	-----

Total = **100%**

Interview-ready answer:

“Our priority formula gives 25% weight to asset criticality, 25% to CVSS, 40% to EPSS and KEV together, and 10% to fix availability and vulnerability age. Before applying the formula, all values are normalized to a 0-to-1 scale. The final weighted result is then converted into a 0-to-10 priority score.”

6. Why Asset Criticality gets 25%?

“Asset criticality represents the business impact of the affected infrastructure. A vulnerability on a highly critical production asset can be more urgent than the same vulnerability on a low-value or isolated asset. So we give asset criticality a significant weight to make the prioritization business-aware.”

7. Why CVSS gets 25%?

“CVSS provides the foundational technical severity of the vulnerability. We still consider it strongly, but we don't allow technical severity alone to determine the final remediation priority.”

8. Why EPSS + KEV get 40%?

 Very important.

“We give the highest combined weight to EPSS and KEV because they provide real-world exploit context. EPSS indicates the likelihood of exploitation, while KEV indicates that a vulnerability is known to have been exploited in the wild. This makes exploit intelligence a major factor in determining urgency.”

9. Why Fix Availability + Vulnerability Age get 10%?

“These are operational factors. Fix availability tells us whether a remediation is actionable, while vulnerability age provides additional context about how long the

vulnerability has existed. Together they contribute to the final score without dominating technical severity and exploit intelligence.”

⚠️ Your document specifies the **combined 10%**, but it does not specify an exact split between fix availability and age. So don't invent one.

10. Why normalize everything to 0–1?

🔥 Good technical question.

“The different inputs have different scales. For example, CVSS is on a different scale from EPSS or an asset criticality score. Normalizing them to a common 0-to-1 range allows us to combine them meaningfully using weighted mathematics.”

Simple example:

CVSS → normalized

EPSS → normalized

Asset Criticality → normalized

Fix Availability → normalized

Age → normalized

↓

Weighted Formula

↓

Final Score

11. Why output a 0–10 score?

“A 0-to-10 score gives us a simple and intuitive representation of the calculated risk. It can then be mapped to operational priority levels such as Low, Medium, High, and Critical.”

Your source does **not** specify the exact boundaries for these levels, so don't give invented thresholds unless your actual implementation has them documented.

12. What is an Asset-Vulnerability pair?

🔥 Important because your cache is based on it.

“An asset-vulnerability pair represents a specific vulnerability on a specific asset. The same CVE can have different priorities on different assets because the asset's criticality and context may be different.”

Example:

CVE-XYZ on Dev Server



Lower Priority

CVE-XYZ on Production Auth Server



Higher Priority

Your database documentation explicitly stores the priority at the **asset + vulnerability** level.

13. Why cache based on Asset ID + Vulnerability ID?

“Because priority isn't only a property of the vulnerability. It depends on which asset is affected. So the combination of Asset ID and Vulnerability ID uniquely identifies the prioritization context we need to reuse a previous score.”

14. What happens if the same CVE affects 100 machines?

🔥 Excellent question.

“We don't assign one universal priority to the CVE. We calculate priority for each asset-vulnerability pair because the same vulnerability can have different risk depending on asset criticality, exposure, dependency, and business context.”

This is a **very strong answer**.

15. Example of different priorities

Interviewer:

“Give me a practical example.”

Tum bolo:

“Suppose the same CVE has a high CVSS score on two systems. One system is an isolated development server, while the other is a public-facing production authentication server. The CVSS component could be similar, but the second asset has much higher criticality and exposure, and potentially higher exploit impact. Therefore our prioritization formula can assign a higher priority to the production authentication server.”

16. What happens if a score already exists?

“The agent first checks the `asset_vulnerabilities` table. If an exact Asset ID and Vulnerability ID pair has already been prioritized, it retrieves and reuses the existing score instead of recalculating it.”

This historical caching is explicitly documented.

17. Why is this cache important?

“It avoids redundant computation and improves performance, especially when the same vulnerability findings appear in repeated scanner runs.”

18. Does the Prioritization Agent use an LLM?

 Important.

Based on your documentation:

“No, the core priority calculation is a deterministic mathematical process. It uses normalized factors and predefined weights rather than asking an LLM to decide the priority.”

This is an important design decision.

Why?

“Because prioritization needs to be predictable, explainable, and consistent. A mathematical formula gives us deterministic results for the same inputs.”

 Very strong interview point.

19. Why not use AI/LLM for prioritization?

“We wanted the prioritization decision to be deterministic and auditable. Since the factors and their weights are explicitly defined, the same inputs always produce

the same score. That makes the result easier to explain to security teams and audit later.”

20. What happens after prioritization?

“After calculating the final score, the enriched information is written back to the CSV and persisted in the database. The prioritized findings then enter the downstream remediation workflow, and the corresponding vulnerabilities can also be tracked through Jira.”

Your documentation describes prioritization as the bridge to the downstream remediation agent and Jira lifecycle.

21. How does Prioritization connect all previous agents?

Very important architecture question

Ye answer definitely practice karo:

“The Prioritization Agent brings together the outputs of the previous stages. The Parser and normalization stages provide the vulnerability data, Vulnerability Enrichment provides CVSS and NVD information, Vulnerability Intel provides EPSS and KEV intelligence, and Asset Criticality provides business impact. The Prioritization Agent combines these inputs into a single actionable score.”

Complete chain:

Scanner

↓

Parser

↓

Normalization

↓

Vulnerability Enrichment

↓

Threat Intelligence

↓

Asset Enrichment

↓

Asset Criticality

↓

Prioritization

↓

Remediation

🔥 This is one of the most important architecture flows in your project.

22. What is the difference between CVSS, EPSS, KEV and Asset Criticality?

Interviewer can ask this together.

“CVSS tells us the technical severity of the vulnerability. EPSS estimates exploit likelihood. KEV indicates known exploitation in the wild. Asset criticality tells us how important the affected infrastructure asset is from a business and operational perspective.”

Easy memory:

CVSS → Severity

EPSS → Probability

KEV → Known exploitation

Asset Criticality → Business impact

23. What if a vulnerability has high CVSS but low EPSS?

“The vulnerability would still have a significant severity contribution because of CVSS, but its overall priority would also depend on factors such as asset criticality, KEV status, fix availability, and age. So high CVSS alone doesn't automatically make it the highest-priority item.”

24. What if a vulnerability has moderate CVSS but is in KEV?

🔥 Great scenario question.

“KEV provides strong evidence of real-world exploitation, so the vulnerability can receive a significantly higher priority even if its CVSS score isn't the highest. Our

formula deliberately gives exploit intelligence a larger combined weight to capture this real-world risk.”

25. What if there is no fix available?

Based on the documented formula:

“Fix availability is one of the operational factors in the prioritization formula. So lack of a fix affects the operational component of the score, while the vulnerability can still receive a high priority based on CVSS, exploit intelligence, and asset criticality.”

Don't claim an exact numeric penalty unless the actual formula/code defines it.

26. How do you make prioritization explainable?

 **Very important for security systems**

“Because our prioritization is formula-based, we can explain the score through its contributing factors—asset criticality, CVSS, EPSS/KEV, and operational factors. This is more transparent than asking an LLM to simply return a priority label.”

27. Most Important — Explain Prioritization Agent in 1 minute

Ye answer **directly interviewer ko bolna:**

****“The Prioritization Agent is the mathematical decision-making stage of our pipeline. It receives the fully enriched vulnerability data, including CVSS, EPSS, KEV, fix availability, vulnerability age, and asset criticality. First, it checks whether the same Asset ID and Vulnerability ID pair has already been prioritized and reuses the stored score if available.**

For new calculations, all factors are normalized to a 0-to-1 range and passed through our weighted formula. Asset Criticality contributes 25%, CVSS 25%, EPSS and KEV together contribute 40%, and Fix Availability plus Vulnerability Age contribute 10%. The resulting value is converted into a 0-to-10 priority score and mapped to Low, Medium, High, or Critical. The final result is then stored in the database and passed to the remediation stage.”**

28. Easy Memory Trick

Prioritization ko **5 steps** mein yaad rakho:

Cache → **Extract** → **Normalize** → **Calculate** → **Classify**

Cache → existing Asset+Vuln score?

Extract → CVSS + EPSS + KEV + Asset Criticality + Fix/Age

Normalize → everything 0–1

Calculate → weighted formula

Classify → 0–10 → Low/Medium/High/Critical

The most important formula

Interview mein confidently bolna:

Important: Tumhare document mein **EPSS & KEV combined = 40%** aur **Fix Availability & Vulnerability Age combined = 10%** diya hai. Isliye interviewer agar pooche ki “*EPSS ko kitna aur KEV ko kitna?*”, tab exact split **tabhi batao jab tumhare implemented formula/code mein defined ho**. Source text us split ko specify nahi karta.

5 Killer Cross-Questions

1. “Why is CVSS alone not enough?”

“Because CVSS represents technical severity but doesn't capture real-world exploitation or the business importance of the affected asset.”

2. “Why is asset criticality included?”

“The same vulnerability can have very different impact depending on the asset it affects.”

3. “Why is EPSS/KEV weighted more heavily?”

“Because they provide real-world exploit evidence and likelihood, which helps identify vulnerabilities that require urgent action.”

4. “Why use a deterministic formula instead of LLM?”

“Prioritization needs to be consistent, explainable, and auditable, so a deterministic weighted formula is more appropriate.”

5. “What does your score actually represent?”

“It represents the remediation urgency of a specific vulnerability on a specific asset, considering technical severity, exploit risk, operational factors, and business impact.”

★ **One sentence that connects your entire prioritization design**

“CVSS tells us how bad the vulnerability is, EPSS and KEV tell us how dangerous it is in the real world, asset criticality tells us how important the affected system is, and the Prioritization Agent combines all of these into one actionable remediation score.”

8.9 Remediation Agent — Interview Preparation

1. What is the Remediation Agent?

Interviewer: “What is the Remediation Agent?”

Tum bolo:

“The Remediation Agent is responsible for taking a prioritized vulnerability and automatically generating, testing, validating, and managing its remediation. It first researches the vulnerability, generates precheck, remediation, and validation scripts using an LLM, tests those scripts in an isolated cloned VM, and only after successful validation and human approval does it apply the fix to the original target VM.”

One-line:

“It converts a prioritized vulnerability into a tested and approved production fix.”

2. Why do you need a separate Remediation Agent?

“Prioritization only tells us which vulnerability should be addressed first. The Remediation Agent handles the actual remediation lifecycle—researching the vulnerability, generating the fix, testing it, correcting failures, getting human approval, and finally applying the validated fix to production.”

Simple:

Prioritization Agent → What should we fix first?

Remediation Agent → How do we safely fix it?

3. Explain the complete workflow

🔥 **This is the most important question.**

Tum bolo:

****“Once a vulnerability is selected from the remediation queue, the backend creates a unique workflow thread and a corresponding Jira ticket. The agent then researches the vulnerability using web search and gathers information such as security advisories, vendor documentation, CVE references, patch notes, and remediation guidance.**

Using the original vulnerability data and this research, the LLM generates three scripts: a precheck script, a remediation script, and a validation script. These scripts are stored in a scripts.json file, and the agent also creates a GitHub branch and Pull Request through GitHub MCP tools.

Whenever the PR is created or updated, the CI/CD pipeline executes the scripts on a cloned sandbox VM in the order precheck, remediation, and validation. Logs from all stages are stored in S3.

If execution or validation fails, the logs are sent back to the LLM as feedback. The agent generates an improved remediation, updates the PR, and retries the validation cycle.

Once validation succeeds, the workflow pauses for Human-in-the-Loop review. If the reviewer requests changes, the workflow resumes using GitHub webhook feedback and goes through another remediation cycle. If the PR is approved and merged, the validated scripts are executed on the original production VM.

Finally, the workflow records execution metrics, marks the Jira ticket as resolved, and stores the workflow as completed.”**

This sequence matches the documented remediation workflow.

4. Step 1 — Workflow Initialization

Interviewer: “What happens when remediation starts?”

“When a prioritized vulnerability is selected, the FastAPI backend creates a unique workflow thread for that vulnerability and records its initial state. At the same time, a Jira ticket is created so that the vulnerability has a traceable lifecycle from remediation start to resolution.”

Why thread ID?

“The thread ID gives each remediation execution its own identity and allows us to maintain and resume its state independently.”

5. Why do you create Jira ticket at the beginning?

“Because remediation is a multi-stage process and we need traceability throughout the lifecycle. Jira provides a human-facing record where we can track the vulnerability, remediation progress, HITL review, and final resolution.”

Your Jira workflow is documented as the centralized tracking and approval mechanism.

6. Step 2 — Vulnerability Research

“Why do you research the vulnerability before generating the script?”

“Because the scanner finding alone may not contain enough information to safely generate a remediation. The agent researches trusted security advisories, vendor documentation, CVE references, and patch notes to understand the root cause, affected versions, available fixes, and recommended remediation approaches.”

7. Is Tavily the one generating the remediation?

 No.

“No. Tavily is used for research. It provides external information and remediation references. The LLM uses that research together with the original vulnerability information to generate the remediation scripts.”

Remember:

Tavily → Research

LLM → Reasoning + Script generation

8. Step 3 — Why generate three scripts?

 **Very likely question.**

Your project generates:

Precheck Script

“It collects the current state of the system before making changes.”

Remediation Script

“It applies the actual security patch or configuration change.”

Validation Script

“It confirms whether the vulnerability has actually been remediated.”

Interview answer:

“We separate precheck, remediation, and validation so that we can capture the system state before the change, apply the fix, and then independently verify the result.”

9. Why not generate only one remediation script?

“A single remediation script wouldn't give us the same level of verification and traceability. The precheck establishes the baseline, the remediation performs the change, and the validation script verifies the result. This separation also helps us debug failures more clearly.”

10. What is scripts.json?

“scripts.json is the generated artifact that contains the remediation-related scripts along with the information needed by the CI/CD pipeline. It acts as the bridge between the agent's script-generation stage and the GitHub Actions execution pipeline.”

Your documentation states that the three scripts are stored together in scripts.json, which becomes the CI/CD input.

11. Step 4 — GitHub Automation

“What happens after scripts are generated?”

“The workflow uses GitHub MCP tools to create or update the remediation branch, commit the generated scripts.json, and open a Pull Request. The PR number is then stored with the workflow thread ID in the pr_mappings table so future GitHub webhook events can be associated with the correct remediation workflow.”

12. Why create a Pull Request?

 **Important**

“The Pull Request provides versioning, traceability, and most importantly a controlled Human-in-the-Loop approval point. The generated remediation can be reviewed before it is allowed to affect production.”

13. Why store PR number and Thread ID together?

This is a strong technical question.

“Because the system may have multiple remediation workflows running. When GitHub later sends a webhook for a PR review, comment, or merge, we need to know exactly which remediation workflow that event belongs to. The PR-to-thread mapping gives us that association.”

14. Step 5 — Sandbox Validation

  **One of the most important design decisions**

“Why don't you execute the generated script directly on production?”

“Because the remediation script is AI-generated and could contain an incorrect or unintended command. We first execute it in an isolated cloned VM that represents the target environment. This gives us a safe environment to test the remediation before allowing any production change.”

Your documented workflow explicitly uses a **cloned VM instead of the original production system**.

15. What is the order of execution?

Exactly:

“The scripts execute in the order: Precheck → Remediation → Validation.”

16. Why validate after remediation?

“Because successful execution of a shell script doesn't necessarily mean the vulnerability has been fixed. The validation stage verifies the actual security state after remediation.”

 **Remember:**

Script succeeded ≠ Vulnerability fixed

17. Where are execution logs stored?

“Execution logs from the precheck, remediation, and validation stages are automatically stored in Amazon S3.”

18. Why store logs in S3?

“S3 provides centralized and persistent storage for execution logs and evidence. These logs are also important for the self-correction loop and later auditing.”

19. What happens if remediation fails?

 **Very important**

“If any stage fails, the workflow retrieves the execution logs from S3 and provides them to the LLM as feedback. The LLM analyzes the failure and generates an improved remediation. The updated scripts are then pushed through the GitHub workflow and another validation cycle is triggered.”

Flow:

Execute

↓

Failure

↓

Logs → S3

↓

LLM Analysis

↓

Improved Script

↓

Update PR

↓

CI/CD



Retry

20. What do you mean by Self-Healing?

“Self-healing means the system can use the failure information from a remediation attempt to improve the remediation automatically rather than requiring an engineer to manually rewrite the script after every failure.”

21. Can it retry forever?

 No.

“No. The remediation loop is bounded by a retry limit. If the workflow continues to fail after the allowed retries, it enters a failed or unresolved state rather than looping indefinitely.”

The documentation explicitly says the self-healing loop continues until success or the retry limit is reached.

22. Step 6 — Human-in-the-Loop

 **Most important architectural principle**

“Why is human approval still required?”

“Even after successful automated validation, the final change affects production infrastructure. We therefore retain a human approval gate to provide governance and prevent an automatically generated remediation from reaching production without review.”

23. What happens while waiting for approval?

“The workflow enters the `WAITING_FOR_HUMAN_APPROVAL` state and its current state is persisted in the workflow-state database. The agent can safely stop instead of remaining active while waiting.”

This is a very important point because it demonstrates **efficient stateful orchestration**.

24. How does the workflow resume?

“GitHub sends a webhook when the reviewer comments, approves, or otherwise changes the Pull Request. Our backend uses the stored PR-to-thread mapping to identify the correct workflow and resume it from the preserved state.”

25. What if the reviewer requests changes?

“The reviewer feedback is received through GitHub webhooks. The workflow resumes, incorporates that feedback into the remediation process, regenerates the scripts, and starts another sandbox validation cycle.”

26. What if the reviewer approves the PR?

“Once the Pull Request is approved and merged, the already-validated remediation scripts are executed on the original target VM. This ensures that only human-approved remediations reach production.”

27. What if the reviewer rejects/closes the PR?

“If the Pull Request is closed without approval, the remediation workflow is terminated and the associated Jira ticket is marked unresolved.”

28. Production Deployment

“How is the production fix applied?”

“The exact scripts that successfully passed sandbox validation are executed on the original target VM only after the Pull Request has been approved and merged. This prevents an unvalidated or unapproved script from being deployed to production.”

29. What happens after production execution?

“Once the remediation is successfully applied, the workflow records execution time, token usage, and estimated LLM cost. The workflow state is marked as COMPLETED, and the Jira ticket is marked as Resolved.”

30. What happens when everything fails?

“If the workflow encounters an unrecoverable error or exhausts its retry attempts, it is marked as FAILED. This gives every remediation workflow a clear terminal state and prevents ambiguous or hanging remediation records.”

31. Why record token usage and LLM cost?

 Nice advanced question.

“Because the remediation process is AI-driven, model usage can contribute significantly to operational cost. Tracking token usage, execution time, and estimated LLM cost allows us to measure the efficiency and cost of the remediation workflow.”

Your project documentation also tracks **tokens used, cost, success rate and MTTR** as dashboard/performance metrics.

32. Why use a cloned VM?

Interviewer may ask again, but answer from a different angle:

“The cloned VM gives us isolation and reproducibility. We can test the remediation in an environment that resembles the target system without risking production. It also gives us a controlled place to observe the precheck, remediation, validation and failure behavior.”

33. Why GitHub MCP in remediation?

“The remediation agent needs to interact programmatically with GitHub to create branches, commit the generated scripts, and manage Pull Requests. GitHub MCP provides the agent with these GitHub capabilities through a standardized tool interface.”

Your documentation explicitly states that GitHub MCP tools are used for the GitHub automation stage.

34. Why GitHub Actions?

“GitHub Actions provides the CI/CD automation that executes the generated scripts and validation pipeline whenever the remediation Pull Request is created or updated.”

Your documented workflow says every PR creation or update triggers the CI/CD pipeline.

35. Why S3?

“S3 provides persistent centralized storage for the logs generated during precheck, remediation and validation. These logs are then used both for auditing and as feedback when self-correction is required.”

36. Why Jira?

“Jira provides lifecycle tracking and human approval visibility. The vulnerability gets a ticket at the beginning, the ticket is updated throughout remediation, and it is ultimately marked resolved or unresolved.”

37. Explain the whole Remediation Agent in 1 minute

This answer **must be practiced**:

******“The Remediation Agent starts when a prioritized vulnerability enters the remediation queue. The backend creates a unique workflow thread and a Jira ticket. The agent researches the vulnerability using web search and combines that information with the original finding. The LLM then generates three scripts—precheck, remediation, and validation—and stores them in scripts.json.

The agent uses GitHub MCP tools to create a branch, commit the scripts, and open a Pull Request. The PR triggers the CI/CD pipeline, which runs the scripts on a cloned sandbox VM in the order precheck, remediation, and validation. Logs from every stage are stored in S3.

If execution or validation fails, the logs are sent back to the LLM, which generates an improved remediation and retries the process. Once validation succeeds, the workflow pauses for human approval. Reviewer feedback is received through GitHub webhooks and can restart the workflow for another validation cycle. After approval and merge, the validated scripts are applied to the original production VM. Finally, the workflow records execution metrics, marks the workflow completed, and resolves the Jira ticket.”******

Remediation Agent — 8 Things to Remember

1. Research

Tavily / web → understand vulnerability.

2. Generate

LLM → precheck + remediation + validation.

3. GitHub

MCP → branch + commit + PR.

4. Staging

GitHub Actions → cloned VM.

5. Validate

Precheck → Remediation → Validation.

6. Self-Heal

Failure → S3 logs → LLM → retry.

7. HITL

Success → PR review → approval/rejection.

8. Production

Approved → original VM → completion → Jira resolved.

Most important scenario questions

Q1. “What if the AI-generated remediation script is wrong?”

“That's why we never directly execute it on production. It first runs on the cloned VM, and the vulnerability is independently validated. If it fails, the logs are fed back to the LLM for self-correction. Human approval is also required before production deployment.”

Q2. “What if the script executes successfully but the vulnerability remains?”

“We don't treat script execution success as remediation success. The validation script and scanner results determine whether the vulnerability was actually resolved. If it remains, the workflow enters the failure and self-correction path.”

Q3. “What if human approval takes hours?”

“The workflow doesn't keep consuming resources while waiting. It enters `WAITING_FOR_HUMAN_APPROVAL`, persists its state, and resumes only when the relevant GitHub webhook event arrives.”

Q4. “What if human rejects the remediation?”

“The production deployment does not happen. Reviewer feedback is used to regenerate the remediation, and the workflow goes through sandbox validation again.”

Q5. “What if all retries fail?”

“The workflow reaches a terminal failed state, and the Jira ticket remains unresolved rather than allowing an unsafe deployment.”

★ One sentence that explains the entire design

“Our remediation architecture follows a simple safety principle: generate automatically, test in isolation, self-correct on failure, require human approval, and only then modify production.”

That is probably the **best conceptual sentence** to remember for the Remediation Agent.

9. Database Architecture

Tumhare project mein **5 main PostgreSQL tables** hain:

vulnerabilities

|

↓

vulnerability_intel

|

↓

assets

|

↓

asset_vulnerabilities



vulnerabilities_history

Lekin logically inka relationship aise samjho:

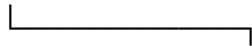
vulnerabilities



vuln_id



vulnerability_intel



assets ——— asset_vulnerabilities



Remediation History

vulnerabilities_history

Sabse important distinction:

vulnerabilities tells us what the vulnerability is.

vulnerability_intel tells us how the threat is evolving.

assets tells us which infrastructure is important.

asset_vulnerabilities tells us how urgent that vulnerability is on that specific asset.

vulnerabilities_history tells us what happened during remediation.

1. vulnerabilities

Interviewer: “What is the purpose of the vulnerabilities table?”

Tum bolo:

“The vulnerabilities table is our canonical reference table for static CVE information. It stores information that defines what a vulnerability is, such as the CVE ID, title, description, CVSS score, CVSS vector, fix version, and published date.”

Tumhare schema mein vuln_id primary key hai.

Important fields:

vuln_id

title

description

cvss_score

cvss_vector

fix_version

published_date

Why keep this information separate?

“We keep static vulnerability information separate from dynamic threat intelligence because the two change at different rates. Basic CVE information such as description and base CVSS is relatively stable, while threat intelligence such as EPSS and KEV status can change frequently.”

 This directly explains why you have **two separate tables**.

2. vulnerability_intel

Interviewer: “What is vulnerability_intel?”

“The vulnerability_intel table stores dynamic threat intelligence associated with a CVE. It contains information such as EPSS score, KEV status, exploit existence, exploit count, and the last time that intelligence was refreshed.”

Fields:

vuln_id

epss_score

kev_flag

exploit_exists

exploit_count

last_updated

3. Why not store EPSS and KEV inside vulnerabilities?

🔥 Very important

“Because they are dynamic signals. EPSS scores can change regularly, and CISA KEV status can change as new vulnerabilities are added. Keeping them separate allows us to refresh the threat intelligence independently without modifying the static CVE reference record.”

This is one of the strongest database-design questions for your project.

4. What is the relationship between these two tables?

“vulnerability_intel.vuln_id references vulnerabilities.vuln_id. So each threat-intelligence record is associated with the corresponding CVE in the main vulnerability table.”

Conceptually:

vulnerabilities

CVE-123

CVE-456

CVE-789

|

| vuln_id

↓

vulnerability_intel

CVE-123 → EPSS, KEV...

CVE-456 → EPSS, KEV...

5. assets

Now comes the **business context** part.

Interviewer: “What is the assets table?”

“The assets table stores the identity and business-context information of infrastructure assets. It contains identifiers such as IP address, hostname and instance ID, along with inferred role, exposure, environment, cloud type, dependency information, asset criticality, and scan timestamps.”

Important fields:

asset_id

ip_address

hostname

instance_id

inferred_role

exposure_score

environment

cloud_type

dependency_level

dependency_score

asset_criticality_score

first_seen

last_seen

6. Why do you need asset_criticality_score in the assets table?

 Important.

“Criticality is an asset-level property. It describes how important the asset is from a business and operational perspective, independent of a particular vulnerability. That's why the criticality score belongs in the assets table.”

7. Why store IP, hostname and instance ID together?

“Because different scanners may identify the same asset using different identifiers. We use a priority-based matching strategy—instance ID first, hostname second, and IP last—to reliably resolve the asset identity.”

This connects directly to your Asset Enrichment Agent.

8. Why first_seen and last_seen?

“first_seen tells us when the asset was first discovered, while last_seen tells us when it was most recently observed. This gives us asset lifecycle information without overwriting the original discovery timestamp.”

9. asset_vulnerabilities

 **This is probably the most important table in your database design.**

Interviewer: “What is the purpose of asset_vulnerabilities?”

“This is the junction table between assets and vulnerabilities. But it does more than simply establish the relationship. It represents a specific vulnerability on a specific asset and stores the contextual information and final priority for that asset-vulnerability pair.”

Important fields:

asset_id

vuln_id

severity_raw

severity_score

component

affected_version

port

fix_available

priority_score

priority_level

first_seen

last_seen

And:

asset_id + vuln_id = composite primary key

10. Why composite primary key?

🔥 Very likely.

“Because the same vulnerability can exist on multiple assets, and one asset can have multiple vulnerabilities. The combination of asset ID and vulnerability ID uniquely represents one asset-vulnerability relationship.”

Example:

Asset A + CVE-101

Asset A + CVE-102

Asset B + CVE-101

Each combination is a different record.

11. Why not use only vuln_id as the primary key?

“Because a CVE is not unique at the asset level. The same CVE can affect hundreds of machines, and each affected asset can have a different risk context and priority.”

🔥 This is extremely important.

12. Why is priority_score stored here?

“Because priority is contextual. The same CVE can have a different priority on different assets depending on asset criticality, exposure, dependency, and threat intelligence. So the final contextual priority belongs to the asset-vulnerability relationship.”

This is the key database-design principle.

Example:

CVE-123

|

└─ Production Auth Server → Priority 9.5

|

└─ Dev Laptop → Priority 4.2

Same vulnerability, different priority.

13. Difference between vulnerabilities and asset_vulnerabilities

🔥 **Must prepare**

“The vulnerabilities table stores global information about the CVE itself. The asset_vulnerabilities table stores the context of that CVE on a specific asset, including component, affected version, fix availability, and the final priority score.”

One-line:

vulnerabilities → What is the CVE?

asset_vulnerabilities → What does this CVE mean on this asset?

14. Difference between assets and asset_vulnerabilities

“The assets table describes the asset independently, including its criticality and dependency. The asset_vulnerabilities table describes the relationship between that asset and a particular vulnerability.”

Easy:

assets → Machine

asset_vulnerabilities → Machine + Vulnerability

15. vulnerabilities_history

Now the final table.

Interviewer: “What is vulnerabilities_history?”

“The vulnerabilities_history table stores the execution history of remediation workflows. Each record represents a remediation attempt for a specific asset and vulnerability, including its outcome, execution time, token usage, cost, and duration.”

Important fields:

thread_id
asset_id
cve_id
score
resolved
cost
token
start_time
end_time
time_taken

16. Why do you need a separate history table?

🔥 Very important.

“Because current vulnerability state and historical remediation execution are different concerns. The asset-vulnerability table tells us the current vulnerability and priority context, while the history table records what happened during remediation attempts over time.”

17. What is thread_id?

“thread_id uniquely identifies a remediation workflow execution. It allows us to connect the remediation history to a specific workflow run.”

This connects your database with LangGraph.

18. Why store token and cost?

“Because the remediation workflow uses LLMs, so token consumption and estimated LLM cost are important operational metrics. Storing them in the history table allows us to analyze the efficiency and cost of remediation workflows.”

Your documentation explicitly identifies execution cost, token usage and execution time as remediation history metrics.

19. Why store start_time, end_time, and time_taken?

“They allow us to measure remediation duration and calculate performance metrics such as mean time to remediate. They also provide traceability for when a specific remediation workflow started and completed.”

20. Why PostgreSQL for history?

“Remediation history is long-term data and is useful for reporting and dashboard analytics, so it belongs in the persistent PostgreSQL layer rather than the lightweight active workflow database.”

21. Very Important: Explain all 5 tables together

Interviewer: “Can you explain your database schema?”

Ye answer practice karo:

****“We designed the PostgreSQL schema around five distinct responsibilities. The vulnerabilities table is the canonical reference for static CVE information such as description and CVSS. The vulnerability_intel table stores dynamic threat intelligence such as EPSS, KEV status and exploit counts, so it can be refreshed independently.**

The assets table stores infrastructure identity and business context such as role, exposure, environment, dependency and criticality. The asset_vulnerabilities table is the relationship between a vulnerability and an asset, and this is where we store context-specific information and the final priority score for that asset-vulnerability pair.

Finally, vulnerabilities_history stores remediation execution history, including workflow ID, resolution status, execution time, token usage and cost. This separation keeps the data model clean and allows us to handle static data, dynamic intelligence, business context, current prioritization, and historical execution independently.”**

22. Why did you separate static and dynamic vulnerability data?

“Because they have different update frequencies. Static CVE information such as title, description and base CVSS changes relatively rarely, while threat intelligence such as EPSS and KEV can change frequently. Separating them allows independent synchronization and avoids unnecessary updates to the canonical vulnerability record.”

🔥 23. Why don't you store priority in vulnerabilities?

This is a **killer question**.

“Because priority is not a property of the CVE alone. It depends on the affected asset. The same CVE can have different priority levels on different machines. Therefore, priority belongs to the `asset_vulnerabilities` table, where we have both the asset and vulnerability context.”

🔥 24. Why don't you store remediation history in `asset_vulnerabilities`?

“Because `asset_vulnerabilities` represents the current asset-vulnerability relationship, while a vulnerability may have multiple remediation attempts over time. The history table allows us to store each execution independently, including success or failure, time, cost, and token consumption.”

25. How does the complete data flow look?

This is worth memorizing:

Scanner Output

↓

Parser / Normalization

↓

vulnerabilities

+

vulnerability_intel

+

assets

↓

asset_vulnerabilities

↓

Prioritization

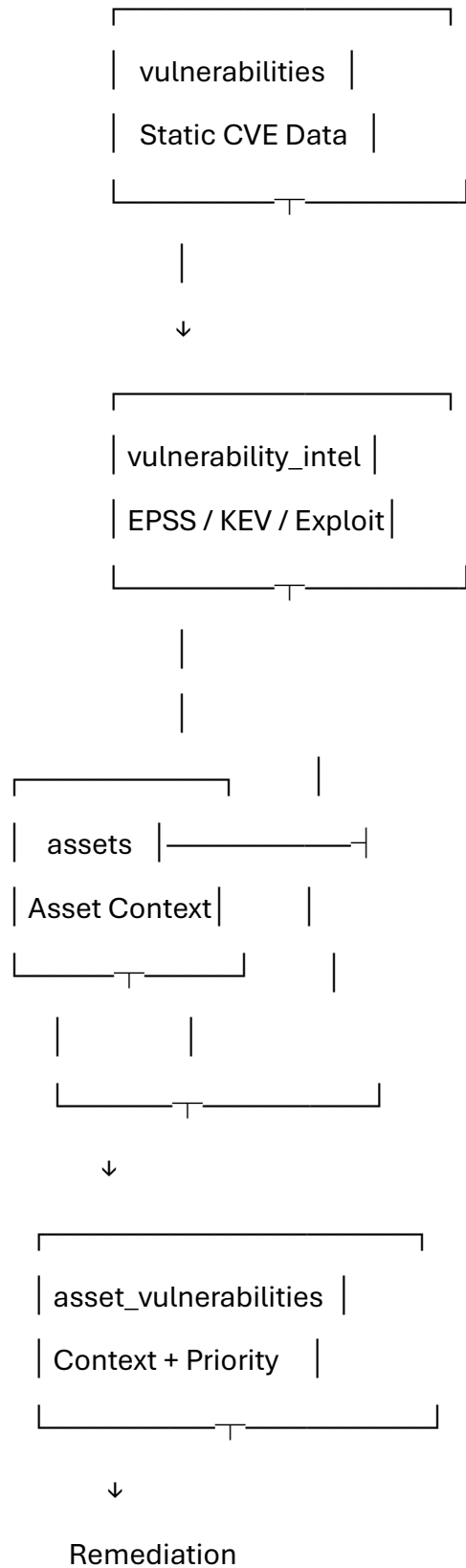
↓

Remediation

↓

vulnerabilities_history

More accurately:



↓

vulnerabilities_history
Execution + Cost + MTTR

🔥 26. What is the difference between current state and history?

“Current-state tables tell us what exists and its current risk context, while the history table tells us what happened during previous remediation executions. Separating these prevents historical records from bloating the active operational tables.”

27. Why synthetic asset data?

🔥 This is an important project-specific question.

Your documentation says production enterprise asset inventory was unavailable, so the platform uses a **synthetically generated asset dataset** representing realistic enterprise attributes such as hostname, IP, instance ID, role, environment, cloud provider and business-criticality indicators. Real-world vulnerability intelligence is used alongside it.

Interview answer:

“We did not have access to production enterprise asset inventory, so we used a **synthetically generated asset dataset** to simulate realistic enterprise environments without exposing sensitive organizational information. For vulnerability intelligence, we used real-world data from sources such as NVD, EPSS, CISA KEV and ExploitDB.”

28. Why use real vulnerability data with synthetic assets?

“This combination allows us to test realistic vulnerability enrichment, prioritization and remediation scenarios while keeping enterprise infrastructure information confidential.”

29. Why PostgreSQL instead of storing everything in SQLite?

🔥 Important cross-question.

“PostgreSQL is our persistent long-term data layer for vulnerability records, assets, prioritization and remediation history. SQLite is used separately for active workflow state and LangGraph checkpointing. This keeps long-running workflow state lightweight while keeping historical and analytical data in a robust relational database.”

🔥 **30. What happens if the same vulnerability appears in multiple scanners?**

This connects database schema with your parser/normalization.

“The scanner outputs are normalized and deduplicated before the vulnerability and asset relationships are persisted. The goal is to represent the underlying vulnerability consistently rather than creating duplicate records simply because multiple scanners reported it.”

Be careful: your documentation does mention scanner deduplication, but don't claim a specific deduplication key unless asked and you know the implementation.

🧠 Easy Memory Trick — 5 Tables

Just remember:

Vulnerability → Intelligence → Asset → Relationship → History

1. vulnerabilities

→ What is the vulnerability?

2. vulnerability_intel

→ How risky/exploitable is it right now?

3. assets

→ Which infrastructure exists and how important is it?

4. asset_vulnerabilities

→ How urgent is this vulnerability on this specific asset?

5. vulnerabilities_history

→ What happened when we tried to fix it?

★ **Best one-minute database answer**

“Our PostgreSQL schema is divided into five logically separate tables. The vulnerabilities table stores static CVE information, while vulnerability_intel stores dynamic threat intelligence such as EPSS, KEV and exploit counts. The assets table stores asset identity and business context like exposure, dependency and criticality. asset_vulnerabilities is the junction table that combines an asset with a vulnerability and stores the context-specific priority score. Finally, vulnerabilities_history stores every remediation execution with its workflow ID, result, execution time, token usage and cost. This separation keeps static data, dynamic intelligence, business context, current risk and historical execution cleanly separated.”

★ **One line to remember for the interviewer:**

“The key design principle is that vulnerability severity is global, but vulnerability priority is contextual to the affected asset.”

That's probably the **single most important database-design concept** in your project.

10. Cron Jobs

1. What is a Cron Job?

Interviewer: “What is a cron job?”

“A cron job is a scheduled background task that runs automatically at a predefined time or interval. In our project, we use cron jobs to periodically synchronize vulnerability and threat-intelligence data into our local PostgreSQL database, so that the runtime agents don't have to repeatedly depend on external APIs.”

Simple:

“Cron jobs automate periodic background data synchronization.”

2. Why did you use Cron Jobs in your project?

★ **Very important.**

“Our system processes potentially thousands of vulnerabilities, so querying external APIs during every remediation workflow would introduce latency, rate-limit dependency, and availability issues. Instead, we use scheduled background jobs to keep local vulnerability and threat-intelligence data updated. The agents can then perform fast local database lookups during runtime.”

3. Why did you create TWO cron jobs?

🔥🔥 Most important question

“We separated them because they manage two different types of data. The NVD Sync Cron maintains relatively stable technical vulnerability information such as CVSS, descriptions, and fix versions. The Vulnerability Intelligence Cron maintains dynamic threat information such as EPSS, CISA KEV status, and exploit counts. These two datasets have different update behavior, so keeping them separate makes the architecture cleaner and allows them to be refreshed independently.”

Easy memory:

Cron 1 → What is the vulnerability?

Cron 2 → What is happening with that vulnerability in the real world?

10.1 NVD Sync Cron Job

4. What is the purpose of the NVD Sync Cron?

“The NVD Sync Cron maintains a local mirror of the NVD vulnerability data. Instead of making an NVD API request for every vulnerability during a scan, we periodically synchronize the NVD data into our local vulnerabilities table.”

5. Why can't you directly call NVD API for every vulnerability?

🔥 Important.

“At enterprise scale, a scanner can produce a very large number of vulnerabilities. Making an external NVD API request for every CVE would create a major bottleneck due to network latency and API rate limits. By maintaining a local mirror, our agents can query PostgreSQL directly at database speed.”

Your document describes this specifically as the reason for the local mirror.

6. What happens on the FIRST NVD Sync?

This is a good technical question.

“On the first run, the job performs a full historical synchronization of NVD CVE data from 2002 to the present, year by year.”

7. What happens on subsequent runs?

🔥 Important.

“After the initial import, the synchronization becomes incremental. The job fetches only CVEs that NVD has added or modified since the last successful synchronization timestamp.”

Why?

“It avoids downloading the entire NVD dataset repeatedly and significantly reduces unnecessary processing.”

8. What if the NVD sync job is down for 3 days?

★ Nice scenario question.

“Because the job uses the last successful synchronization timestamp, the next run can fetch the complete gap instead of only fetching the most recent day's data. So if the job was down for three days, the next run catches up with all changes from that period.”

This is a strong **incremental synchronization** design.

9. What data does NVD Sync store?

“It updates the static vulnerability fields such as CVE ID, title, description, CVSS score, CVSS vector, fix version, and published date in the vulnerabilities table.”

10. What does “upsert” mean in NVD Sync?

“Upsert means insert the record if it doesn't exist, or update the existing record if it already exists. This lets the synchronization job safely maintain the local vulnerability mirror.”

11. Why don't you overwrite fix_version with NULL?

🔥 Very good detail.

“Because an existing valid fix version should not be lost just because a subsequent NVD response doesn't contain a fix version. So if the incoming value is null, we preserve the previously stored fix version.”

This is an example of **data preservation during synchronization**.

12. Why does static vulnerability data stay local?

“Because base vulnerability information such as title, description and CVSS is relatively stable after publication. There is no need to query the external source repeatedly during every scan. Keeping it locally gives us fast lookups and removes API dependency from the runtime pipeline.”

13. What is the difference between vulnerabilities and vulnerability_intel?

🔥 This question will almost certainly come.

“The vulnerabilities table stores the relatively stable technical baseline from NVD, while vulnerability_intel stores dynamic threat signals such as EPSS, KEV and exploit information. The two cron jobs maintain these two datasets separately.”

10.2 Vulnerability Intelligence Cron

14. What is the purpose of the Vulnerability Intelligence Cron?

“Its purpose is to maintain a local mirror of dynamic threat intelligence. It periodically downloads intelligence from sources such as FIRST EPSS, CISA KEV and ExploitDB, processes the data, and stores the resulting intelligence in the PostgreSQL vulnerability_intel table.”

15. What data does it download?

According to your project:

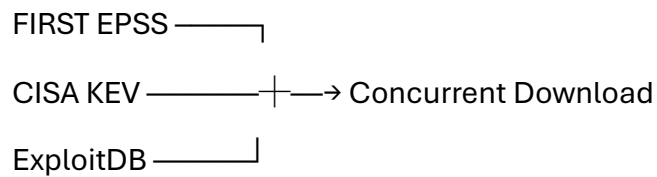
“It downloads more than 250,000 records of threat intelligence, including the FIRST.org EPSS catalog, the CISA KEV feed, and ExploitDB data.”

16. Why use asyncio.gather?

🔥 Technical question.

“The different intelligence sources can be fetched independently, so we use asynchronous concurrent downloading to retrieve them efficiently rather than waiting for one source to finish before starting the next.”

Example:



17. What happens after downloading the data?

“The job processes the downloaded datasets in memory, maps exploit-related information to CVE IDs, determines the relevant threat-intelligence signals, and then performs a large batch upsert into PostgreSQL.”

18. What does KEV flag mean in your database?

“The KEV flag indicates whether a vulnerability appears in the CISA Known Exploited Vulnerabilities catalog. According to our workflow, if a vulnerability appears in KEV, it receives the highest exploit evidence flag.”

19. What is exploit count used for?

“Exploit count provides additional evidence about the availability or presence of known exploit information associated with a vulnerability. It is one of the signals used by our threat-intelligence layer.”

20. Why keep threat intelligence local?

🔥 🔥 Very important.

Your project gives **three reasons**.

1. Performance

“Local database lookups are much faster than repeatedly making network calls.”

2. Resilience

“If an external endpoint is temporarily unavailable, our runtime pipeline can continue using the cached intelligence.”

3. Security / OPSEC

“If we queried external services at runtime with our internal vulnerability list, that could expose information about which vulnerabilities exist in our environment. Keeping the intelligence local reduces that dependency and exposure.”

Your document explicitly emphasizes these three reasons.

21. What does OPSEC mean here?

If interviewer asks:

“OPSEC stands for Operational Security. In our context, it means reducing unnecessary exposure of sensitive information about an organization's vulnerability landscape to external services.”

22. Does the runtime agent ever call the external APIs?

Important nuance:

“The primary strategy is to use the local PostgreSQL mirror. The agent can use API fallback mechanisms when specific intelligence is missing from the local cache, but the bulk intelligence synchronization is handled offline by the cron job.”

This matches your architecture better than saying **“we never use APIs.”**

23. Why not download all this data every time the agent runs?

“Because it would be highly inefficient. The threat-intelligence datasets are large and repeatedly downloading them for individual remediation runs would introduce unnecessary network traffic, latency, and processing overhead. The cron job separates this heavy background work from the real-time remediation pipeline.”

🔥 Very good system-design answer.

24. How does the Cron Job improve scalability?

“It decouples heavy external-data synchronization from the real-time remediation workflow. The agents don't have to repeatedly download or query large external datasets, so their runtime work is mostly local database lookups. That allows the system to process a much larger number of vulnerabilities efficiently.”

25. What if CISA or FIRST is temporarily unavailable?

“The existing locally cached intelligence remains available to the runtime agents. The next successful cron execution can refresh the missing or outdated information. So the runtime remediation pipeline isn't completely dependent on the availability of those external endpoints.”

26. What happens if the cron job itself fails?

The source doesn't describe a detailed alert/retry mechanism for cron itself, so interview-safe answer:

“The local cache continues to provide the previously synchronized data. When the cron job runs successfully again, it can synchronize the missing period based on the stored synchronization point. The key design principle is that temporary synchronization failure should not immediately block the runtime pipeline.”

Don't claim a specific monitoring or alerting mechanism unless you implemented it.

27. Why PostgreSQL for cron data?

“Because PostgreSQL provides persistent relational storage for the large vulnerability and threat-intelligence datasets, and our runtime agents can perform fast indexed lookups against this local data instead of relying on external APIs.”

28. Cron Job vs Agent

Another likely question:

“What's the difference between the cron job and the agent?”

“The cron job is a scheduled data-maintenance process. It prepares and refreshes the local intelligence database in the background. The agent is the runtime component that consumes that prepared data to perform vulnerability enrichment or prioritization.”

Easy:

Cron → Prepare data

Agent → Use data

29. NVD Cron vs Intel Cron

 **Must memorize**

NVD Sync Cron Vulnerability Intel Cron

Static technical data Dynamic threat data

NVD FIRST EPSS + CISA KEV + ExploitDB

vulnerabilities vulnerability_intel

CVSS EPSS

Description KEV

Fix version Exploit count

Published date Threat signals

Relatively stable Changes frequently

Interview answer:

“The NVD cron maintains the stable technical baseline, while the Intel cron maintains the dynamic threat context.”

30. Explain both cron jobs in 1 minute

Ye answer practice karo:

**“We use two separate cron jobs to maintain local vulnerability intelligence. The first is the NVD Sync Cron, which maintains a local mirror of relatively stable CVE information such as title, description, CVSS, CVSS vector, fix version and published date. The first run performs a full historical import, while subsequent runs are incremental based on the last successful synchronization timestamp.

The second is the Vulnerability Intelligence Cron, which runs on a 24-hour schedule and synchronizes dynamic threat intelligence such as EPSS, CISA KEV and ExploitDB data. These sources are downloaded concurrently, processed and batch-upserted into the vulnerability_intel table.

We keep both datasets locally so the runtime agents can perform fast database lookups without depending on external APIs for every vulnerability. This improves performance, resilience and operational security.”**

Easy Memory Trick

NVD Cron:

Stable → NVD → vulnerabilities table

Intel Cron:

Dynamic → EPSS/KEV/ExploitDB → vulnerability_intel table

Whole idea:

“Sync once in background, query locally at runtime.”

Dashboard — Interview Preparation

1. What is the purpose of your dashboard?

Interviewer: “What is the purpose of the dashboard?”

Tum bolo:

“The dashboard acts as the operational command center of our vulnerability remediation platform. It provides a high-level view of vulnerability volume, remediation performance, agent efficiency, cost, token usage, success rate, and severity distribution. The dashboard gets its historical analytics from PostgreSQL through the dashboard API.”

Tumhari documentation ke according dashboard data specifically vulnerabilities_history table se serve hota hai through /api/v1/dashboard.

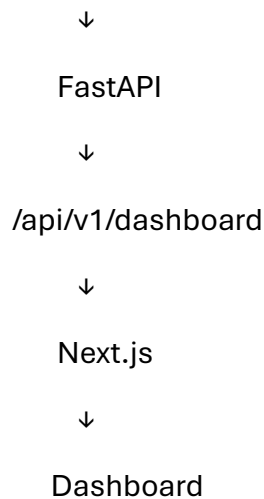
2. Where does the dashboard data come from?

★ Very important

“The dashboard reads historical remediation data from the PostgreSQL vulnerabilities_history table through the /api/v1/dashboard endpoint. So the frontend doesn't calculate these metrics itself; the backend retrieves the historical records and provides the required analytics to the dashboard.”

Flow:

vulnerabilities_history



3. Why use PostgreSQL for dashboard data?

“Dashboard analytics require historical data across many remediation workflows. PostgreSQL provides persistent relational storage and allows the backend to query historical remediation records efficiently. This also keeps the dashboard independent from the active workflow state stored in SQLite.”

🔥 Remember:

SQLite → Current workflow state

PostgreSQL → Historical analytics

4. What metrics are shown on the dashboard?

Your documented metrics are:

1. Average Cost per Vulnerability

Measures the financial efficiency of the LLM-driven remediation process.

2. Average MTTR per Vulnerability

Measures how quickly vulnerabilities are remediated.

3. Total Vulnerabilities

Shows the total number of vulnerabilities processed historically.

4. Total Solved Vulnerabilities

Shows how many vulnerabilities were successfully remediated.

5. Tokens Used

Tracks LLM API consumption.

6. Success Rate

Shows the overall remediation success percentage.

7. Vulnerabilities by Severity

Shows the historical distribution across severity levels.

8. Fleet Distribution / Recent Activity

Provides recent operational/audit visibility.

5. Explain Average Cost per Vulnerability

“Average cost per vulnerability represents the average estimated cost incurred by the AI-driven remediation workflow for each vulnerability. It helps us understand the financial efficiency of using LLMs in the remediation pipeline.”

If interviewer asks:

“Why is cost important?”

“Because our workflow uses LLMs for research, remediation generation and self-correction. Monitoring cost helps us understand whether the automation is operationally efficient and helps identify workflows with unnecessarily high model usage.”

6. What is MTTR?

 Very important.

“MTTR stands for Mean Time To Remediate. It represents the average time taken by the system to remediate vulnerabilities from the start of the remediation workflow until completion.”

Your dashboard uses MTTR specifically to measure **remediation speed**.

Formula conceptually:

Don't claim a different formula unless your implementation uses one.

7. Why is MTTR important in your project?

“The main goal of our system is to reduce the time required to move from vulnerability detection to a verified fix. So MTTR is one of the most important metrics for measuring whether the automation is actually improving the remediation process.”

8. What is Success Rate?

“Success rate represents the percentage of remediation workflows that completed successfully compared with the total workflows processed. It provides a high-level indication of how reliably the automated remediation pipeline is working.”

Conceptually:

9. What does “Total Vulnerabilities” mean?

“It represents the total historical volume of vulnerabilities processed by the platform.”

Important:

“It is a historical processing metric, not necessarily the current number of open vulnerabilities.”

This distinction is useful if interviewer catches you.

10. What does “Total Solved Vulnerabilities” mean?

“It represents the number of vulnerabilities that were successfully remediated and completed through the workflow.”

11. Why track token usage?

 Important for an AI-agent project.

“Token usage helps us monitor how much LLM processing the remediation workflows consume. Since token consumption directly affects model cost and can also indicate inefficient tool-calling or prompts, tracking it helps us optimize the agents.”

This connects directly to one of your documented challenges where excessive tool calls increased latency and cost.

12. What is the Historical Token Consumption graph?

“It shows how many tokens were consumed by the remediation workflows over the selected historical period. It helps us identify trends in LLM usage and understand whether model consumption is increasing or decreasing.”

13. Why show Vulnerabilities by Severity?

“It gives the security team a quick view of the overall risk distribution in the processed vulnerability population—for example, how many findings were Critical, High, Medium or Low.”

14. What is Recent Activity / Fleet Distribution?

“It provides operational visibility into recently processed remediation tasks and the broader distribution of the vulnerability workload. It acts as an audit-oriented view of recent system activity.”

Your documentation specifically describes it as an **audit log of recently processed system tasks**.

15. Why not calculate dashboard metrics in the frontend?

🔥 Good architecture question.

“The frontend should primarily present the data rather than reproduce business logic. We centralize the metric calculation and data retrieval in the backend so that the same logic is consistent and the frontend remains lightweight.”

16. How does the dashboard get updated?

Your stack includes WebSocket.

Interview-safe answer:

“The dashboard retrieves analytics through the FastAPI `/api/v1/dashboard` endpoint, while WebSocket can be used for real-time workflow updates. So historical metrics come from the dashboard API, whereas real-time remediation progress can be pushed through the WebSocket connection.”

Important distinction:

REST/API → Dashboard analytics

17. Does the dashboard control the remediation workflow?

“The dashboard provides monitoring and operational visibility. The actual remediation workflow is managed by the backend agentic system. The frontend doesn't directly perform the remediation itself.”

18. What happens if PostgreSQL is unavailable?

“The dashboard API won't be able to retrieve the historical records, so the frontend should handle the database/API failure gracefully. Importantly, a dashboard availability issue should not be confused with a remediation workflow failure because active workflow state is managed separately.”

 This demonstrates architecture understanding.

19. Why is dashboard data stored in PostgreSQL instead of SQLite?

“SQLite is used for active workflow state and LangGraph checkpointing, whereas PostgreSQL is intended for persistent historical data and analytics. Dashboard queries need historical remediation records, so PostgreSQL is the appropriate source.”

20. What is the role of vulnerabilities_history?

 Very important.

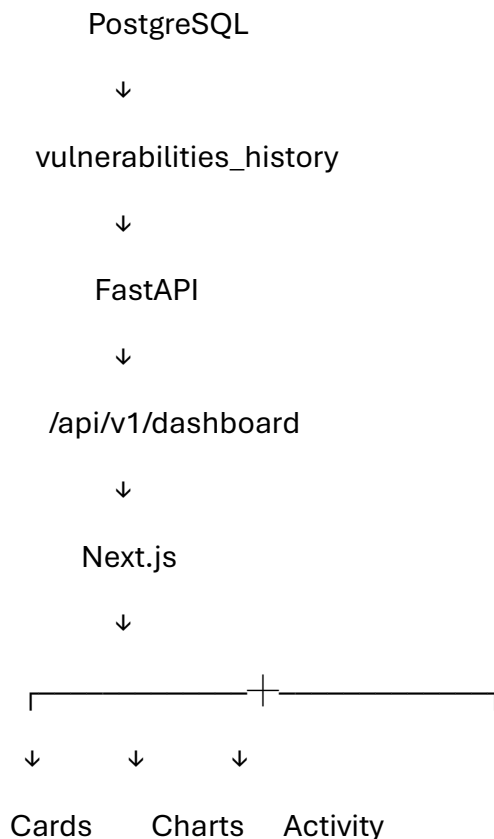
“vulnerabilities_history is the historical record of remediation executions. Each entry represents a remediation attempt and stores information such as the workflow thread ID, asset, CVE, priority score, resolution status, cost, token usage, start and end time, and total execution duration. The dashboard uses this historical data to calculate its metrics.”

21. Explain the dashboard architecture.

“The Next.js dashboard is the presentation layer. It requests analytics from the FastAPI /api/v1/dashboard endpoint. FastAPI queries the PostgreSQL vulnerabilities_history table and returns the required metrics. The frontend then

visualizes these metrics using cards, charts, severity distributions, and recent activity views.”

Architecture:



22. Interviewer: “What does the dashboard prove about your system?”

Very good question.

“The dashboard gives measurable evidence of how the automation is performing. Instead of only saying that the system automates remediation, we can evaluate it through metrics such as MTTR, success rate, token usage, cost, total vulnerabilities processed, and total vulnerabilities solved.”

23. How do you use the dashboard to identify problems?

“For example, an increase in average cost or token consumption may indicate inefficient LLM usage or excessive tool calls. A drop in success rate may indicate problems in remediation generation or validation. An increase in MTTR may indicate bottlenecks in execution, retries, or human approval.”

🔥 This is a strong real-world answer.

24. Your screenshot shows values like 4m 5s MTTR and 86.7% success rate. What do they mean?

Careful: those are **displayed dashboard values**, not necessarily fixed project-wide results.

You can say:

“Those are the current dashboard metrics for the dataset/run represented in the dashboard. For example, the displayed MTTR shows the average remediation time for the available historical records, and the success rate represents the proportion of successfully completed workflows in that dataset.”

Don't present the screenshot's **30 vulnerabilities / 26 solved / 86.7%** as permanent benchmark claims unless your evaluation section explicitly establishes them as final results.

 25. Why is the dashboard called Executive Telemetry?

If interviewer asks based on the UI:

“Because it provides high-level operational telemetry rather than exposing every implementation detail. It summarizes system performance through cost, MTTR, token usage, success rate, vulnerability volume, and severity distribution.”

26. What is the Remediation Command Center?

The second screenshot is a different frontend view.

You can explain:

“The Remediation Command Center is the operational interface where the user can submit new scanner results for processing. It provides a drag-and-drop interface for scan result files and acts as the entry point for starting the vulnerability processing pipeline.”

The UI shown supports scan-result uploads such as Trivy and other payload formats, but **your documented backend section specifically says the dashboard API serves historical analytics**. Keep those two UI responsibilities separate.

Simple:

Dashboard → Monitor

27. Most important — Explain Dashboard in 1 minute

Ye answer practice karo:

****“Our frontend dashboard acts as the operational command center of the Zero-Touch Vulnerability Remediation platform. It gets historical remediation data through the FastAPI /api/v1/dashboard endpoint, which reads from the PostgreSQL vulnerabilities_history table. The dashboard provides metrics such as average cost per vulnerability, MTTR, total vulnerabilities processed, total solved vulnerabilities, token consumption, success rate, and severity distribution.**

These metrics help us evaluate both security and operational performance. For example, MTTR measures remediation speed, success rate measures the reliability of the automation, token and cost metrics help us optimize LLM usage, and severity distribution provides an overall risk view. We also use the frontend to provide operational visibility into recent remediation activity, while real-time workflow updates can be delivered through WebSocket.”**

Agent Performance Evaluation — Interview Preparation

1. Why did you need an evaluation framework?

Interviewer: “Why did you evaluate the agent?”

Tum bolo:

“Because our system is autonomous and it generates and executes remediation scripts, we cannot evaluate it only by checking whether the workflow runs. We also need to measure how accurately it generates the first fix, how often remediation succeeds, how much time and cost it takes, and how frequently human reviewers approve its proposed changes. These metrics help us measure reliability, efficiency, and safety before production deployment.”

One-line:

“Since the system can make security changes, we need quantitative evidence that the agent is reliable, efficient, and safe.”

2. What are the main objectives of your evaluation?

Your documentation gives four main objectives:

1. Initial Precision & Code Accuracy

How often does the first generated remediation work?

2. Value for Money

How much does it cost to remediate one vulnerability?

3. Speed

How quickly does the agent actually perform the remediation?

4. Trust & Safety

How often do human reviewers approve the generated remediation?

Interview answer:

“Our evaluation focuses on four dimensions: first-pass accuracy, cost efficiency, execution speed, and human trust. We want to know whether the agent gets the remediation right initially, how expensive it is, how fast it operates, and whether engineers are comfortable approving its changes.”

3. What is Resolution Success Rate — RSR?

🔥 Very important.

“Resolution Success Rate is the ratio of successfully resolved vulnerabilities to the total number of processed vulnerability tickets.”

Formula: $RSR = (\text{Count of tickets marked 'Resolved' via merged PRs}) / N$

Where **N = total unique vulnerability tickets ingested.**

Example:

Suppose:

Total tickets = 100

Resolved = 85

Then: 85/100

What does it tell us?

“RSR measures the overall reliability and effectiveness of the remediation engine.”

4. What is First-Pass Success Rate — FPSR?

🔥🔥 Very important for AI

“First-Pass Success Rate measures how many vulnerabilities are successfully resolved on the very first remediation execution without requiring retries or self-correction.”

Formula: FPSR = (Count of vulnerabilities resolved on first execution without retry) / N

Why is this important?

“It tells us how accurately the LLM understands the vulnerability and generates the remediation on its first attempt. A low FPSR means the system frequently needs retries or self-correction.”

5. RSR vs FPSR

🔥 Likely cross-question.

“RSR measures whether the vulnerability was eventually resolved, while FPSR measures whether it was resolved on the first attempt without retries.”

Example:

100 vulnerabilities

90 eventually resolved

60 resolved on first attempt

Then:

RSR = 90%

FPSR = 60%

So:

“The system may have a high final success rate while still requiring many retries.”

That's an excellent insight to mention.

6. What is Mean Active Execution Time — MAET?

“MAET stands for Mean Active Execution Time. It measures the average time the agent actively spends computing, executing scripts, and processing logs, while excluding the time the workflow is simply waiting for human approval.”

Formula: MAET = Average of (End-to-end lifecycle time – Time spent waiting for human to merge the PR)

Why exclude human waiting time?

🔥 Important.

“Because the purpose of MAET is to measure the agent's actual processing efficiency. If we included the time waiting for a human to approve a Pull Request, the metric would mix system performance with human response time.”

7. MAET vs MTTR

This is a very likely question because your dashboard also has **MTTR**.

Tum bolo:

“MTTR measures the overall remediation duration, while MAET focuses specifically on the time the agent is actively executing and processing the remediation workflow, excluding the human approval waiting period.”

Easy:

MTTR → Overall remediation duration

MAET → Agent's active work time

8. What is PR Acceptance Rate?

“PR Acceptance Rate measures how often human reviewers approve and merge the remediation Pull Requests compared with the total submitted PRs that were either approved or rejected.”

Formula: $AR = (\text{Approved PRs}) / (\text{Approved PRs} + \text{Rejected PRs})$

Example:

Approved = 80

Rejected = 20

$AR = 80 / (80 + 20)$

9. Why is PR Acceptance Rate important?

🔥 Very important.

“Because our architecture deliberately retains Human-in-the-Loop approval before production. The acceptance rate tells us whether security engineers trust the fixes generated by the agent and consider them safe and acceptable.”

Meaning:

High AR → More human confidence

Low AR → Generated fixes may be incorrect, risky, or incomplete

10. What is Average Token Consumption?

“Average Token Consumption measures the average number of LLM input and output tokens consumed per vulnerability during the evaluation period.”

Formula: Avg Tokens = Sum of (Input Tokens[i] + Output Tokens[i]) for each ticket i, divided by N

Why is this important?

“Because token consumption directly affects LLM cost and can also indicate whether the agent is using unnecessarily long prompts, excessive tool calls, or repeated retries.”

This directly relates to your documented tool-calling challenge.

11. What is Average Token Cost per Resolution?

“It represents the average financial cost of LLM token usage for successfully resolved vulnerabilities. It helps us evaluate the economic efficiency of the autonomous remediation system.”

Why important?

“An automated system isn't useful if it successfully remediates vulnerabilities but does so at an unnecessarily high cost.”

12. What can cause token cost to increase?

 Good scenario question.

“High token usage can result from large prompts, lengthy execution logs, repeated self-correction attempts, excessive tool calls, or inefficient agent reasoning. That's why token and cost metrics are important for optimization.”

Your documentation specifically mentions an LLM getting stuck in a tool-calling loop, which increased both **latency and cost**.

13. How do you evaluate the accuracy of the remediation agent?

“We primarily look at First-Pass Success Rate to measure initial remediation accuracy, and Resolution Success Rate to measure eventual remediation reliability. We also use PR Acceptance Rate as a human validation signal.”

Three perspectives:

FPSR → Initial AI accuracy

RSR → Overall system reliability

PR Acceptance Rate → Human trust

14. Why is FPSR more useful than only RSR for an AI agent?

 Excellent question.

“Because RSR can remain high even if the system requires many retries. FPSR gives us a more direct measure of the quality of the first remediation generated by the LLM. It tells us whether the agent actually understood the vulnerability correctly at the beginning.”

15. How do you evaluate safety?

“Safety is evaluated through multiple controls rather than one metric. The remediation is first tested in a cloned sandbox environment, then independently validated, and only after successful validation does a human review the Pull Request. PR Acceptance Rate then provides a measurable signal of human trust in the generated remediation.”

16. Why is human approval part of evaluation?

“Because the final production decision remains with a human security engineer. The approval rate helps us determine whether the agent's proposed changes are consistently acceptable and safe from an engineering perspective.”

17. What is the evaluation window?

Your formulas are defined over:

“A given evaluation window, where N represents the total number of unique vulnerability tickets ingested during that period.”

Interview answer:

“We calculate the metrics over a defined evaluation window and use the number of unique vulnerability tickets processed during that window as the denominator where applicable.”

18. Why use unique vulnerability tickets?

“We use unique tickets to avoid counting the same remediation workflow multiple times and to make the metrics representative of the actual vulnerability workload.”

19. How do these metrics work together?

This is a **very strong architecture-level answer**.

“No single metric gives the complete picture. RSR tells us whether the system eventually succeeds, FPSR tells us how accurate the first remediation is, MAET tells us how efficiently the agent works, PR Acceptance Rate tells us whether humans trust the generated changes, and token/cost metrics tell us whether the system is economically efficient.”

20. Suppose RSR is high but FPSR is low. What does that mean?

 Scenario question.

“It means the system is eventually successful, but it relies heavily on retries or self-correction. So the overall remediation capability is good, but the initial quality of the generated scripts needs improvement.”

This is exactly the kind of insight an interviewer likes.

21. Suppose FPSR is high but PR Acceptance Rate is low. What could that indicate?

“The remediation may technically pass automated validation, but human reviewers may still have concerns about code quality, safety, completeness, or operational impact. It would indicate that technical success and human trust are not fully aligned.”

22. Suppose MAET is suddenly very high. What could be the reason?

“It could indicate long-running script execution, complex remediation logic, heavy log processing, repeated self-correction, or inefficient tool usage. Since MAET excludes human waiting time, the issue would primarily be within the agent's active processing path.”

23. Suppose token cost suddenly increases.

“I would investigate whether there are more retries, larger prompts, excessive execution logs being sent to the LLM, or unnecessary tool calls. This is particularly relevant because our project has already encountered excessive tool-calling behavior.”

24. Why don't you use only Success Rate?

“Success rate alone doesn't explain why the system succeeded or how efficiently it succeeded. A system could have a high success rate but require many retries, consume too many tokens, or take too long. That's why we use multiple KPIs covering accuracy, efficiency, cost, and trust.”

 Very strong answer.

25. How would you improve the evaluation framework?

Be careful not to imply these are already implemented unless documented.

You can say:

“For future evaluation, we could segment the metrics by vulnerability severity, scanner, remediation type, model, or retry count. That would help identify where the agent performs well and where it needs improvement.”

This is a **future improvement suggestion**, not a claim about current implementation.

 **26. Most Important — Explain your complete evaluation framework in 1 minute**

Ye answer practice karo:

“Because our system is an autonomous remediation platform, we evaluate it across four major dimensions: accuracy, reliability, speed, and cost or trust.

Resolution Success Rate measures how many vulnerabilities are eventually resolved. **First-Pass Success Rate** measures how many are fixed on the first attempt without retries, which directly reflects the initial quality of the LLM-generated remediation. **Mean Active Execution Time** measures the time the agent actively spends processing the workflow while excluding human approval wait time. **PR Acceptance Rate** measures how often human reviewers approve and merge the generated remediation. Finally, token consumption and cost metrics measure the efficiency and financial impact of the LLM. Together, these KPIs give us a quantitative view of whether the system is reliable, fast, safe, and economically practical.”

1. Interviewer: “Explain the overall architecture of your project.”

Ye answer practice karo:

“Our system follows a three-tier architecture: Presentation, Orchestration, and Persistence.

The Presentation tier is built using Next.js and provides the Executive Dashboard and Remediation Command Center. The Orchestration tier is built with FastAPI and Python and acts as the central backend, connecting the frontend with our LangGraph-based multi-agent workflows. The Persistence tier uses two databases: SQLite for active workflow state and LangGraph checkpoints, and PostgreSQL for persistent telemetry and historical remediation data.

The agents are organized as specialized workflow stages, such as parsing, normalization, asset enrichment, vulnerability intelligence, prioritization, and remediation. LangGraph manages the state and transitions between these agents. For human approval, the workflow is interrupted after the remediation Pull Request is created, the state is saved in SQLite, and a GitHub webhook later resumes the correct workflow after human action.”**

2. What are the three tiers?

Presentation Tier

“The Presentation tier is our frontend layer, built using Next.js. It is responsible for user interaction, dashboard visualization, scanner-data ingestion, and displaying real-time remediation progress.”

Orchestration Tier

“The Orchestration tier is built using FastAPI and Python. It acts as the central router and execution layer for our AI agents and handles REST APIs, WebSockets, and LangGraph workflows.”

Persistence Tier

“The Persistence tier contains SQLite and PostgreSQL. SQLite manages volatile workflow state and LangGraph checkpoints, while PostgreSQL stores persistent analytics and historical remediation data.”

Easy memory:

Frontend → FastAPI/Agents → Database

3. Why did you divide the system into three tiers?

“We separated the concerns so that each layer has a clear responsibility. The frontend focuses on presentation, the backend focuses on orchestration and business logic, and the database layer focuses on persistence. This makes the architecture easier to maintain and scale independently.”

4. What does the Executive Dashboard do?

“The Executive Dashboard is primarily a telemetry interface. It displays global KPIs such as average cost per vulnerability, mean time to resolve, success rate, and token consumption. It retrieves this information from the backend through REST APIs.”

Important distinction:

Dashboard → Monitoring

Remediation Center → Operations

5. What does the Remediation Command Center do?

“The Remediation Command Center is the operational interface. It allows users to upload raw scanner output such as JSON or CSV and monitor the execution of the agent workflow in real time through WebSocket streams.”

6. Why both REST and WebSocket?

 Very likely question.

“We use REST for request-response operations such as file uploads and retrieving dashboard metrics. We use WebSocket for real-time streaming of agent execution logs and workflow updates. REST is suitable for normal API communication, while WebSocket is suitable for continuous real-time updates.”

Remember:

REST → Request/Response

WebSocket → Real-time streaming

7. What is the role of FastAPI?

“FastAPI is the central backend layer. It exposes REST endpoints for file uploads and dashboard metrics, provides WebSocket endpoints for real-time logs, and connects the frontend with the LangGraph-based agent workflows and databases.”

8. Why is FastAPI called the central router?

“Because requests and events from different parts of the system pass through the backend. It receives input from the frontend, starts or interacts with agent workflows, handles GitHub webhook events, and exposes data back to the frontend.”

9. What is the role of LangGraph in the architecture?

“LangGraph orchestrates the multi-agent workflow. It represents the agents as stateful graph nodes and controls how the workflow moves from parsing to normalization, enrichment, prioritization, remediation, and human approval. It also allows the workflow to pause, resume, and handle conditional paths.”

10. Why use multiple specialized agents instead of one large AI agent?

🔥 🔥 Very important

“We split the workflow into specialized agents because each stage has a different responsibility. For example, the Parser Agent handles scanner-specific extraction, the Enrichment Agent handles vulnerability intelligence, the Asset Agent handles business context, the Prioritization Agent calculates risk, and the Remediation Agent handles patch generation and validation. This separation makes the system more modular, easier to debug, and reduces the amount of context any single agent needs to process.”

Your architecture describes this as specialized AI agents passing shared state through a defined graph.

11. How do these agents communicate?

“The agents don't operate as completely independent applications. LangGraph manages a shared workflow state, and each specialized node reads the current state, performs its task, and updates that state for the next stage.”

Flow:

Scanner

↓

Parser

↓

Normalization

↓

Asset Enrichment

↓

Vulnerability Intel

↓

Prioritization

↓

Remediation

12. Why is state management important?

“Because the workflow is long-running and can branch, retry, or pause for human approval. The next agent needs the output and status of the previous stage, so maintaining shared state is essential for consistent execution.”

13. Why two databases?

  **One of the most important architecture questions**

“We use two databases because active workflow state and historical business data have different requirements. SQLite is used as the lightweight volatile state store and LangGraph checkpoint saver, while PostgreSQL stores persistent vulnerability, asset, remediation, and telemetry data.”

Easy:

SQLite → What is happening now?

PostgreSQL → What has happened historically?

14. What is stored in SQLite?

“SQLite stores active tasks, workflow state, and LangGraph checkpoints. For example, a remediation workflow can be in states such as PENDING or WAITING_FOR_HUMAN_APPROVAL.”

15. What is stored in PostgreSQL?

“PostgreSQL stores persistent vulnerability and remediation information, including historical metrics and telemetry used by the dashboard.”

16. Explain the Human-in-the-Loop architecture.

  **Extremely important**

****“After the Remediation Agent successfully creates and validates a Pull Request, the workflow triggers a LangGraph interruption. The task state is saved in SQLite as WAITING_FOR_HUMAN_APPROVAL, so the agent does not need to keep running while waiting.**

When the human reviews the Pull Request, GitHub sends a webhook event to our backend. The backend uses the PR ID to find the corresponding workflow thread,

resumes the suspended LangGraph workflow, and then continues based on the human decision.”**

This is one of the strongest architectural features of the project.

17. Why use `__interrupt__`?

If interviewer goes deep:

“The interruption allows the LangGraph workflow to intentionally pause at the Human-in-the-Loop stage while preserving its current state. This is better than keeping the process running and continuously polling for human input.”

18. Why not keep the agent running while waiting for approval?

“That would unnecessarily consume resources and keep an active process alive even though no computation is happening. Instead, we persist the state, stop the active execution, and resume only when an external event indicates that human feedback is available.”

🔥 Great system-design answer.

19. How does the system know which workflow to resume?

“We store the Pull Request ID along with the workflow thread ID. When a GitHub webhook arrives, the backend uses the PR ID to locate the corresponding workflow thread and resumes that specific workflow.”

20. What if there are multiple remediation workflows waiting for approval?

This is a strong follow-up.

“Each workflow has a unique thread ID, and each Pull Request is mapped to the corresponding thread. So when a webhook arrives, the system can resume the correct workflow independently of other workflows that may also be waiting for approval.”

21. Explain the end-to-end integration.

🔥🔥 Most important overall question

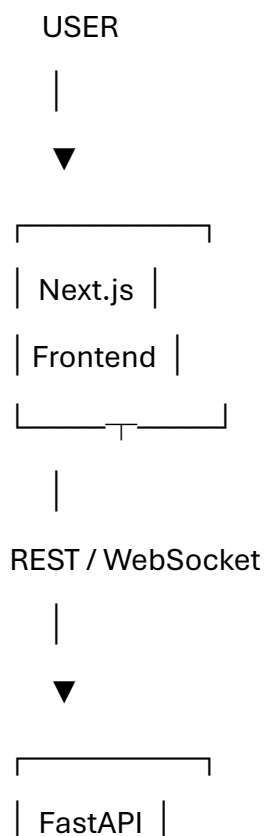
Tum bolo:

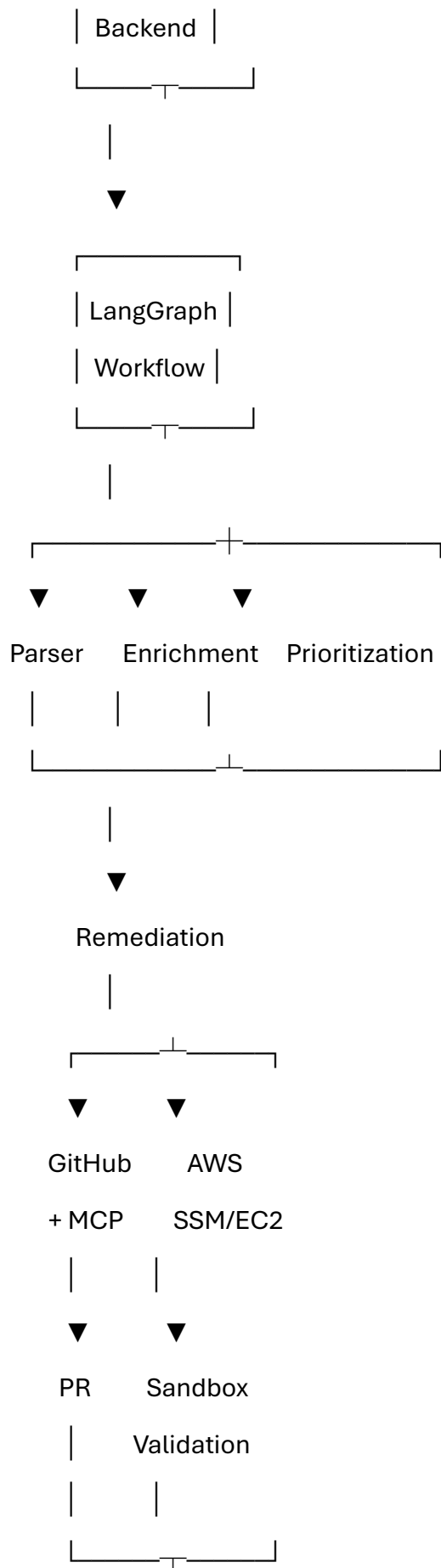
***“The process starts when the user uploads scanner data through the Next.js Remediation Command Center. FastAPI receives the file and initiates the LangGraph workflow. The Parsing and Normalization agents convert the scanner output into a common structure. The Asset Enrichment and Vulnerability Intelligence agents add infrastructure context and threat intelligence. The Prioritization Agent calculates the final risk score.

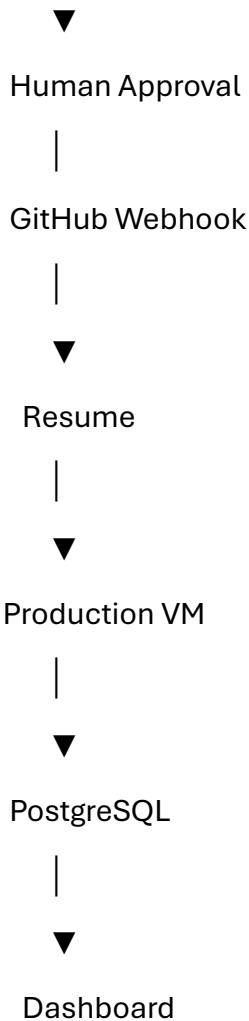
The selected vulnerability is then passed to the Remediation Agent, which researches the vulnerability, generates remediation scripts, and uses GitHub MCP to create a remediation branch and Pull Request. GitHub Actions executes the scripts in a cloned sandbox VM, while S3 stores the execution logs and evidence.

After successful validation, the LangGraph workflow pauses for human approval. The state is persisted in SQLite, and GitHub Webhooks notify FastAPI when the reviewer acts. The workflow resumes using the stored PR-to-thread mapping. If approved, the validated remediation is deployed to the original target VM. Finally, the remediation metrics are persisted in PostgreSQL and become available to the Executive Dashboard.”**

22. Explain the complete architecture in one diagram







23. Why is PostgreSQL called the Telemetry Warehouse?

“Because it stores finalized historical remediation data and performance metrics that are used for analytics and dashboard reporting. It is not responsible for maintaining the live state of an active LangGraph execution.”

24. How does data move from SQLite to PostgreSQL?

According to your architecture:

“Resolved remediation data from the local SQLite workflow state is periodically migrated into PostgreSQL through the migration utility. This allows the active workflow database to remain lightweight while historical analytics are stored persistently.”

25. How does the architecture improve scalability?

“The architecture separates the frontend, orchestration, workflow state, and persistent analytics. Because the agent workflows are modular and the state is separated from historical data, different parts of the system can be scaled independently. Multiple remediation workflows can also be managed using independent LangGraph threads.”

Be careful not to claim a specific production deployment topology unless implemented.

26. How does the architecture improve reliability?

“There are several mechanisms: persistent workflow state, checkpointing, retry and self-correction logic, sandbox validation before production, human approval, and clear terminal states such as completed or failed. Together, these prevent a single execution failure from silently corrupting the remediation lifecycle.”

27. How does the architecture reduce LLM hallucination?

 Important.

Your document specifically says specialized agents passing shared state help prevent hallucination and context-window limitations.

Interview answer:

“Instead of giving one LLM the entire vulnerability lifecycle, we divide the work into specialized agents with well-defined responsibilities. Each agent receives the relevant context for its specific task. This reduces unnecessary context, makes the outputs more controlled, and makes the overall workflow easier to validate.”

28. How do you handle failure in the architecture?

“Failures are handled at multiple levels. Individual agents can have validation or retry mechanisms, remediation failures trigger self-correction, workflow state is persisted for recovery, and unrecoverable workflows move to a terminal failed state. This ensures that every workflow has a traceable outcome rather than silently stopping.”

 **29. Most Important — “Why did you design the system this way?”**

Tum bolo:

“We designed the system around separation of concerns. The frontend handles user interaction and visualization, FastAPI handles communication and orchestration, LangGraph manages stateful agent workflows, SQLite handles active workflow state, and PostgreSQL handles persistent analytics. Specialized agents perform individual security tasks, while the remediation stage is isolated and human-approved before production. This makes the system more modular, auditable, safer for production use, and easier to scale.”

30. Full Integration — 90-second answer

Ye **final architecture answer** tumhe interview ke liye pakka prepare karna chahiye:

****“Our Zero-Touch Vulnerability Remediation Platform follows a three-tier architecture consisting of Presentation, Orchestration, and Persistence.**

The Presentation tier is built with Next.js and provides two main interfaces: the Executive Dashboard for historical telemetry and the Remediation Command Center for scanner-data ingestion and real-time workflow monitoring.

The Orchestration tier uses FastAPI and Python as the central backend. LangGraph manages the stateful multi-agent workflow. Specialized agents handle parsing, normalization, asset enrichment, vulnerability intelligence, prioritization, and remediation.

We use SQLite for active workflow state and LangGraph checkpointing, while PostgreSQL stores persistent vulnerability, remediation, and telemetry data.

Once a vulnerability reaches remediation, the agent performs vulnerability research, generates remediation scripts, creates a GitHub Pull Request through MCP, and validates the scripts on a cloned sandbox VM through the CI/CD pipeline. Logs are stored in S3. If the remediation fails, the LLM uses the logs for self-correction and retries.

After successful sandbox validation, the workflow enters a Human-in-the-Loop interruption state. Its state is persisted in SQLite, and GitHub Webhooks notify the backend when the human reviews the PR. The backend uses the PR-to-thread mapping to resume the correct workflow. Only after approval and merge is the validated remediation executed on the original target VM.

Finally, the workflow metrics are persisted in PostgreSQL and exposed through the dashboard for monitoring cost, MTTR, success rate, token usage, and remediation performance.”**

★ The 10 architecture facts you MUST remember

1. Next.js → Presentation
2. FastAPI → Backend/API
3. LangGraph → Agent orchestration + state
4. SQLite → Active workflow/checkpoint
5. PostgreSQL → Persistent analytics/history
6. MCP → External tool interaction
7. GitHub Actions → CI/CD execution
8. SSM/EC2 → Infrastructure execution
9. GitHub Webhook → HITL event → Resume workflow
10. S3 → Logs/evidence

One killer line:

“The architecture separates presentation, orchestration, workflow state, persistent analytics, and infrastructure execution, while LangGraph acts as the stateful control plane connecting the specialized security agents.”

1. Non-CVE Vulnerability Handling

Interviewer: “What was one challenge you faced?”

“One challenge was handling vulnerabilities that did not have a standard CVE identifier. Our downstream enrichment and prioritization pipeline relies heavily on CVE-based intelligence, so findings without a CVE were difficult to standardize, store, and enrich consistently. We addressed this by introducing a CVE recovery step where the system attempts to infer the CVE from the vulnerability title and description. If the CVE still cannot be reliably identified, the record is rejected and kept separately instead of introducing uncertain data into the pipeline.”

Tumhare documentation mein exactly ye challenge identify kiya gaya hai, aur enrichment workflow mein missing CVEs ke liye Gemini fallback bhi documented hai.

Short version:

“The challenge was that some scanner findings don't have CVEs, while our enrichment pipeline depends on CVE-based intelligence.”

2. Cross-Identifier Asset Correlation

Interviewer: “How did you identify the same asset across different scanners?”

“Different scanners may identify the same asset using different identifiers, such as an IP address, hostname, or cloud instance ID. This creates a correlation problem because the system could treat one physical or logical asset as multiple assets. We handled this through a priority-based asset matching strategy: instance ID first, hostname second, and IP address last.”

Your documentation explicitly lists this as a challenge, and the Asset Lookup Agent implements the same priority.

Why that order?

“Instance ID is the most reliable cloud identity, hostname is comparatively stable, and IP can change, so IP is used only as the last fallback.”

3. Sequential Remediation Queue Processing

 This is a **system-design challenge**.

Interviewer: “What scalability issue did you face?”

Interviewer: “What was the remediation queue challenge, and how did you solve it?”

******“Initially, our remediation workflow was effectively sequential. When one vulnerability reached the Human-in-the-Loop stage, the workflow had to wait for human approval before the next vulnerability could be processed. This created a throughput bottleneck because the agent was idle while waiting for human feedback.

We solved this by making the workflow stateful and allowing a remediation thread to pause at the Human-in-the-Loop stage without blocking the overall remediation pipeline. Once the workflow enters `WAITING_FOR_HUMAN_APPROVAL`, its state is persisted, and the system immediately picks up the next pending vulnerability and starts its remediation workflow.

Later, when the human reviews the Pull Request, the GitHub webhook identifies the corresponding workflow thread and resumes that paused remediation independently.

So now, human approval for one vulnerability does not block the remediation of other vulnerabilities.”******

4. Truncated CI Pipeline Error Logs

Interviewer: “What challenge did you face with the CI/CD pipeline?”

“We faced an issue where CI pipeline error logs were truncated when we tried to retrieve them through the API. That was a problem for our self-correction loop because the LLM needs sufficient error context to understand why the remediation failed. Incomplete logs can lead to incorrect diagnosis and poor remediation regeneration.”

Your documentation explicitly lists **truncated CI pipeline error logs when fetching via API** as a challenge.

Why is this important?

Because:

Remediation Failed

↓

Error Logs

↓

LLM Analysis

↓

Corrected Script

Agar logs incomplete hain:

Bad Logs

↓

Wrong Diagnosis

↓

Wrong Fix

5. LLM Tool-Calling Loop

  **One of your strongest real-world challenges**

Interviewer: “What was an AI-specific challenge?”

“We encountered a case where the LLM got stuck in a tool-calling loop for around three minutes. It repeatedly called external tools, particularly Tavily, which

increased both latency and cost. We addressed this through prompt engineering and by reducing unnecessary tool calls.”

This is directly documented in your project.

Why is this a serious problem?

“In an agentic system, uncontrolled tool calls can increase both runtime and API cost. It can also delay the actual remediation.”

Strong technical point:

“This taught us that tool access needs to be controlled, not simply made available to the LLM.”

6. GitHub MCP Metadata Token Overhead

Interviewer: “What challenge did MCP integration create?”

“When integrating GitHub MCP tools, the tool metadata itself was quite large. That metadata became part of the LLM’s context, which increased input token consumption on each invocation. This increased both token overhead and cost.”

Your documentation explicitly identifies **massive GitHub MCP metadata** as a source of token overhead.

Why does this matter?

“In an agentic workflow, the same tool definitions can be exposed repeatedly across multiple model calls, so unnecessary metadata can become a significant cost multiplier.”

7. GitHub Webhook Noise

 **Very likely interviewer question**

Interviewer: “What problem did you face with GitHub Webhooks?”

“GitHub webhooks generated events very frequently, including events caused by the agent’s own Pull Requests and comments. This made it difficult to distinguish genuine human actions from events generated by the automation itself. Since the workflow should resume only when a human actually reviews or comments on the PR, this created noise in the HITL workflow.”

Interviewer: “How did you solve the GitHub webhook noise problem?”

Tum bolo:

“Initially, GitHub was sending webhook events for many actions, including events generated by the agent itself. The problem was that our workflow should resume only when meaningful human feedback is received.

We changed the HITL workflow to an event-driven design where the backend uses GitHub webhook events together with the stored PR-to-thread mapping. The webhook handler identifies the Pull Request and maps it to the exact suspended workflow. We then process only the relevant review, comment, or PR event needed to resume that workflow, instead of treating every GitHub event as a reason to resume the agent.”

Your documentation confirms that the webhook setup listens for **issue comments, pull requests, and pull request reviews**, and the PR-to-thread mapping identifies exactly which workflow should resume.

Architecture:

GitHub Event



FastAPI Webhook



Identify PR



PR → Thread Mapping



Check Relevant Human Action



Resume Correct LangGraph Thread

Most important point:

“We don't resume a workflow just because a webhook arrived. We use the PR-to-thread mapping and the relevant GitHub event to identify the correct suspended workflow.”

Your documentation explicitly mentions this issue.

Architecture challenge:

GitHub

↓

Webhook

└─ Human Review

└─ Agent Comment

└─ Agent PR Update

└─ PR Event

└─ Other Events

But the workflow really cares about:

Human Review / Feedback

↓

Resume Workflow

8. How would you explain all challenges together?

🔥 Most important answer

Interviewer: “What were the major challenges you faced in the project?”

Tum bolo:

******“We faced challenges at multiple layers of the system. At the data level, we had to handle non-CVE findings and correlate the same asset across different scanner identifiers. At the workflow level, a sequential remediation queue reduced throughput because workflows could remain blocked during human approval. At the CI/CD level, truncated error logs made automated failure analysis difficult.

On the AI side, we encountered excessive tool-calling loops that increased latency and cost, and large GitHub MCP metadata increased token overhead. We also faced webhook noise because GitHub generated events for both human actions and automation-generated actions. These challenges were important because they showed us that building an agentic system is not only about making the workflow autonomous—it also requires careful control of data quality, state management, tool usage, observability, and external event handling.”******

9. Interviewer: “Which challenge was the most difficult?”

There isn't one documented “hardest” challenge, so don't pretend the document says that.

A safe answer:

“One of the more interesting challenges was the LLM tool-calling loop because it directly affected both latency and cost. The agent could spend several minutes repeatedly invoking tools instead of progressing through the workflow. We had to refine the prompts and tool usage strategy to make the agent more controlled.”

This is based on the documented three-minute loop.

10. Interviewer: “What did you learn from these challenges?”

“The main lesson was that an agentic system needs strong engineering controls around the AI. Reliable automation requires clean input data, deterministic parts where possible, bounded retries, controlled tool access, persistent workflow state, and clear human approval points. The LLM should be used where reasoning is valuable, but deterministic logic should handle predictable decisions.”

This is a strong synthesis of your architecture.

11. Interviewer: “Which part should use AI and which part should be deterministic?”

 Excellent question.

Tum bolo:

“We use AI where interpretation or reasoning is required—for example, adaptive parsing, ambiguous CVE recovery, vulnerability research, and remediation generation. We use deterministic logic for areas where consistency and auditability are more important, such as asset matching, scoring formulas, database synchronization, validation rules, and final priority calculation.”

This is actually one of the strongest design principles visible throughout your project.

12. Interviewer: “How did you prevent infinite loops?”

“We use bounded retry logic in the remediation and self-correction workflows. So if a generated remediation or parser continues to fail, the system doesn't keep retrying indefinitely; it eventually enters a terminal failure or unresolved state.”

13. Interviewer: “How did challenges affect the architecture?”

Very strong answer:

“The challenges pushed the architecture toward more explicit state management and separation of concerns. We separated static vulnerability data from dynamic threat intelligence, separated asset lookup from criticality scoring, introduced persistent workflow state and Human-in-the-Loop interruption, and moved large external-data synchronization into cron jobs. We also learned to treat observability and tool control as first-class parts of the architecture.”

 Challenge → Lesson Mapping

Ye table yaad kar lo:

Challenge	What it taught us
Non-CVE findings	Need data sanitization/recovery
IP vs hostname vs instance ID	Need reliable asset correlation
Sequential queue	Need better concurrency/state handling
Truncated CI logs	Need reliable observability
LLM tool loop	Need bounded/control tool use
MCP metadata overhead	Need efficient tool/context exposure
Webhook noise	Need event filtering and careful workflow resumption

 14. Best 45-second answer for interview

Agar interviewer bas bole:

“Tell me about the challenges you faced.”

Ye bolo:

**“We faced challenges in data quality, workflow orchestration, AI behavior, and external integrations. For data quality, some findings lacked CVEs and different scanners identified the same asset differently, so we needed CVE recovery and priority-based asset correlation. In workflow orchestration, sequential queue processing reduced throughput when workflows were waiting for human approval. We also faced truncated CI logs, which affected automated failure analysis.

On the AI side, we encountered a three-minute tool-calling loop that increased Tavily cost and latency, and GitHub MCP metadata increased token overhead. Finally, GitHub webhooks generated noisy events from both humans and automation. These challenges helped us improve the system around deterministic processing, bounded retries, state management, controlled tool usage, and event-driven Human-in-the-Loop handling.”**

★ One sentence to remember

“The biggest lesson was that autonomous AI remediation requires more than a good LLM—it requires deterministic controls, persistent state, observability, bounded retries, and human governance.”

Future Scope

1. Non-CVE Vulnerability Support

Interviewer: “What is the first future improvement?”

“Currently, our enrichment pipeline is heavily based on CVE identifiers because we use CVE-based sources such as NVD, EPSS and KEV. A future improvement would be to support vulnerabilities that don't have a CVE ID. The system would need to identify, store, enrich and prioritize these findings using alternative identifiers or security metadata instead of simply rejecting them.”

Why is this needed?

“Because not every security finding necessarily has a standard CVE identifier, and we don't want those findings to be lost from the vulnerability management pipeline.”

Short answer:

“We want to extend the pipeline beyond CVE-based findings so that non-CVE security findings can also be fully processed.”

Your documentation explicitly lists non-CVE support as future scope.

2. Cross-Identifier Asset Mapping

Interviewer: “What is cross-identifier asset mapping?”

“Different scanners can identify the same infrastructure asset differently. One scanner may report an IP address, another may report a hostname, and another

may provide an instance ID. Future work would be to build a stronger asset-correlation mechanism that can confidently unify these identifiers and determine that they actually represent the same asset.”

Example:

Scanner A → 10.10.10.25

Scanner B → auth-server-01

Scanner C → i-0abc123

↓

Same Asset

↓

asset_id = XYZ

Why is this important?

“Accurate asset identity is essential because asset criticality and vulnerability priority depend on the correct asset context. If the same server is treated as three different assets, the risk calculation and remediation history can become inaccurate.”

Your documentation explicitly identifies cross-identifier mapping as future scope.

3. Approved Remediation Knowledge Repository

🔥 This is an interesting AI-focused future scope.

Interviewer: “What is an approved remediation knowledge repository?”

“The idea is to maintain a knowledge repository that maps an asset and vulnerability combination to a previously approved remediation script. So, if the same vulnerability appears again on a similar asset, instead of generating a new script from scratch, the system could retrieve a previously approved and validated remediation.”

Concept:

Asset + Vulnerability

↓

Knowledge Repository

↓

Approved Script



Validate / Execute

Why is this useful?

“It could reduce LLM dependency, reduce token cost and latency, and increase consistency because we reuse a remediation that has already been reviewed and approved.”

Your documented future scope specifically proposes an **Asset + Vulnerability** → **Approved Script** mapping to enable more autonomous future remediation.

4. Would this eliminate Human-in-the-Loop?

🔥 Likely follow-up question.

Don't say an automatic **yes**.

“The idea is to increase the level of automation for known and previously approved remediation patterns. Whether human approval can be skipped would depend on the organization's security policy and the confidence level of the stored remediation. The current system deliberately retains Human-in-the-Loop approval before production.”

This is a strong answer because your current architecture explicitly retains human approval before production.

5. Multi-Agent Concurrent Remediation Framework

Interviewer: “What is the fourth future scope?”

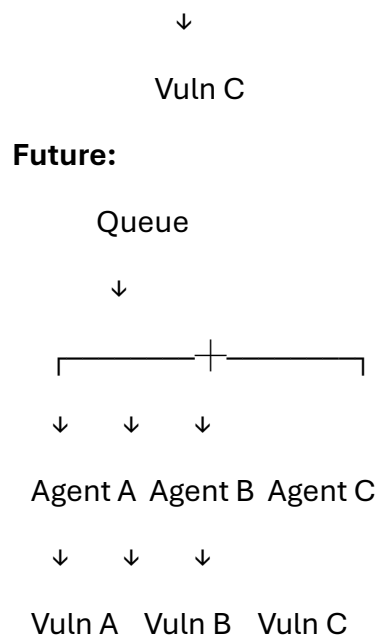
“Currently, the remediation workflow can be constrained by sequential processing when multiple vulnerabilities are waiting in the queue. A future improvement is to introduce parallel remediation agents so that multiple independent vulnerabilities can be remediated concurrently instead of processing them one after another.”

Current limitation:

Vuln A → Remediation → Wait



Vuln B



Benefit:

“This would increase remediation throughput and reduce overall execution time, especially for large enterprise workloads.”

Your documentation explicitly lists **parallelized remediation agents** as future scope for improving throughput and reducing execution time.

6. Interviewer: “Which future scope is the most valuable?”

There isn't an objectively documented answer in your report, so frame it as **your engineering opinion**, not as a project fact.

A strong answer:

“I would consider concurrent remediation a high-impact improvement for scalability because it directly increases throughput. At the same time, the approved remediation repository could significantly reduce LLM cost and latency for recurring vulnerabilities. So I would prioritize them based on the organization's main bottleneck—throughput versus remediation-generation cost.”

7. Interviewer: “Which future scope would improve AI reliability?”

“The approved remediation knowledge repository would help because previously reviewed and validated remediation scripts could be reused for recurring vulnerabilities. That reduces dependence on generating a new remediation from scratch every time and can make the overall process more consistent.”

8. Interviewer: “Which future scope improves data quality?”

“Cross-identifier asset mapping, because it improves our ability to correctly unify the same asset across different scanner outputs. This directly improves downstream asset criticality, prioritization, and remediation tracking.”

9. Interviewer: “Which future scope improves coverage?”

“Non-CVE vulnerability support, because it expands the range of security findings that the platform can process instead of limiting the workflow to findings that have standard CVE identifiers.”

10. Most Important — Explain the Future Scope in 1 minute

Ye answer practice karo:

******“We have four major areas for future enhancement. First, we want to support non-CVE vulnerabilities so that security findings without standard CVE identifiers can also be enriched and prioritized. Second, we want stronger cross-identifier asset mapping so that IP addresses, hostnames and cloud instance IDs from different scanners can be reliably correlated to the same asset.

Third, we want to build an approved remediation knowledge repository that maps an asset and vulnerability combination to previously reviewed remediation scripts. This could reduce repeated LLM generation and improve consistency for recurring vulnerabilities.

Finally, we want to introduce parallel remediation agents so that independent vulnerabilities can be processed concurrently, improving throughput and reducing overall remediation time.”******

Extra Questions:-

Interviewer: “What exactly did you contribute to the project?”

Best interview-ready answer

“This was a team project, and I contributed to both the development of multiple AI agents and the supporting vulnerability-management modules.

On the AI side, I contributed to the development of the Parser Agent, Normalization Agent, and Remediation Agent. In the Parser Agent, I worked on converting

heterogeneous scanner outputs into a standardized format using an adaptive parsing approach. In the Normalization Agent, I contributed to mapping the extracted scanner data into a common vulnerability schema so that downstream agents could process it consistently. In the Remediation Agent, I contributed to the workflow that performs vulnerability research, generates remediation scripts, validates them in a sandbox, and handles the self-correction and approval flow.

Apart from the agents, I also worked on Jira integration, scanner output analysis, prioritization formula research, and audit logging and traceability. For Jira, I worked on automating ticket creation and lifecycle tracking. For prioritization, I contributed to researching and designing how factors like CVSS, EPSS, KEV, asset criticality, fix availability, and vulnerability age should influence remediation priority. I also worked on traceability so that the complete remediation lifecycle could be tracked from identification to final resolution.

Overall, my contribution was not limited to a single module. I worked across the agentic pipeline and also on the integration and tracking layers, which gave me a good understanding of the complete vulnerability remediation lifecycle.”

1. “How would you scale this system if the number of vulnerabilities increases from 5,000 to 100,000 per week?”

“The first step would be to horizontally scale the agent execution layer instead of increasing only the capacity of a single worker. Since our workflow is already separated into specialized agents and maintains workflow state independently, multiple remediation workers can process independent vulnerabilities concurrently. We would also rely more heavily on the local PostgreSQL intelligence cache and batch operations so that external APIs don't become the bottleneck. The database and API layers would also need to be scaled independently based on their load.”

Tumhare documentation specifically horizontal scaling of remediation agents ko future scalability direction ke roop mein mention karti hai.

2. “What is the biggest scalability bottleneck in your current architecture?”

Strong answer:

“The most important potential bottleneck is the remediation execution stage because it involves LLM calls, sandbox execution, CI/CD, validation, and potentially Human-in-the-Loop waiting. We addressed one major issue by making the HITL wait non-blocking, so a workflow waiting for approval doesn't prevent another vulnerability from starting remediation. For larger scale, we would further parallelize independent remediation workflows.”

Ye tumhare solved queue problem ko bhi show karta hai.

3. “How did you solve the single-queue bottleneck?”

🔥 Very likely.

“Initially, a workflow waiting for human approval could delay subsequent vulnerabilities. We changed the workflow so that reaching `WAITING_FOR_HUMAN_APPROVAL` persists that workflow's state and releases the execution path to process the next pending vulnerability. When the human review arrives through the GitHub webhook, the original workflow is resumed using its stored thread mapping. So Human-in-the-Loop becomes a non-blocking pause rather than a blocking queue stage.”

4. “Can multiple vulnerabilities be remediated at the same time?”

Tumhe yahan exact current implementation aur future scope distinguish karna hai.

“Our architecture supports independent workflow state through separate threads, and we have designed the system so that a workflow waiting for human approval does not block the next vulnerability. However, fully parallelized remediation-agent execution is also identified as a future scalability enhancement. The goal would be to run multiple independent remediation agents concurrently to increase throughput.”

Your documentation lists a Multi-Agent Concurrent Remediation Framework as future scope.

5. “Why don't you just use one huge AI agent for all vulnerabilities?”

“A single large agent would create a much larger context, make the workflow harder to control, and increase the risk of unnecessary token consumption. Our specialized-agent architecture allows parsing, enrichment, prioritization, and remediation to scale and operate independently. It also reduces the amount of context each agent needs.”

6. “How does your local vulnerability cache help scalability?”

🔥 Important.

“We don't want thousands of vulnerabilities to trigger thousands of external NVD, EPSS, or KEV API requests during runtime. We maintain local PostgreSQL mirrors through scheduled cron jobs. Runtime agents therefore perform mostly local database lookups, while external APIs are used only as targeted fallbacks where required. This removes network APIs from the critical path for most requests.”

Your documentation explicitly uses local NVD/EPSS/KEV intelligence for faster runtime processing and reduced external dependency.

7. “Why are batch operations important for scalability?”

“If we process tens of thousands of vulnerabilities, doing one database query or insert per record would create a huge number of round trips. Our enrichment and asset-processing stages therefore use bulk lookups, batch inserts, batch upserts, and concurrent API requests where appropriate. This significantly reduces per-record overhead.”

Example:

Bad:

10,000 records

→ 10,000 DB calls

Better:

10,000 records

→ Bulk lookup

→ In-memory matching

→ Batch upsert

8. “How does your Asset Enrichment Agent scale?”

“We avoid per-row database calls. For instance ID, hostname, and IP, we collect unique values and perform bulk lookups. Then we build in-memory maps and resolve each row locally. In Agent 2, assets are processed concurrently, and the database write is done using a batch upsert. This makes the processing much more efficient for large CSVs.”

This is based directly on your Asset Agent design.

9. “What about the 50 vulnerabilities on the same asset?”

“Asset criticality is an asset-level property, so we don't recompute it for every vulnerability row. We deduplicate by asset ID and evaluate the asset once, then reuse the result for all vulnerabilities on that asset. This reduces repeated computation and unnecessary LLM calls.”

10. “How does your Parser Agent scale when a new scanner is added?”

“Our adaptive parsing approach reduces the need to manually maintain a separate parser for every scanner format. The agent analyzes the input structure and generates the required parsing logic dynamically. That makes onboarding additional scanner formats less dependent on manually writing parser code.”

Don't say “*zero effort for every new scanner*”—your documentation supports reduced manual parser maintenance, not a guarantee of zero adaptation.

11. “Will the LLM become a bottleneck at scale?”

🔥🔥 Very likely.

“Yes, LLM calls can become a scalability and cost bottleneck because they involve network latency, token processing, and external API limits. That's why our architecture uses deterministic processing wherever possible and invokes the LLM only where reasoning is actually needed. For example, in asset criticality, deterministic inference happens first and Gemini is only used when all major signals are unknown. We also use local intelligence caches to reduce unnecessary external calls.”

This is a very strong answer.

12. “How did you reduce unnecessary LLM calls?”

“We use deterministic logic before AI wherever possible. For example, Asset Criticality uses role, exposure, environment, and cloud inference rules first, and only calls Gemini when all major signals are unresolved. Similarly, vulnerability intelligence is primarily retrieved from the local database instead of repeatedly asking an external service.”

13. “What happens if thousands of vulnerabilities all need remediation?”

“I would separate the workload into independent remediation tasks, maintain each workflow's state separately, and process them through multiple remediation workers. The queue would distribute work across workers, while Human-in-the-Loop tasks would remain paused independently instead of blocking unrelated tasks.”

14. “How would you scale the FastAPI backend?”

“The FastAPI layer is stateless enough to be scaled horizontally behind a load balancer, while workflow state is persisted separately in the database. That means multiple backend instances can handle incoming requests without relying on in-memory workflow state.”

⚠ This is an architectural scaling approach; don't claim you deployed multiple FastAPI replicas unless you actually did.

15. “What happens if one FastAPI instance goes down?”

“Because active workflow state is persisted rather than being dependent only on process memory, another backend instance can continue handling requests and events. The important requirement is that the state and workflow identity are stored in the shared persistence layer.”

16. “How would you scale PostgreSQL?”

“At higher scale, we would focus on indexing frequently queried fields, efficient batch operations, connection management, and possibly read replicas or a managed PostgreSQL deployment depending on workload. The exact strategy would depend on the read-write pattern and production load.”

Note: read replicas are a future scaling approach, not something I would claim your current implementation has.

17. “Why is SQLite a concern for horizontal scaling?”

🔥 Advanced question.

“SQLite is appropriate for lightweight active workflow state in our current architecture, but it is not the ideal shared state store if we need many distributed

workers writing concurrently across multiple instances. At larger scale, we would consider moving the active workflow/checkpoint state to a shared database-backed persistence layer designed for concurrent distributed workloads.”

This is a thoughtful answer because it shows you understand the trade-off instead of blindly defending SQLite.

18. “How does Human-in-the-Loop affect scalability?”

“Human approval can become a bottleneck if the system treats it as a blocking execution step. Our design avoids that by pausing the workflow and persisting its state. The system can continue processing other vulnerabilities while waiting for the human response.”

🔥 This is one of your strongest scalability points.

19. “How would you handle 1,000 PRs waiting for approval?”

“I would keep each workflow independently persisted in a waiting state rather than maintaining an active process for each PR. GitHub webhooks would resume only the workflows for which a relevant event occurs. This makes the waiting state lightweight compared with keeping thousands of active polling processes.”

20. “How would you handle external API rate limits?”

“We minimize runtime dependency on external APIs by maintaining local intelligence mirrors. For unavoidable fallbacks, we batch requests and handle them selectively rather than making one request per vulnerability. This reduces both rate-limit pressure and network overhead.”

21. “What happens if NVD or CISA goes down during remediation?”

“Most of the required intelligence is already cached locally, so the runtime workflow can continue using the existing mirror. Only missing or uncached information would require an external fallback request. This makes the critical remediation path more resilient.”

22. “How do you scale your WebSocket connections?”

Possible advanced question:

“For a larger deployment, WebSocket connections would need to be distributed across backend instances with shared state or a messaging layer so that workflow events can reach the correct connected client. The important architectural principle is that the remediation workflow itself should not depend on a single WebSocket connection.”

Again, this is a design consideration, not a claim that you implemented a distributed WebSocket broker.

23. “How does S3 help when logs become huge?”

“Instead of keeping large execution logs on the application server, we store them in S3. This separates artifact storage from compute and prevents large logs from bloating the application or workflow database.”

24. “How would you scale the dashboard?”

“The dashboard should remain a read-oriented layer, while PostgreSQL acts as the historical data source. As the history grows, we can optimize queries, add appropriate indexes, introduce aggregation or precomputed metrics, and separate analytical workloads from transactional workflow workloads.”

25. “How do you prevent duplicate remediation when the same vulnerability appears multiple times?”

“We identify vulnerabilities in the context of the affected asset, and the database maintains the asset-vulnerability relationship. We also cache previous priority calculations for an Asset ID + Vulnerability ID pair, which avoids unnecessary recalculation. For remediation orchestration, unique workflow and ticket mappings help maintain traceability.”

26. “What would be your first bottleneck at 100x scale?”

A mature answer:

“I wouldn't assume one bottleneck without measuring it. I would monitor queue depth, database latency, LLM latency and token usage, CI execution time, external API latency, and remediation throughput. Then I would scale the specific constrained layer. In this architecture, likely candidates are LLM calls, remediation execution, database writes, or external integrations.”

🔥 This answer is better than randomly saying “PostgreSQL will be the bottleneck.”

27. “What metrics would you monitor for scalability?”

“I would monitor throughput, queue depth, average remediation execution time, MTTR, LLM latency, token consumption, cost per vulnerability, database latency, API error rates, retry frequency, and the number of workflows waiting for human approval.”

This connects nicely to your Agent Performance Evaluation section.

🔥 28. “How would you prove that your system scales?”

Excellent interviewer question.

“I would run load tests with increasing vulnerability volumes and measure throughput, latency, queue wait time, database performance, LLM usage, and remediation success rate. I would compare the results as the number of concurrent workflows increases and identify where throughput starts degrading.”

🔥 29. “What is your scalability strategy in one answer?”

Ye 1-minute answer prepare karo:

“Our scalability strategy is based on decoupling the different workloads. We keep expensive external intelligence synchronization in background cron jobs and use local PostgreSQL caches for runtime lookups. We use batch database operations instead of per-record queries, and asset inference can process independent assets concurrently. Most importantly, remediation workflows maintain independent state, so a workflow waiting for Human-in-the-Loop approval no longer blocks the next vulnerability. For larger workloads, we can horizontally scale remediation workers and run independent remediation agents concurrently. The persistence, API, and frontend layers can also be scaled independently based on their respective loads.”